

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Benyoucef BENKHEDDA University - Algiers 1



Faculty of Sciences
Department of Computer Science

End-of-Studies Project for the Award of the Master's Degree in Computer Science

Specialization: Networks and Embedded Systems

Topic

**From Reactive Supervision to Proactive Prediction: Multi-Agent
Architecture for Anticipating IT Resource Needs.**

Topic Proposed By

MRS. BENZAID CHAFIKA – *University of Oulu*
MR. ZERAOULIA KHALED – *USTHB*

Supervised By

MRS. ALI SAOUCHE NAZIHA – *University of Algiers 1*

Realized By

MR. YASSA RAFIK
MR. BOUABACHE YOUNES

Class of: 2025/2026

Acknowledgment

First and foremost, we extend our deepest gratitude to ALLAH, who has granted us the strength, knowledge, and patience to complete this journey.

His guidance is the cornerstone of our every endeavor.

We would also like to express our sincere gratitude to Ms. Ali Saoucha Naziha, our supervisor, for her valuable guidance, patience, and availability throughout the preparation of this thesis. Her confidence in this project, constant encouragement, constructive feedback, and expertise enabled us to successfully complete this work and significantly improve the quality of our thesis. We also acknowledge Ms. Benzaid Chafika and Mr. Zeraoulia Khaled for the work carried out with us during this project.

Dedications

I dedicate this work first and foremost to my parents, the two pillars of my existence. To my father, whose silent sacrifices and unwavering strength have always paved the way ahead of me, often before I even realized the weight of his efforts. To my mother, whose endless love, gentle guidance, and constant presence have been my safest refuge through every step of this journey. They are the foundation upon which I stand today and the reason I have been able to become who I am. No words can truly measure their devotion nor express the depth of my gratitude for a love that has never asked for anything in return.

To my family, especially my grandmothers and all my relatives, I offer my heartfelt thanks. Their faith in me has never wavered, even in moments when my own certainty faltered. Their encouragement has been a quiet light, always present, always guiding.

And to my friends, who walked beside me throughout this academic journey, I extend my sincere appreciation. In their company, I found relief in laughter, strength in support, and warmth in shared struggles. They turned difficult days into meaningful memories and made this path lighter than it would have been alone.

Bouabache Younes

Dedications

I do not really know how to write these words, so I will simply speak from the heart and say my piece. First of all, Alhamdulillah for granting me patience, easing my path, and guiding me to where I am today. Finishing this would not have been possible without His mercy and guidance. He willed it for me, and He carried me forward through every difficulty and every moment of doubt. For that, I can only say: Alhamdulillah.

To my parents, I do not even know where to begin. For as long as I can remember—and I'm tearing up as I type this—you have provided me with everything I needed, and never once have I desired something that you did not strive to give me. You sacrificed so much for me and never stopped believing in me. There are no words that could truly do justice to everything you have done, nor could they fully express the gratitude I feel. So I will simply say this: Mom and Dad, I love you, and thank you for everything. I only hope that, in this lifetime, I can repay even a fraction of all that you have given me.

To my little sister, whom I love so much: I wish you all the best in your life. I believe you are capable of achieving far more than I ever could, and I know you will. Little sis, I love you and I thank you for always supporting me, checking up on me, and helping me with everything I need.

To an extraordinary soul: my grandma, who always has a smile on her face and makes sure we keep ours too. She has loved me, spoiled me, and been there for me however she could or knew how, for as long as I can remember. I love you from the bottom of my heart, and I pray that Allah grants you a long life and never deprives us of your presence in ours.

I would also like to express my deepest gratitude to my uncle Maamar and his wife. Uncle, you have been like a second father to me. You have always been there for me in every possible way, never hesitating to support me, and you have left a lasting mark on my life through your unconditional love and constant presence. I love you dearly.

Last but not least, to my two brothers from different parents, Lotfi (lowlow) and Abd al Bari (bebe). Through everything we have been through together and all the late nights, we are still here. I just want to say I love you and I appreciate you.

Finally, I would like to thank myself for persevering and for never giving up, even during overwhelming times. I hope to continue growing and becoming a better version of myself. I'm exhausted and emotional as I write this, so it is what it is, and I'll go with it.

Rafik out peace.

YASSA Rafik

Abstract

The rapid adoption of cloud computing has intensified workload heterogeneity and dynamic resource demand, making efficient and timely resource management more critical than ever. Existing approaches are predominantly reactive and fail to fully exploit the elasticity of cloud environments because they respond to resource pressure only after thresholds are already breached and degradation has materialized, leading to potential performance issues and service-level agreement violations under rapidly shifting workloads.

This thesis explores the transition of cloud resource management from a reactive approach to a proactive one by designing and implementing a Multi-Agent System that distributes supervisory intelligence across autonomous agents that collectively monitor, forecast, and scale resources preemptively, replacing centralized reactive control with localized predictive decision-making.

A prototype was deployed and evaluated in a Kubernetes environment against a reactive baseline and under injected failure conditions. Results confirm that the proposed system successfully anticipates resource pressure and initiates corrective actions before critical saturation occurs, while maintaining operational continuity. This presents proactive Multi-agent systems as viable alternatives to reactive cloud resource management.

Keywords: Cloud Computing, Multi-Agent Systems, Resource Management, Autoscaling, MAPE-K, Forecasting, Sizing, Provisioning, Proactive Scaling, Kubernetes.

Résumé

L'adoption rapide du cloud computing a augmenté l'hétérogénéité des charges de travail et la dynamique de la demande en ressources, rendant la gestion efficace et en temps réel des ressources plus cruciale que jamais. Les approches existantes sont principalement réactives et ne parviennent pas à exploiter pleinement l'élasticité des environnements cloud, car elles ne réagissent à la pression sur les ressources qu'une fois les seuils déjà dépassés et la dégradation déjà constatée, ce qui peut entraîner des problèmes de performances et des violations des accords de niveau de service face à des charges de travail en évolution rapide.

Cette thèse explore la transition de la gestion des ressources cloud d'une approche réactive vers une approche proactive, en concevant et en mettant en œuvre un système multi-agents qui répartit l'intelligence de supervision entre des agents autonomes qui, collectivement, surveillent, prévoient et adaptent les ressources de manière préventive, remplaçant ainsi le contrôle réactif centralisé par une prise de décision prédictive décentralisée.

Un prototype a été déployé et évalué dans un environnement Kubernetes par rapport à une référence réactive et dans des conditions de défaillance simulées. Les résultats confirment que le système proposé anticipe avec succès la pression sur les ressources et déclenche des actions correctives avant que la saturation critique ne se produise, tout en maintenant la continuité opérationnelle. Cela présente les systèmes multi-agents proactifs comme des alternatives viables à la gestion réactive des ressources cloud.

Mots-clés: Informatique en nuage, systèmes multi-agents, gestion des ressources, mise à l'échelle automatique, MAPE-K, prévision, dimensionnement, provisionnement, mise à l'échelle proactive, Kubernetes.

الملخص

أدى الانتشار السريع للحوسبة السحابية إلى تضاعف تنوع أحمال العمل والطلب الديناميكي على الموارد، مما جعل إدارة الموارد بكفاءة وفي الوقت المناسب أكثر أهمية الآن من أي وقت مضى. تتميز الأساليب الحالية بطابعها التفاعلي في الغالب، وتفشل في استغلال مرونة البيئات السحابية استغلالاً كاملاً؛ لأنها لا تستجيب لضغط الموارد إلا بعد تجاوز المستويات المحددة بالفعل وظهور تدهور في الأداء، مما يؤدي إلى مشاكل محتملة في الأداء وانتهاكات لاتفاقيات مستوى الخدمة تحت أحمال عمل سريعة التغير.

تستكشف هذه الأطروحة انتقال إدارة موارد السحابة من أسلوب رد الفعل إلى أسلوب استباقي من خلال تصميم و تطبيق نظام متعدد الوكلاء يوزع الذكاء الرقابي على وكلاء مستقلين يقومون بشكل جماعي بمراقبة الموارد والتنبيه وضبطها بشكل استباقي، مما يستبدل التحكم المركزي القائم على رد الفعل باتخاذ قرارات تنبؤية محلية.

تم تطبيق وتقييم نموذج أولي في بيئة Kubernetes بالمقارنة مع أساس تفاعلي وتحت ظروف فشل محدثة. تؤكد النتائج أن النظام المقترح يتنبأ بنجاح بالضغط على الموارد ويبدأ في اتخاذ الإجراءات التصحيحية قبل الوصول إلى حالة التشبع الحرجة، مع الحفاظ على استمرارية العمل. هذا يقدم أنظمة الوكلاء المتعددين الاستباقية كبديل قابلة للتطبيق لإدارة الموارد السحابية.

الكلمات المفتاحية: الحوسبة السحابية، أنظمة الوكلاء المتعددين، إدارة الموارد، التوسع التلقائي، التنبؤ، تحديد الحجم، التزويد، التوسع الاستباقي، MAPE-K, Kubernetes.

Table of Contents

Table of Contents	i
List of Figures	v
List of Tables	vii
List of Abbreviations	viii
General Introduction	2
1 Theory and Problem Formalization	7
1.1 Cloud Computing	7
1.1.1 Characteristics	7
1.1.2 Service Models	8
1.1.3 Deployment Models	8
1.2 Cloud-Native Resource Management	8
1.2.1 Modern Cloud Infrastructure	9
1.2.2 Autoscaling: The Dominant Management Mechanism	11
1.2.3 Resource Management	13
1.2.4 Why the Topology Matters	15
1.3 Structural Limitations of Centralized Supervision Architectures	16
1.3.1 Sequential Pipeline Latency	16
1.3.2 Model Rigidity and Concept Drift	17
1.3.3 Fault Intolerance	18
1.3.4 Metric Overload and Operator Cognitive Load	19
1.3.5 The Architectural Root and Its Implications	20
1.4 Formalization of the Resource Management Pipeline	20
1.4.1 Workload Forecasting	21
1.4.2 Resource Sizing	22
1.4.3 Resource Provisioning	23
1.4.4 Error Propagation and Pipeline Coupling	24
1.4.5 Implications for Decentralized Design	25

1.5	Multi-Agent Systems: Positioning and Research Hypothesis	26
1.5.1	Structural Alignment with Centralized Limitations	27
1.5.2	Scope Constraints and Research Hypothesis	28
2	Conceptual Design	31
2.1	Introduction	31
2.2	Design evolution	31
2.3	General Assumptions	32
2.4	Global Architecture	32
2.5	MAS Architecture	33
2.5.1	Tier 2: The Operational Pipeline	34
2.5.2	Tier 1: Coordination and Oversight	35
2.5.3	Agent Distribution	36
2.5.4	Topology	37
2.5.5	Operational Flow	38
2.5.6	Agent Behavioral Architecture:	39
2.5.7	Policy Definition	39
2.5.8	Human Oversight Model	40
2.6	Communication	40
2.6.1	Communication Model	40
2.6.2	Communication Semantics	42
2.7	Knowledge Base	45
2.7.1	Conceptual Definition	45
2.7.2	Knowledge Base Domains	46
2.7.3	Concurrency Control	47
2.8	Agent Internal Design	48
2.8.1	Monitor and Resource Agent (MRA)	48
2.8.2	Predictive Forecasting Agent (PFA)	52
2.8.3	Resource Sizing Agent (RSA)	57
2.8.4	Domain Coordinator Agent (DCA)	62
2.8.5	Supervision Agent (SA)	67
2.9	Conclusion	72
3	Implementation and Deployment	75
3.1	Introduction	75
3.2	Experimental Environment and Infrastructure	75
3.2.1	Cluster Infrastructure	75
3.2.2	Workload Profile	77
3.2.3	Monitoring and Observability Layer	78
3.3	Implementation Technologies	79

3.3.1	Agent Runtime and Application Layer	79
3.3.2	Containerization and Cluster Orchestration	80
3.3.3	Monitoring and Observability	81
3.3.4	Forecasting and Inter-Agent Coordination	81
3.4	MAS Deployment Architecture	82
3.4.1	Namespace Organization	82
3.4.2	Messaging Infrastructure and MQTT Broker	84
3.4.3	Persistent Storage and Knowledge Base Realization	85
3.4.4	TimesFM Integration and Model Cache	86
3.4.5	Agent Deployment Topology	87
3.4.6	Local Image Management and Deployment Pipeline	88
3.4.7	Node Affinity and Placement Constraints	88
3.4.8	Scheduling Policies and Resource Constraints	89
3.5	Supervisory Policy and Parameterization	90
3.5.1	Monitoring and Pressure Classification Parameters	90
3.5.2	Forecast Horizon and Prediction Window	92
3.5.3	Scaling Decision Logic and Cooldown Policy	93
3.5.4	Forecast Uncertainty and Sizing Margins	95
3.6	Conclusion	97
4	Experimental Evaluation and Validation	99
4.1	Introduction	99
4.2	Evaluation Objectives	99
4.3	The Reactive Baseline: Definition and Justification	101
4.3.1	What the Baseline Is	101
4.3.2	Justification of the Baseline Choice	101
4.3.3	Reactive Characteristics of the HPA	102
4.4	Evaluation Methodology	103
4.4.1	Baseline Reactive Configuration	103
4.4.2	Proactive MAS configuration	106
4.4.3	Workload Protocol	109
4.4.4	Comparative Data Collection and alignment process	111
4.4.5	Metrics Definition	113
4.5	Experimental Run Execution	117
4.5.1	Run B: MAS Proactive Evaluation	117
4.5.2	Inter-Run Reset Procedure	121
4.5.3	Run A: Reactive HPA Baseline Evaluation	122
4.5.4	Metric Computation Script and Derivation	124
4.5.5	Experimental Results and Interpretation	125

4.6	DCA Supervisory Resilience Evaluation	129
4.6.1	MRA Failure Detection and Recovery (Mode B)	130
4.6.2	RSA Failure Detection and Recovery (Mode B*)	132
4.6.3	Immediate Escalation of Unresolvable Conditions (Mode C)	132
4.6.4	EMERGENCY_STOP Persistence and Recovery	133
4.6.5	Additional Resilience Observations	135
4.6.6	Evaluation Summary	136
4.7	Computational Cost and Deployment Overhead	137
4.7.1	Agent Runtime Footprint	137
4.7.2	Forecasting Overhead	137
4.7.3	Cost-Benefit Interpretation	138
4.8	Conclusion	138
	General Conclusion	140
	Bibliography	143

List of Figures

2.1	Global System Architecture	33
2.2	MAS Architecture	34
2.3	MAS Topology	37
2.4	MAS Topology Expansion	38
3.1	k3d Cluster Roles and Components	77
3.2	Proposed Resource Management Architecture	84
4.1	MAS Agent Scale-Down Before HPA Baseline.	104
4.2	Initial State Before HPA Deployment.	104
4.3	CPU Telemetry Verification.	104
4.4	HPA Manifest Creation.	105
4.5	HPA Baseline Deployment.	105
4.6	HPA Baseline Removal.	106
4.7	MAS Agent Readiness Verification.	106
4.8	HPA Disabled During MAS Evaluation.	106
4.9	Initial State Before MAS Evaluation.	107
4.10	PFA Forecast Trigger Log.	109
4.11	Workload Ramp Script.	111
4.12	MRA Status Confirmation.	118
4.13	RSA Status Confirmation.	118
4.14	PFA Readiness Confirmation.	119
4.15	MAS Workload Execution Log.	120
4.16	RSA Scaling Actions Log.	120
4.17	MAS Replica Count and CPU Utilization.	121
4.18	HPA Status Before Load Injection.	122
4.19	HPA Workload Execution Log.	123
4.20	HPA Replica Count and CPU Utilization.	123
4.21	HPA controller events.	124
4.22	M3: CRITICAL pressure duration results.	126
4.23	M4: WARNING-to-Scale Delay results.	127

4.24	M5: Replica-Seconds Overhead Results	128
4.25	M6: Forecast RMSE.	128
4.26	Simulation 1: DCA Recovery Log.	131
4.27	Simulation 1: Timeout and Escalation.	131
4.28	Simulation 2: DCA Recovery Log.	132
4.29	CAPACITY_OVERFLOW Escalation.	133
4.30	EMERGENCY_INSUFFICIENT Escalation.	133
4.31	EMERGENCY_STOP Processing.	134
4.32	EMERGENCY_STOP Persistence.	134
4.33	Simulation 5: PFA Recovery.	135

List of Tables

2.1	Global table of message types	44
2.2	Domain Access Across Agents	47
2.3	MRA Publications	50
2.4	MRA Knowledge Base Access	51
2.5	PFA Knowledge Base Access	56
2.6	RSA Knowledge Base Access	61
2.7	DCA Knowledge Base Access	66
2.8	SA Knowledge Base Access	71
3.1	Workload Configuration	77
3.2	Monitoring component placement and scheduling mechanism	79
3.3	Kubernetes namespace organization and architectural role of each partition	83
3.4	MQTT coordination topics and communication roles	85
3.5	Persistent storage resources, backends, and architectural purpose	86
3.6	MAS agent deployment granularity and operational scope	88
3.7	Scheduling constraints: type, scope, mechanism, and architectural purpose	90
4.1	MAS policy configuration active during the proactive evaluation experiment	107
4.2	Controlled workload ramp scenario used during evaluation	110
4.3	Comparative Evaluation Results	130

List of Abbreviations

AIOps	Artificial intelligence for IT operations
API	Application Programming Interface
BDI	Belief-Desire-Intention
CI/CD	Continuous Integration and Continuous Delivery/Deployment
CPU	Central Processing Unit
DCA	Domain Coordinator Agent
DEPAS	Decentralized Probabilistic Auto-Scaling,
DevOps	Development and IT Operations
FIPA-ACL	Foundation for Intelligent Physical Agents – Agent Communication Language
gRPC	Google Remote Procedure Call
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
IT	Information Technology
KQML	Knowledge Query and Manipulation Language
MAS	Multi-Agent System
MAPE-K	Monitor–Analyze–Plan–Execute with Knowledge
MRA	Monitor and Resource Agent
MQTT	Message Queuing Telemetry Transport
OS	Operating System
PFA	Predictive Forecasting Agent
PromQL	Prometheus Query Language
QoS	Quality of Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMSE	Root Mean Squared Error
RSA	Resource Sizing Agent
SA	Supervision Agent
SLA	Service Level Agreement
SLO	Service Level Objective
SOA	Service-Oriented Architecture

SPOF	Single Point of failure
SQLite	Structured Query Language Lite
SRE	Site Reliability Engineer
TCP	Transmission Control Protocol
TM-Forum	TeleManagement Forum
TimesFM	Time Series Foundation Model
UTC	Coordinated Universal Time
VM	Virtual Machine
VPA	Vertical Pod Autoscaler
WAL	Write Ahead Logging

General Introduction

General Introduction

The infrastructure that supports modern digital services has undergone a structural transformation over the past decade. Immense applications have been decomposed into constellations of independently deployable microservices. On-premise hardware has given way to elastic cloud resources managed through container orchestration platforms; deployment cycles that once spanned weeks now complete multiple times per day. Beneath this agility lies a persistent operational tension: the systems that manage and supervise cloud infrastructure have not evolved at the same pace as the systems they govern.

Problem Statement

The majority of production environments today still rely on a fundamentally reactive approach mostly due to its simplicity and ease of deployment and configuration. In this approach thresholds are configured, violations are detected, and corrective actions are triggered after degradation has already materialized. This reactive posture differs from a proactive approach in which corrective actions are anticipated rather than triggered after degradation. Instead of responding to violations, proactive systems attempt to predict future resource states and act within a time horizon that precedes saturation. In dynamic cloud environments, this anticipation window is critical: it determines whether the system can prevent violations or merely react to them. In fast-moving cloud workloads, the gap between when an anomaly begins and when the supervision layer responds is precisely the window in which service-level objectives (SLOs) and service-level agreements (SLAs) are violated.

While proactive supervision approaches have been extensively researched and proposed as alternatives, they remain far from universal adoption in production environments. This hesitation is based on the fact that proactive systems are fundamentally contingent on the accuracy of their forecasts because an inaccurate one can result in greater system degradation than the limitations of reactive approaches. This dependency on forecast precision becomes especially problematic given the centralized structure through which most supervision architectures operate, which is precisely what limits the reliability of those forecasts. Centralized monitoring controllers must aggregate telemetry from thousands of services, apply global prediction models trained on population-level data, and serialize

decision-making through a single pipeline before issuing any corrective action. Each step in this chain introduces latency. In cloud-native environments where CPU saturation can cascade across dependent services within seconds, the delay between anomaly onset and corrective action is not a tuning parameter; it is a structural property of the architecture.

This centralized design further undermines the reliability of both approaches through stale offline-trained models, a single point of failure under peak load, and global models that average away node-level behavioral patterns, all leading to a supervision and observability approach where effective intervention still depends on human expertise applied after the fact.

This thesis challenges both the reactive posture and the centralized structure widely adopted by supervision architectures. It investigates whether distributing supervisory intelligence across autonomous, locally adaptive agents can restructure the resource management cycle from a sequence of backward-looking corrections into a continuous process of anticipation and preemptive adjustment. The argument developed here is architectural in nature: the limitations of current approaches, whether reactive or proactive, are not primarily algorithmic nor solvable through isolated component fixes or improvements, but rather structural ones. They arise from concentrating prediction, decision-making, and remediation logic in a single centralized component that is both spatially and temporally removed from the infrastructure it governs. A multi-agent system (MAS), where each managed service is paired with a dedicated set of autonomous agents responsible for local metric collection, workload forecasting, and policy-bounded sizing and provisioning decisions, addresses these structural limitations at their root.

Research Questions

This thesis is organized around a central research question and two supporting sub-questions that bound its experimental scope.

RQ1 (Primary): Can a multi-agent architecture, in which autonomous agents perform localized workload forecasting and scaling decisions, provide a more scalable and responsive resource management layer than both reactive and centralized supervision in a cloud-native environment?

RQ2: To what extent does distributing the forecasting-sizing-provisioning pipeline across local agents reduce the reaction latency between workload change onset and corrective resource adjustment, relative to a centralized baseline?

RQ3: Under what conditions do the responsiveness and resource efficiency advantages of the MAS approach materialize, and where do its limitations become apparent?

These questions are intentionally scoped. This work does not attempt to replace production-grade observability platforms, nor does it address the full problem of root-cause

analysis in distributed systems. The focus is tightly defined: the architectural restructuring of the resource supervision loop, and the empirical assessment of whether decentralized agent-based control constitutes a viable alternative to reactive and centralized management.

Contributions

This thesis makes four primary contributions:

- **Structured analysis of reactive and centralized supervision limitations:** A systematic characterization of the limitations, namely the crippling latency to react to demand pressure, the difficulty of finding the most optimal thresholds, provisioning delays, sequential pipeline latency, global state dependency, prediction rigidity, and fault intolerance, that constrain the scalability and responsiveness of existing reactive and centralized supervision architectures. This analysis provides the formal motivation for the architectural shift investigated in the remainder of the thesis.
- **Conceptual mapping from supervision requirements to MAS properties:** A principled argument connecting the identified operational requirements of proactive resource management (low reaction latency, local adaptability, horizontal scalability, and fault resilience) to the structural properties of multi-agent systems, specifically autonomy, locality awareness, adaptive learning, and selective inter-agent coordination.
- **Design and implementation of a MAS-based supervision architecture:** A concrete architecture in which autonomous agents triads, each operating at the granularity of an individual managed service, perform localized resource observation, workload forecasting, and confidence-weighted scaling decisions through direct interaction with the container orchestration layer. The architecture replaces the reactive approach and the centralized prediction and control logic with a hierarchy of specialized agents whose coordination is governed by explicit policy authority and supervisory escalation mechanisms, enabling proactive scaling actions without dependence on thresholds nor a global system state.
- **Empirical evaluation under controlled workload conditions:** A structured experimental campaign validating the proactive, reactive, emergency, and supervisory coordination behaviors of the implemented architecture, together with a comparative evaluation against a native orchestrator-provided autoscaling baseline. The evaluation characterizes the conditions under which anticipatory scaling decisions can be made prior to threshold breach, documents the boundaries and limitations of the approach,

and provides a transparent account of experimental constraints encountered during validation.

Thesis Structure

The remainder of this thesis is organized into four chapters and a conclusion.

- **Chapter 1:** formalizes the theoretical and problem context.
It characterizes cloud-native resource management, develops a precise formalization of the forecasting-sizing-provisioning pipeline, analyzes the structural limitations of reactive approaches and centralized architectures, and positions the multi-agent approach within the autonomic computing literature before stating the formal research hypothesis.
- **Chapter 2:** presents the proposed architecture.
It introduces the full Multi-Agent System conceptual design, defines the responsibilities of the individual agents, describes the communication and coordination mechanisms, details the forecasting and scaling workflows, and explains the supervisory resilience model implemented within the architecture.
- **Chapter 3:** describes the implementation and deployment of the proposed system.
It presents the experimental environment, software stack, deployment topology, communication infrastructure, persistence mechanisms, forecasting integration, policy configuration, and the practical realization of the architecture.
- **Chapter 4:** presents the experimental evaluation and validation.
It defines the evaluation methodology, introduces the reactive HPA baseline, describes the workload protocol and measurement process, analyzes the proactive scaling behavior of the MAS, evaluates the supervisory resilience mechanisms, and discusses the computational cost and deployment overhead of the proposed approach.
- **Conclusion :** synthesizes the findings, acknowledges the limitations of the current implementation, and identifies concrete directions for future investigation.

Chapter 1

Theory and Problem Formalization

Chapter 1

Theory and Problem Formalization

1.1 Cloud Computing

Cloud computing as defined by the National Institute of Standards and Technology (NIST) is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction [1].

1.1.1 Characteristics

This cloud model is composed of five essential characteristics defined as follows [1]:

- **On-demand self-service:** consumers can provision computing resources automatically, with no human interaction required with the service provider.
- **Broad network access:** resources are accessible through the network and standard mechanisms, supporting diverse client platforms such as laptops and smartphones.
- **Resource pooling:** the provided resources are shared across multiple consumers using a multi-tenant model where physical and virtual resources are dynamically assigned based on demand.
- **Rapid elasticity:** resources can be scaled up or down quickly and often automatically to match the demand. The resources often appear virtually unlimited to the consumer.
- **Measured service:** resource usage is automatically monitored, controlled, and reported to ensure transparency and enable pay-per-use or charge-per-use billing.

1.1.2 Service Models

Three service models define the level of control and responsibility shared between the provider and consumer [1]:

- **Software as a Service (SaaS):** the consumer uses applications hosted on the cloud infrastructure of the provider. It is accessed via a browser or client interface with no control over the underlying infrastructure.
- **Platform as a Service (PaaS):** the consumer deploys their own applications using tools and languages supported by the provider. the consumer retains control over the deployed applications without managing the underlying infrastructure.
- **Infrastructure as a Service (IaaS):** the consumer provisions fundamental computing resources and can deploy and run any software while the provider manages the physical infrastructure.

1.1.3 Deployment Models

The deployment models presented below describe how cloud infrastructure is provisioned and who has access to it [1]:

- **Private cloud:** dedicated to a single organization and it may be managed internally or by a third party with the possibility of being hosted on or off premises.
- **Community cloud:** shared among organizations with common concerns. It can be managed by one or more of those organizations or a third party and may exist on or off premises.
- **Public cloud:** open to the general public and hosted on the premises of the cloud provider. It is operated by a business, academic or government entity.
- **Hybrid cloud:** a combination of two or more different cloud infrastructures that remain independent but are connected by a technology enabling data and application portability.

1.2 Cloud-Native Resource Management

Resource management in cloud-native environments presents a structurally distinct class of problems that differs from traditional systems administration in kind rather than in degree. The combined properties of container orchestration, elastic provisioning, and service-level accountability produce an environment in which the nature of the management challenge is shaped not only by its scale but also its architecture. Establishing what makes this

environment difficult to manage, and why existing approaches leave exploitable room for improvement, is a necessary foundation for any architectural proposal that claims to address any specific shortcomings or limitations.

1.2.1 Modern Cloud Infrastructure

A cloud infrastructure is the collection of hardware and software that enables the five essential characteristics of cloud computing [1].

Microservices

A microservice is a fine-grained, loosely coupled service representing a single, well-defined business capability within an application. Unlike monolithic applications where all functions are tightly grouped in a single deployable unit, a microservice is independently deployable and exposes its functionality through well-defined interfaces using lightweight communication mechanisms. In cloud-native environments, microservices are commonly packaged into containers that contain their processes, libraries, and dependencies to ensure reproducibility, transparency, and resource isolation [2].

The emergence of microservices is closely aligned with the rise of cloud computing, together forming the backbone of cloud-native application development [3]. The conceptual foundation of microservices can be traced to Service-Oriented Architecture (SOA), which advocates for software systems to be designed as independent, reusable, and automated services. Traditional monolithic architectures tightly couple and combine all components into a single program. This introduces critical scalability, maintainability, and availability constraints, which led the industry to seek an alternative. As a result, microservices emerged as the dominant architectural style, decomposing the monolith into loosely coupled services and enabling organizations to be more productive by facilitating DevOps and CI/CD pipelines [2]. Enterprise adoption has grown substantially, increasing from 33% in 2020 to 76% by 2024, also 72% of adopting organizations reporting improved application resilience [3].

Containerization technologies serve as an essential enabler of microservices architectures since they provide isolated and consistent runtime environments optimized for cloud infrastructure [3]. In practice, microservices are typically packed as container images using technologies such as Docker and published to an image registry, ensuring environmental consistency across development, testing, and production stages [2]. Docker currently leads the containerization landscape with 83.7% market share, and containerized microservices demonstrate 68% higher deployment frequency and 39% faster startup times relative to VM-based deployments [3]. By extension, service orchestration becomes container orchestration, where Kubernetes automates the deployment, coordination, and management of these containerized microservices at scale [2].

Containerization and Orchestration

Containerization is a function isolation mechanism that leverages the Linux kernel to isolate resources, creating containers as different processes in the host OS [2].

Modern infrastructures are increasingly relying on containers, whose adoption in production applications increased from 41% in 2023 to 56% in 2025 [4]. This makes containerization an important aspect of cloud computing, because unlike VMs, containers share the OS kernel with the host but maintain a dedicated userland filesystem containing only necessary binaries, libraries, and resource files, making them low-overhead and fast to deploy, which is the reason behind their big success in cloud-native scenarios [2]. Containers provide lightweight and self-sufficient execution environments that contain everything required to run a specific application, from its code to any necessary components or dependencies [5].

Orchestration is the automated configuration, management, and coordination of the interrelated microservices to build elastic and scalable functionalities [2]. This implies that a container orchestrator is the core component in a containerized cluster, where it is responsible for resource allocation, scheduling, monitoring, load distribution, and fault tolerance [6]. Currently, Kubernetes is the leading orchestrator used for production deployments, with 82% of container users running it in production [4]. The logical unit of deployment is called a pod, which consists of one or more containers that share a network namespace and storage volumes. Its resource requirements are governed by the "requests" parameter that guides scheduling decisions, and the "limits" parameter, which bounds resource consumption [7]. The scheduler within the orchestration system determines where to place each pod based on its declared resource requirements, affinity constraints, and real-time availability of the nodes to execute the requested workloads [7].

Workload Heterogeneity

Workload heterogeneity is a phenomenon that complicates the management of modern cloud environments. At any given moment, a production cluster hosts a mixture of workload types such as stateless HTTP services and stateful databases, where each has different execution times, resource usage patterns, resource types, and QoS requirements. [6]. Analysis of large production cluster systems has provided substantial evidence supporting this claim: Reiss et al. [8] have shown through an analysis of a 29-day production trace taken from Google that workloads are heterogeneous in the types of resources they consume, usage intensities, and the execution time required for task completion. Additionally, the trace uncovered substantial inefficiencies and fragmentation between allocated resources and their actual usage. The paper identifies workload heterogeneity as one of the main causes of this inefficiency.

Elastic Resource Management

Elasticity is an essential cloud characteristic that helps it deal with the fluctuating demand of these heterogeneous workloads. It defines the ability of a cloud application to dynamically acquire and release resources in response to changes in demand. To achieve this elasticity in an automated manner, auto-scaling systems are used to continuously and automatically adjust the amount of allocated resources horizontally or vertically to match the current or predicted demand [9], [10].

As a final note, resource management in cloud-native environments stands out as a highly dynamic and complex process shaped by continuous decisions regarding service scaling and resource allocation. These decisions determine how many service instances are maintained, how capacity is distributed according to current demand, and when resources are provisioned from or released back to the underlying infrastructure.

1.2.2 Autoscaling: The Dominant Management Mechanism

Overview

Autoscaling is the principal mechanism through which resource management is implemented in cloud environments. It can be defined as the dynamic adjustment of application resources in response to changing demand, with minimal human intervention, in order to balance SLA compliance and operational cost [9].

Autoscaling Decisions

The autoscaling decision-making process can be framed around three fundamental questions: when to scale, what to scale, and how much to scale.

- **When to scale?** This decision determines the timing of scaling. It can be reactive, based on current observations compared to pre-defined threshold values, or it can be proactive and anticipate future demand to compensate for sudden changes and provisioning delays [9].
- **What to scale?** This decision concerns the scaling dimension, which means whether resources are adjusted horizontally by adding or removing instances, or vertically by adjusting the allocated resources of existing instances [9].
- **How much to scale?** This decision specifies the magnitude of the scaling action required to satisfy the workload demand. It can be defined with static threshold-based rules that have a fixed step size, inferred from a model, or through optimization depending on the technique used [9].

Autoscaling Techniques

Based on the survey of Llorido-Botran et al. [9], on autoscaling techniques in cloud environments, five main classes can be identified: static threshold-based rules, control theory, reinforcement learning, queuing theory, and time series analysis. Threshold-based methods are widely adopted due to their simplicity. These methods trigger scaling actions when predefined conditions are met. Reinforcement learning approaches learn scaling policies using trial-and-error learning through interaction with the environment without requiring an explicit system model. Queuing-theory methods estimate system performance using analytical queue models, while control theory methods rely on controllers and system models to regulate resource allocation. Finally, time series analysis techniques forecast future resource demand to enable proactive scaling decisions.

The survey further discusses the limitations of each class, where it states that threshold-based rules depend heavily on carefully tuned thresholds and stable workload patterns; reinforcement learning may suffer from long training periods and large state spaces; queuing theory models are rigid and often assume simplified systems dynamics that may not hold under highly variable workloads; control theory relies on accurate systems models, so its performance depends strongly on how well the model captures the target system; while time series based methods are sensitive to prediction accuracy, which depends on the workload pattern, selected metrics, and parameter configuration [9].

As a final note, the survey identified that reactive autoscaling approaches may struggle to cope with fast and abrupt workload surges due to resource provisioning delays and their reliance on corrective actions. Consequently, proactive approaches that anticipate demand are better suited to maintain performance under rapidly changing workloads [9].

Challenges and Trade offs

The ever-growing number of workloads and their heterogeneity in cloud environments require autoscaling mechanisms to keep up with the changing demand while maintaining efficiency and reliability. This imposes challenges and trade-offs that autoscaling systems need to manage and balance. The fundamental trade-off is between cost efficiency and quality-of-service (QoS) or service-level agreement (SLA) compliance [9]. This trade-off requires finding a middle ground between these three problems: under-provisioning may reduce operational costs but it leads to SLA violations; over-provisioning ensures performance at the expense of wasted resources and added costs; and frequent and excessive scaling actions lead to oscillatory behavior that introduces instability and unexpected failures [9], [10].

1.2.3 Resource Management

Reactive Approach

Reactive auto-scaling refers to the allocation and deallocation of resources only when necessary, based on the current state of the system [10], [11]. It remains the most commonly adopted approach in the market [11], primarily because of its apparent simplicity and ease of deployment [9]. In practice, reactive auto-scaling operates by monitoring runtime metrics and triggering scaling actions once a predefined threshold is crossed. For example, when average CPU usage exceeds a set value, additional compute capacity is added [11]. Reactive auto-scaling is the most commonly used in the market where major cloud providers such as Amazon and Azure offer solutions that depend on user-defined rules and scaling actions when specific conditions are met [11], and platforms like Kubernetes and Docker Swarm share a similar reactive foundation [12].

The core limitation of this approach is inherent to its design: the system must wait until a threshold is breached before any action is taken [13]. Add to that the fact that resource provisioning is not instantaneous, which means the desired effect often arrives when it is too late [9]. This lag is particularly problematic under sudden or bursty workload spikes, where reactive scaling cannot respond fast enough to prevent performance degradation and QoS violations [9], [12].

Beyond the latency issue, threshold-based rules carry a secondary problem of choosing the right metrics and calibrating the thresholds correctly is difficult, and their effectiveness is tightly tied to workload behavior [9]. For applications with unpredictable or irregular patterns, this technique is simply not the right fit [9], [10]. Additionally, reactive scaling in microservice environments is further complicated by the delays introduced by complex inter-service call chains, making timely responses even harder to achieve [12].

For applications with regular and predictable workload changes, reactive approaches remain a reasonable choice due to their simplicity. However, for applications with dynamic or bursty workloads, their limitations make them insufficient on their own, which motivates the transition toward proactive and hybrid scaling strategies [9], [10].

Proactive Approach

Proactive auto-scaling takes a fundamentally different stance from reactive methods because instead of waiting for a threshold to be breached, it anticipates future resource needs and acts ahead of time [9], [12], [13]. It does so by leveraging historical workload data and applying predictive techniques to estimate the resources that will be required before the actual demand materializes [11]. This anticipation is what makes proactive scaling valuable because provisioning new resources always involves a non-trivial delay, acting in advance is often the only way to ensure continuity of service [9], [10].

The performance of any proactive system is directly tied to the accuracy of its work-

load prediction model [11]. When prediction is reliable, proactive scaling delivers clear advantages over reactive approaches: it ensures better resource allocation, reduces wastage and associated costs, and prevents sudden performance drops by maintaining QoS ahead of demand spikes [12]. It also addresses one of the central weaknesses of reactive systems, the inability to handle sudden workload fluctuations, by estimating the required resources before those fluctuations occur [11].

The Proactive approach requires decomposing the decision-making process into a structured pipeline. Three functional stages can be identified from the literature: workload forecasting, resource sizing, and provisioning [9], [10]. Based on the work of Saxena and Singh [14], we can describe workload forecasting as the process of extracting patterns from historical resource utilization data in order to predict future resource demand and improve provisioning decisions. The authors additionally state that prediction-based analysis should be considered as a cornerstone of effective workload management, workload sizing, and capacity planning, which a view supported by other works such as [15]. The next stage involves sizing resources to satisfy the demand based on the forecasting stage predictions. This produces sizing plans, which can be expressed either as the required number of replicas or as the resource allocation assigned to each instance. These plans are then passed to the provisioning stage as targets to apply to the infrastructure, whose execution is subject to provisioning delays. [9], [10], [16].

A clear sequential and asymmetric dependency can be observed between the three stages. Errors introduced at the forecasting stage can propagate into the sizing stage and affect the resulting sizing plans and resource targets. These targets are then enforced by the provisioning stage, which allows forecasting inaccuracies to directly influence the behavior of the underlying infrastructure. As a result, demand underestimation leads to under-provisioning, causing insufficient capacity and potential SLA violations, while demand overestimation leads to over-provisioning, causing excessive resource allocation, increased operational costs, and reduced resource efficiency. This shows that the sizing stage depends on the reliability of the demand forecast it receives, while the provisioning stage depends on the reliability of both preceding stages.

As consequence, capacity allocation must be designed to satisfy SLAs and SLOs under conditions of uncertain demand, since neither under-provisioning nor over-provisioning is acceptable at the operational scale of production cloud deployments.

The Autonomic Control loop MAPE-K

The Monitor–Analyze–Plan–Execute with Knowledge (MAPE-K) loop is a widely recognized feedback control framework for autonomic and self-adaptive systems introduced more than 20 years ago by Kephart et al. [17]. It represents a cyclical framework for managing adaptation in self-adaptive systems. It comprises four key phases (Monitor, Analyze, Plan, and Execute) that operate in conjunction with a shared knowledge base (K).

The loop enables a system to monitor its internal and external states, analyze observations, formulate and execute decisions to optimize its performance or adapt to changes [18].

This loop is the canonical feedback architecture used to model autonomic cloud resource management. It is considered both a reactive and proactive approach depending on how the Analyze and Plan stages are implemented. In this context, the Monitor phase collects telemetry data in real time; the Analyze phase determines if a scaling action is required based on the monitored information and it can operate in a proactive or reactive way depending on the implementation; the Plan phase constructs a resource-sizing model and produces a scaling plan; and the Execute phase provisions or deprovisions resources according to the resulting scaling decisions [10].

Gheibi et al. [19], in a systematic review of 109 publications on machine learning in MAPE-based self-adaptive systems, found that supervised learning dominates over unsupervised learning in learning tasks, with regression being the primary method for predicting and analyzing resource usage, and reinforcement learning being the primary method for updating and changing adaptation rules and policies. This observation is significant because it indicates that researchers have focused extensively on improving the performance of individual components within the pipeline, while the pipeline’s centralized, sequential, and global state dependent properties have largely remained unchanged.

1.2.4 Why the Topology Matters

The typical implementation of the MAPE-K framework is based on a centralized control approach in which an autonomic manager executes the major components of the MAPE-K loop: monitoring, analysis, and planning [17]. Brell Sanwouo et al. [18] argue that this classical structure introduces limitations in proactivity, scalability, and distributed decision-making, thereby motivating a reconsideration of the architecture on these grounds. Danny Weyns et al. [20] similarly identify scalability and coordination constraints as motivations for decentralized self-adaptation. These limitations become increasingly apparent in cloud computing environments characterized by high scheduling frequency and highly dynamic workload patterns [8], where the workload heterogeneity documented by Zhong et al. [6] and Reiss et al. [8] reduces the effectiveness of static global policies derived from aggregated workload observations.

The objective of this section is not to critique the specific limitations of any particular tool or platform. The current autoscalers and established MAPE-K controllers are well-engineered components that operate as intended within their design parameters. The issue is that these design parameters employ a specific set of architectural choices, such as reactive triggering, centralized prediction, and dependency on global state, whose limitations become increasingly significant as the scale, dynamism, and heterogeneity of systems continue to grow. Section 1.3 examines these constraints systematically in order

to develop the structural case for a decentralized alternative.

1.3 Structural Limitations of Centralized Supervision Architectures

The resource management pipeline is traditionally governed by a centralized architecture where a single logical controller collects telemetry from the cluster, coordinates analysis and scaling decisions through a shared control layer, and then dispatches them to the infrastructure [18]. This arrangement reflects an understandable engineering preference for global visibility, simpler coordination, and unified auditability. Yet the architectural properties that make centralized control convenient at small scale become structural liabilities as the system grows. Four specific limitations are examined here: sequential pipeline latency, model rigidity, fault intolerance, and metric overload. These limitations arise from the centralized topology itself rather than from any particular implementation choice.

1.3.1 Sequential Pipeline Latency

Centralized management architectures implement the MAPE-K feedback loop as a strictly sequential chain. Telemetry is collected, coordinated through a centralized management pipeline, analyzed and interpreted by a control component, passed to a planning module, and finally delivered to the infrastructure as a scaling directive. Brell Sanwouo et al. [18] identify this overall structure as a centralized, reactive, and sequential process for monitoring, analyzing, planning, and executing system adaptations, whose increasing complexity and dynamic nature introduce limitations in proactivity, scalability, and distributed decision-making capacity. This implies that each transition within this chain can introduce additional latency, because in systems that receive telemetry from large numbers of microservice instances, the aggregation process alone may introduce measurable delays before analysis can even begin.

The practical consequence is a consistent gap between the moment a workload deviation begins and the moment a corrective scaling action is initiated, because the autoscaling approach is reactive and scaling decisions take time [21]. Under steady-state conditions, this gap may remain tolerable because the system responds after the change happens and the service recovers before the next evaluation cycle. However, under bursty or rapidly shifting demand, the operational timeline becomes significantly compressed and constrained. Abdullah et al. [22] state that most of the existing state-of-the-art autoscaling methods are reactive and they describe them as “burst-oblivious” because they fail to detect and mitigate burstiness, which can lead to response-time SLO violations and oscillatory scaling decisions. Further work such as [23], [24] also supports a similar claim that under

rapidly increasing load, reactive scaling delays can allow response-time degradation and QoS violations to occur before additional capacity is provisioned.

This limitation cannot be addressed solely through threshold calibration, because latency is inherently embedded within the collect–aggregate–decide chain of events and therefore cannot be completely eliminated through threshold tuning alone. Reducing these latencies consequently requires adjustments to the chain itself.

1.3.2 Model Rigidity and Concept Drift

Many centralized systems of resource management use historical telemetry data collected from the heterogeneous workloads of the existing infrastructure to create global predictive models of future resource needs. This approach creates two primary constraints.

First, a centralized or global model captures generalizations about historical workloads at an aggregate or population level rather than an individual level. Consequently, the model is less capable of capturing workload-specific behavioral patterns and characteristics. In contrast, models trained on individual level data can better fit the unique dynamics of each service, reducing noise from unrelated data and improving forecasting accuracy.

As we demonstrated in Section 1.2, cloud-based workloads are very different from one another in the resource types they consume and the intensity of their consumption [8]. However, in a global model, metrics from all services are considered the same and contribute to the construction of a common predictive modeling scheme. Cortez et al. examine workload characterization at scale and present empirical evidence of strong per-subscription consistency in workload behavior, which can improve prediction accuracy [25]. This observation motivates the use of workload-specific prediction models.

Second, due to concept drift, centralized prediction models are typically updated through periodic retraining cycles. This creates a temporal gap between the assumptions encoded in the model and the actual behavior of the system.

Saxena et al. note that resource demands in cloud environments exhibit considerable variation over time, driven by the nature of submitted workloads and fluctuating user requests [26]. As workload patterns evolve, the accuracy of forecasting models trained on historical observations can degrade, reducing their effectiveness in predicting future demand. The inherently dynamic nature of cloud environments further increases this challenge, as unexpected changes in workload behavior can render previously accurate models less reliable. Consequently, predictive mechanisms must continuously adapt to changing conditions; otherwise, forecasting errors may result in over-provisioning or under-provisioning of resources and, ultimately, SLA violations [27], [28].

Model retraining and adaptation require additional monitoring, validation, and deployment time, and thus prediction quality may remain degraded during periods of workload evolution [29]. Given that workload characteristics in modern production environments

are in constant evolution, retraining periods may be required frequently enough to become a persistent operational condition rather than an exceptional event.

The research community is well aware of this problem and has invested substantially in online learning and self-updating model variants. However, from an architectural standpoint, these approaches address the symptom of prediction staleness while leaving its structural cause intact. Consistent with the broader critique of centralized MAPE-K architectures presented by Brell Sanwouo et al. [18], the limitation may therefore reside not only in model quality but also in the topology of the control architecture itself.

1.3.3 Fault Intolerance

A centralized controller can introduce single points of failure within the control plane. If the monitoring aggregator, prediction engine, or planning component becomes unavailable due to software fault, resource exhaustion, or network issues, all proactive supervision methods would no longer be available. The infrastructure continues to operate, but the management layer responsible for anticipating demand changes and issuing preemptive scaling actions would be absent. From an architectural and structural standpoint, concentrating the supervision functions in a single component increases the possibility that the management layer itself may experience resource contention or coordination delays under peak operational conditions, which is exactly when the layer becomes the most important.

These claims are well supported and repeatedly documented in the cloud literature. Straesser et al. describe conventional autoscalers as centralized controllers that form a single point of failure (SPOF) and state that autoscalers are tightly coupled to global monitoring systems. These systems are also identified as SPOFs, which impair the ability of autoscalers to issue correct scaling actions when the monitoring pipeline faces faults or data loss [30]. In their quest for resource efficiency under a Kubernetes environment, Ahmed et al. report that existing HPAs have architectural limitations such as a SPOF in centralized designs and the lack of scaling coordination in decentralized designs, which affect effective scaling under fluctuating workloads [31]. Their follow-up work on SecureSmart HPA explicitly targets resilience against resource disruptions caused by faults, cyberattacks, or maintenance. They note that traditional HPAs do not handle such fault scenarios well, which leads to degraded autoscaling behavior [32]. Calcavecchia et al. propose DEPAS, a decentralized probabilistic auto-scaling algorithm and state that centralized management solutions are not scalable and introduce a single points of failure, especially in large-scale cloud settings [33]. In a follow-up work, Caprarescu and Petcu extend this fault-tolerance motivation by adapting DEPAS to heterogeneous systems [34].

In cloud-native environments, partial failures are expected outcomes that this type of platform is designed to handle gracefully at the workload level. Even though replicated centralized controllers improve availability and architectural fault tolerance, they do not

address the sequential latency and global-model limitations discussed above, since they retain the centralized decision-making constraint identified in this paper.

1.3.4 Metric Overload and Operator Cognitive Load

The fourth limitation is not architectural in nature but still has a significant impact. Cloud-native systems generate vast volumes of telemetry (alerts, metrics, logs, and traces), often produced by fine-grained operational units. In principle, having such extensive telemetry data theoretically increases visibility; however, in practice it also creates significant operational challenges when threshold-based mechanisms are used to examine large metric spaces. [35]

Depending on how aggressively thresholds are configured, such mechanisms are prone either to producing excessive false alarms or to missing important anomalies. Within the AIOps literature, this phenomenon is frequently discussed as a consequence of applying relatively static detection rules to highly dynamic and heterogeneous telemetry environments, increasing the operational burden on practitioners [35] .

Beyond alerting, the cognitive burden placed on operators by large-scale observability solutions is immense, frequently leading to extended and intense working hours as well as emergency overtime shifts. This induces fatigue, which leads to lack of focus, lower efficiency, and higher error rates. As noted by Notaro et al. [36], identifying failure causes in modern cloud environments requires correlating various operational data sources from different dynamic and distributed environments. While important advancements have been made in the development of AIOps tools, most effective diagnosis and root cause analysis still relies heavily on human operational knowledge and interpretation, especially in complex production settings where capturing diagnostic knowledge is challenging. As a consequence, this dependency on human reasoning introduces yet another form of sequential bottleneck, because the responsiveness of human-dependent management loops will only be as fast as the operator can interpret and diagnose the behavior they are observing. In addition, the volume of telemetry received from modern cloud implementations exceeds any individual's or team's capability to process without extensive automation.

Centralized observability architectures can lead to both limitations mentioned above. As metric cardinality grows due to the number of monitored services, sending all of that telemetry data to one aggregation point will expose the central pipeline to operational noise that it cannot filter out effectively. Operators will then be swamped and required to comprehend dashboards that provide a single view of the telemetry gathered from heterogeneous workloads.

Decentralized architectures can address this at the source. Local agents filter, aggregate, and act on their own service's telemetry before sending any signals to the global layer. This reduces both the volume of data in transit and the number of events that require

human attention, because only the events that cross service boundaries are escalated.

1.3.5 The Architectural Root and Its Implications

The four limitations examined above share a common structural origin. Sequential pipeline latency is a consequence of routing decisions through a logically centralized sequence. model rigidity results from using broadly aggregated telemetry to make predictions and train predictive models. Fault intolerance occurs when critical functions are concentrated within a small number of tightly-coupled control components. Metric overload and cognitive load result from attempting to aggregate and process telemetry at a scale that cannot be efficiently handled by any individual component or engineering team.

The evidence in the literature is consistent on this point: incremental improvements to individual components of the centralized pipeline, such as improving accuracy in detecting anomalies, increasing the speed of aggregation, or creating more sophisticated alerting rules, cannot fully resolve limitations that originate from the architecture itself instead of any individual component [18], [19]. To address these architectural constraints, Brell Sanwouo et al. [18] suggest that we need to move away from the traditional MAPE-K loop in order to develop a new approach that improves the overall design and architectural effectiveness.

This suggests that we need to move towards a new architecture that lays the foundation for a coherent system with distributed decision-making, management, and prediction all at a fine-grained level of the infrastructure, while ensuring a strong fault tolerance.

1.4 Formalization of the Resource Management Pipeline

The limitations identified in Section 1.3 are directly linked to the structure of the resource management pipeline itself and can be analyzed through its structural properties. This analysis serves two purposes. First, it shows that the limitations are not simply caused by specific implementations but rather inherited from centralized architectures. This means they cannot be fully eliminated by algorithmic or implementation-level improvements. Second, it provides the baseline against which any credible architectural alternative must be evaluated. As a consequence, a decentralized system is only a valid alternative if it can still perform the same essential functions (forecasting, sizing, and provisioning) while meeting the same operational requirements and performance expectations with fewer limitations.

1.4.1 Workload Forecasting

Workload forecasting constitutes the predictive foundation upon which proactive autoscaling systems are built. Instead of reacting to resource saturation once it has already manifested, a forecast-driven architecture anticipates future demand and provides the provisioning layer with sufficient lead time to initiate scaling actions before any performance degradation occurs [9], [24]. The effectiveness of this proactive approach depends on the ability of the forecasting component to estimate future workload demand with sufficient lead time for provisioning actions to be completed before SLA violations occur [9], [24].

Most of the research on workload forecasting in the cloud considers the forecasting task to be a time-series regression problem where a model trained on historical data predicts future workload based on a fixed-length window of past measurements [15]. Masdari & Khoshnevis describe the major classes of forecasting methods used in this area and classify them into three categories: statistical methods, machine learning based models, and hybrid approaches. Additionally, they define input window length and forecast horizon as key design parameters that influence the prediction accuracy and practical usefulness of the forecasted estimates [15]. The trade-off between forecast accuracy and provision lead time is tied closely to the length of the forecast horizon. A shorter forecast horizon means more accurate forecasts, while a longer one will allow provisioning actions to happen sooner, but it will also compound the prediction error for each successive forecast [15]. This trade-off is a system constraint and not just a simple model issue. In order to provision resources before the predicted peak load, the forecasting horizon must be sufficiently long to exceed the effective provision latencies of the target infrastructure [9].

The complexity of the forecasting problem may vary significantly between different deployments. Univariate forecasting approaches model each resource metric or dimension (i.e., CPU utilization) in isolation, while multivariate forecasting approaches typically model the correlated resource dimensions (i.e., CPU and memory utilization) based on the assumption that inter-dimension dependencies have a larger predictive signal [37]. For example, Patel and Bedi proposed a multivariate attention-based architecture that utilizes an attention mechanism to capture the interdimensional correlations. They found that the multivariate approach outperformed the univariate forecasting models in terms of forecast error, but it required greater computation cost [37]. On the other hand, if there is not enough density or correlation of the historical data across all of the input dimensions, then the added complexity of the multivariate forecasting model may not translate into proportional accuracy gains [15].

The second aspect of forecasting focuses on how to handle non-stationary workload patterns. Cloud workloads have temporal dependencies over different time scales, often exhibiting daily and weekly periodicity under normal conditions yet remaining subject to sudden changes in their distribution due to application events, changes in user behavior, or

perturbations in the infrastructure [8], [25]. Forecasting models trained on historical regimes may underperform when the underlying data-generating process evolves, a phenomenon formalized in the machine learning literature as concept drift [38]. This imposes a practical requirement on forecasting components deployed in production settings: the model must either adapt continuously to distributional shifts or be complemented by mechanisms that detect and respond to forecast degradation [15], [38].

For the purposes of this thesis, the forecasting stage is treated as a component that produces at each inference cycle a forecast of future demand over the specified horizon and with an associated representation of uncertainty around the forecast. Predictive models are known to be imperfect. As a result, the downstream sizing and provisioning logic must consider this uncertainty rather than treating the forecast output as the perfect and precise demand that will materialize.

1.4.2 Resource Sizing

Sizing represents the stage where demand forecasts are translated into concrete resource-allocation targets. While the forecasting stage operates in the demand domain by producing estimates of future resource consumption and request arrival rates, sizing operates in the resource domain: its output is a capacity specification expressed as a number of service replicas, CPU cores, or quantity of memory that can accommodate predicted load requirements while satisfying the performance constraints defined by service level agreements [39]. Furman et al. [39] represent capacity allocation for cloud services as a constrained optimization problem where the objective function is to minimize the total amount of provisioned capacity while still meeting probabilistic service level obligations. This means that at every time t over the planning horizon T and for every job class i , we must meet the following constraint:

$$P(D_i(t, s(t)) > \tau_i) \leq \alpha_i \quad \forall t \in [0, T], \forall i \in \mathcal{I}$$

where the vector $s(t)$ represents the time-varying provisioned capacity for each resource type at time t . $D_i(t, s(t))$ denotes the service delay experienced by a class- i job arriving at time t under the provisioned capacity $s(t)$. The threshold τ_i is the maximum acceptable delay for class i (it is pre-defined based on the SLA), and α_i is the maximum tolerable probability that this delay is exceeded. This formulation is in the context of cloud workloads characterized by parallel processing, batch arrivals, and different service requirements. It has been validated using operational data obtained from Huawei Cloud [39].

Combined together, the points above highlight a limitation of reactive strategies that is directly relevant to the argument of this thesis. In a reactive system, capacity adjustments are triggered only after demand changes have been detected in real-time metrics. However, the constraint above requires capacity to be sufficient for the future load that will be

experienced, not the load that has already been detected. Satisfying this constraint reactively requires the system to respond faster than the demand change can escalate into SLA violations. This cannot be guaranteed with a reactive approach, especially with a non-negligible provisioning latency on top.

Revising sizing decisions in containerized environments incurs non-negligible operational overhead. Rzadca et al. [40] note that vertical scaling operations in large-scale cluster environments carry considerable operational risks and costs while horizontal scaling changes require stabilization policies to prevent abrupt oscillation. Autopilot, for instance, uses deferred downscaling policies with two to three days delay windows in order to mitigate this instability. More broadly, Lin et al. [41] explore the costs associated with switching capacity configurations in the context of dynamic right-sizing for server-based data centers. They establish that the cost of flipping between capacity configurations includes energy consumption, migration overhead, wear-and-tear, and the associated operational cost. Taking on such a change can cost an estimated 1 hour of server runtime per flip. This analysis provides motivation for using a lazy provisioning approach, as making frequent configuration changes will accumulate switching costs that exceed any savings realized via an accurate capacity tracking process, unless the variation in workload is both significant and predictable enough to justify implementing the change.

This argument about switching costs can also be applied to containerized deployments since there are similar switching costs for HPA/VPA, eviction cascades, and initialization overhead. As a consequence, a well-designed sizing stage should maintain a balance between responding to changes in demand and reducing the costs associated with frequent reconfigurations.

1.4.3 Resource Provisioning

Provisioning is the execution-layer process that implements the sizing plan through the acquisition or release of compute resources. The relationship between provisioning and sizing is a plan-to-action translation, where sizing determines what capacity is required and in what quantity, and provisioning determines when and how that capacity is obtained from the underlying infrastructure.

Provisioning latency is the most significant operational challenge at this stage. It is the time delay between initiating a scaling directive and having the capacity provisioned and available. For containerized workloads, this provisioning latency encompasses the time taken to retrieve an image from a registry, create a container, start up the application, and integrate it into the running service. Lorigo-Botran et al. [9] state that provisioning actions in cloud environments may require several minutes before becoming fully operational. This is confirmed by Mao and Humphrey [16] for virtual machine startup times in cloud environments. When designing the forecasting phase of an automated scaling system,

it is important to consider provisioning latency and the manner in which it limits the ability to accurately predict demand and provision resources. A scaling system that issues a provisioning directive at the moment demand is observed may receive the resulting capacity after the demand peak has already caused queuing delay to accumulate and SLA violations to materialize. This concept of a lack of timely provisioning is the primary motivation for the use of proactive auto-scaling methodologies, as noted in the research conducted by Lorigo-Botran et al. [9]. In order for an automated scaling system to be categorized as proactive, it must forecast the expected demand and issue a provisioning request with enough lead time to address the demand before it occurs. This imposes a design requirement that the forecasting horizon must at least extend as far into the future as the infrastructure’s provisioning latency, so that scaling actions can be initiated before the demand they are intended to serve has materialized [9], [10].

This dependency between forecasting horizon and provisioning latency is the link that connects the first and third stages of the pipeline. In reactive and centralized management architectures, this dependency has an important impact. A system that computes a single global forecast with a uniform planning horizon may struggle to adapt that horizon efficiently to the provisioning characteristics of individual services, which may differ substantially. By contrast, a decentralized MAS architecture allows each service agent to maintain a local forecasting model informed by its provisioning latency and observed service behavior. This enables per-service forecasting horizons, improved accuracy, and reduced global coordination overhead.

1.4.4 Error Propagation and Pipeline Coupling

The three stages of a proactive autoscaling pipeline identified in the previous sections are not independent stages. They represent an interconnected sequence of events that occur in a sequential order and the errors introduced at one stage can have a negative impact on the subsequent stages, which potentially increases downstream provisioning inaccuracies and operational instability [23], [42].

A key source of pipeline error occurs during the forecasting stage because it is the stage where the future demand is predicted. This demand differs from the actual one to some extent, which directly affects the inputs to the capacity sizing stage. This means forecasting inaccuracies can impact sizing and provisioning decisions. Chen and Li [42] provide evidence from an empirical study showing that workload predictions directly affect capacity allocation decisions and resource utilization. They also demonstrate the effects of forecasting inaccuracies throughout the pipeline, affecting both operational costs and SLA outcomes. When these results are considered together, underestimation errors in the forecasting stage appear to produce more severe downstream consequences than equivalent overestimation errors. This impacts how sizing decisions should be taken because a proper

sizing mechanism requires a safety margin above the predicted demand estimate. This margin should be calibrated based on the uncertainty of the forecast.

Another source of coupling originates from the feedback loop that closes the pipeline by connecting the final stage back to the forecasting stage. Operational metrics collected during system execution are used to update the forecasting model. This means that inaccurate forecasts may indirectly influence future forecasts through the metrics generated by their associated provisioning decisions, unless suitable error-correction mechanisms are in place. This problem can be considered and categorized as a concept drift problem. Lu et al. [38] describe concept drift as the phenomenon that occurs when historical training data becomes misaligned with the current data-generating process, which often causes model degradation. In an autoscaling context, this occurs when provisioning decisions affect the operational metrics that are collected and then fed back into the forecast model, which may reinforce forecasting errors instead of correcting them. Qu et al. [10] classify autoscalers as non-adaptive, self-adaptive, and self-adaptive with controller-switching based on their level of adaptability. They argue that adaptability is important due to unexpected changes in workload characteristics, which motivates the need for autoscalers to adapt their models and settings to better fit the changing conditions.

To the best of our knowledge, no current method successfully forecasts accurately, sizes correctly, and provisions on time under realistic workloads. Therefore, we should consider the three stages of an autoscaler to be a single unified system, not just a composition of independently optimized components.

1.4.5 Implications for Decentralized Design

The analysis developed in preceding subsections introduces two observations that are directly connected to the architectural direction applied in Chapter 2. The first concern is workload heterogeneity and the limits of global forecasting models. The accuracy of forecasting is critical, as prediction errors propagate throughout the system, influencing capacity sizing decisions and ultimately affecting both provisioning efficiency and SLA/SLO compliance [23], [42].

A management architecture that applies a single global forecasting model across a heterogeneous set of workloads must deal with the diversity observed in production cloud environments. Reiss et al. showed that workloads differ significantly in the types of resources they consume, usage intensities, and execution durations [8]. Similarly, Saxena and Singh found that workload characteristics vary widely, and this heterogeneity reduces the accuracy of global forecasting models that are applied across an entire fleet [14]. Consistent with these findings, Masdari and Khoshnevis, in their survey of workload forecasting, concluded that there is no single forecasting technique that can provide consistently accurate predictions across the diverse workload types found in production

cloud environments [15]. Collectively, this evidence indicates that a global forecasting model may not be appropriate for a heterogeneous fleet of services.

The second observation is related to the provisioning latency constraint. Proactive autoscaling requires forecasts to be generated sufficiently in advance to account for the time needed to provision resources before the anticipated demand actually materializes [9], [22]. This timing requirement highlights a fundamental dependency between the forecasting horizon and the provisioning latency of available resources in the environment. Deochake et al. reinforce the constraint by showing that startup and shutdown times can differ dramatically across deployment environments. In their experiments, Docker containers started and stopped within milliseconds, while VMs required tens of seconds [43]. As a result, the required lead time to produce an effective forecast depends on the workload type and the runtime platform.

In this case, a decentralized architecture is an appealing alternative to consider. Autonomous agents can maintain local state and act based on locally perceived information and available resources [44], [45]. This motivates the idea of implementing the autoscaling pipeline as a distributed multi-agent system, where each service is managed by a dedicated agent that maintains a local forecasting model, local knowledge of its own workload behavior, and provisioning latency. This design would enable efficient and adaptive service-specific management and also reduce the need for centralized coordination, since most decisions are handled locally and only exceptional conditions require higher-level intervention.

1.5 Multi-Agent Systems: Positioning and Research Hypothesis

The preceding sections lead to the conclusion that an architectural change is required, or at least warrants further exploration. Although proactive resource management in cloud environments already exists and is extensively studied, its effectiveness and full potential are still mainly constrained by the way it is integrated within the centralized architecture and the workload-specific characteristics. The limitations and challenges identified throughout the chapter stem primarily from the centralized management of the forecasting-sizing-provisioning chain, which introduces latency and reduces sensitivity to local workload dynamics. Consequently, a more suitable architectural approach is to distribute intelligence closer to individual services through a decentralized multi-agent system that would enable a more responsive and adaptive resource management approach.

1.5.1 Structural Alignment with Centralized Limitations

Each of the four structural limitations identified in Section 1.3 can be related to architectural properties commonly associated with Multi-Agent Systems, suggesting mechanisms that could mitigate these limitations.

Sequential latency: in centralized management pipelines results from routing monitoring data and decision logic through shared aggregation and planning components. Weyns et al. [46] demonstrate that centralized coordination mechanisms become increasingly difficult to scale in large and highly dynamic environments because they depend on globally aggregated information and centralized computation. In a MAS, each agent executes its local observation-analysis-decision loop independently based on locally available information [44]. Because agents primarily communicate with neighboring agents rather than relying on a single aggregation point, communication overhead can be reduced through localized interactions among agents as the number of agents increases [44]. This distributed structure, enabled by autonomy and locality of agents is a good answer to sequential latency. It reduces dependency on centralized decision-making and improves scalability in dynamic, large-scale cloud environments.

Global model rigidity: arises when predictive models are trained with telemetry data collected by a central system [38]. Gheibi et al. distinguish batch learning from online learning in self-adaptive systems, where online learning is defined as incrementally updating models with the arrival of new data from the environment, while batch learning models are only updated at the end of a scheduled retraining cycle [19]. In decentralized MAS, agents build and maintain local predictive models that are adapted to the specific service workloads they manage, which improves model accuracy. By constantly adapting the local predictive models through online learning, the dependency on global retraining cycles is eliminated. As a result of this online adaptation, the time between workload drift and model adaptation is reduced from the old periodic retraining cycles to the update frequency of each agent’s local learning loop.

Fault intolerance: in centralized architectures arises from the presence of a central coordination component that constitutes a single point of failure [44]. By contrast, the distributed nature of MAS improves reliability because tasks handled by failed agents can be reassigned to other agents within the system [44]. Failures in decentralized MAS architectures are more likely to produce partial degradation of management functionality rather than complete loss of capabilities.

Metric overload and cognitive load: emerge from attempting to process population-scale telemetry through a single analysis pipeline. Distributing analysis across agents

reduces the volume of telemetry processed by human operators or any single component and improves scalability by limiting communication overhead to local interactions among neighboring agents [44]. In a MAS, local agents can filter and aggregate monitoring signals before propagating only coordination-relevant events to the global supervisory layer. This design reduces the volume of information exposed to human operators, shifting attention from continuous telemetry inspection toward exception-driven supervision.

The relationship between the identified limitations and MAS architectural properties is based on structural correspondences between the limitations and the organizational properties of decentralized agent systems: local autonomy addresses sequential latency; decentralized state addresses global model rigidity; distributed fault isolation addresses fault intolerance; and distributed telemetry processing and selective information sharing address metric overload and cognitive load. This correspondence provides the principled basis for the architectural proposal in Chapter 2 and distinguishes this work from approaches that apply machine learning improvements within an unchanged centralized topology.

1.5.2 Scope Constraints and Research Hypothesis

A MAS architecture addresses the structural limitations identified in Sections 1.3 and 1.4, but it does not address all challenges in cloud resource management simultaneously. This thesis does not aim to replace production-grade observability platforms, solve the general problem of distributed root-cause analysis, or provide a comprehensive AIOps replacement. The experimental scope is deliberately bounded to the resource management and supervision loop: the cycle through which workload signals are translated into provisioning actions, and the associated latency and accuracy of this cycle under realistic cloud workload profiles.

Within this scope, and grounded in the formal pipeline structure of Section 1.4 and the structural alignment argument above, the research hypothesis of this thesis is stated as follows:

Research Hypothesis. A multi-agent supervision architecture, in which autonomous agents perform localized workload forecasting and resource sizing at the service level and interact with the container orchestration layer to trigger proactive scaling actions, is expected to exhibit lower reaction latency and higher SLA/SLO compliance under dynamic workload profiles than a centralized, threshold-based reactive baseline operating over the same infrastructure, while reducing reliance on centralized global control and shared prediction models.

This hypothesis is addressed through three research questions stated in the General Introduction (RQ1–RQ3). To validate it, the next chapter presents the full conceptual

design of a multi-agent supervision architecture. This includes the architecture, topology, agent roles, communication mechanisms, among other elements.

Chapter 2

Conceptual Design

Chapter 2

Conceptual Design

2.1 Introduction

In Chapter 1 we succeeded to establish that the limitations of centralized supervision architectures are structural rather than algorithmic, and that distributing the forecasting-sizing-provisioning pipeline across locally autonomous agents addresses those limitations at their root. This chapter translates that argument into a concrete system, presenting the design of a multi-agent supervision architecture deployed over a cloud native infrastructure.

2.2 Design evolution

The architecture we present in this chapter is the outcome of four successive design iterations. The first retained the centralized baseline as the primary system and introduced MAS components as incremental extensions, which was a direct contradiction of the thesis’s core argument that centralization is the structural problem, not a foundation to build upon. The second iteration discarded the baseline entirely in favor of a simpler agent-native design, which moved in the right direction but left agent responsibilities unevenly defined, with coordination gaps that had no clean resolution. In the third, our work addressed those gaps through a more rigorous decomposition, but it was over engineered with too many variable dependencies. It was impractical to execute reliably within the hardware and time constraints of the work. The final design was drawn from the strongest properties of the two preceding iterations: the structural clarity of the third and the directness of the second, in addition to a small number of deliberate new decisions. The result is an architecture that is realistic to implement, internally consistent, and sufficiently expressive to test the research hypothesis under realistic conditions. The remainder of this chapter describes it in full.

2.3 General Assumptions

The objective of this section is to outline the assumptions adopted to set the scope of the proposed architecture and to define its operating perimeter. These assumptions establish the conditions the architecture is examined in and help focus the discussion on its intended design objectives.

The defined assumptions are presented as follows:

- **Secure operating environment:** The system is assumed to operate in a fully secure environment, free from malicious threats, vulnerabilities, and any exploitation attempts.
- **Reliable network conditions:** Communication between system components is assumed to take place over a stable and reliable network environment.
- **Optimal parameter configuration:** All thresholds, limits, timing values, and operational parameters are assumed to be appropriately defined.

2.4 Global Architecture

The system is organized into four architectural components, each with a distinct and non-overlapping responsibility.

- **The multi-agent system:** constitutes the intelligence layer of the architecture, executing a distributed pipeline that spans infrastructure observation, proactive demand forecasting, autonomous scaling, failure recovery, and human oversight mediation. Decomposing this pipeline across specialized, locally autonomous agents is a direct structural response to the limitations established in Chapter 1.
- **Communication Framework:** mediates all inter-agent data flow using asynchronous messaging, ensuring that agents are loosely coupled. This prevents any blockage between the agents while in coordination. The mechanism is also well aligned with the distributed and autonomous nature of the agents.
- **The Knowledge Base:** maintains shared and durable state across the system (operator policies, infrastructure topology, metric history, audits...). Because agents are distributed, a persistent shared store is necessary to preserve the continuity of context across restarts, failures, and deliberation cycles.
- **The cloud native infrastructure:** is the infrastructure that the system is designed to manage and supervise. It is both the source of the metrics agents reason over and the execution surface through which scaling decisions are enforced, making it the direct target of the improvements we aim to demonstrate in this work.

Figure 2.1 illustrates the four architectural components of the proposed system and their structural relationships :

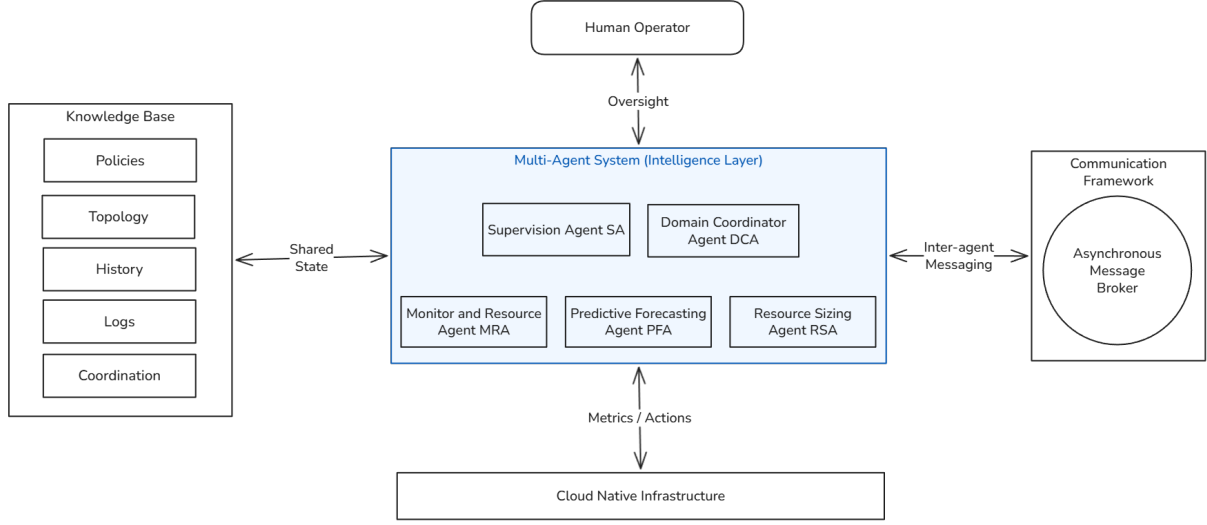


Figure 2.1: Global System Architecture

2.5 MAS Architecture

The proposed multi-agent supervision system is built on a cohesive and distributed architecture. This architecture is designed to address the limitations identified in Chapter 1; some of them are resolved and the impact of others is reduced. Conceptually, this design is a hybrid derivation of the standard forecasting-sizing-provisioning pipeline and the traditional MAPE-K (Monitor–Analyze–Plan–Execute with Knowledge) autonomic control loop[47],[48].

Instead of encapsulating the entire MAPE-K loop within a single centralized controller, the architecture fractures the loop and distributes its phases across specialized, locally autonomous agents. The Monitor and Resource Agent (MRA) assumes the *Monitor* role by extracting continuous telemetry. The *Analyze* phase is separated into a dedicated forecasting component, handled exclusively by the Predictive Forecasting Agent (PFA). Finally, the *Plan* and *Execute* phases (corresponding to the sizing and provisioning steps of the resource management chain) are regrouped inside one agent named the Resource Sizing Agent (RSA).

Spreading this hybrid control chain over an asynchronous communication framework and sharing persistent state through a central Knowledge Base gives structural decomposition and fault tolerance to the system. The MAS is organized into two logical tiers that separate operational execution from coordination and oversight. This separation ensures that the former remains structurally independent of the latter’s operational availability.

Figure 2.2 illustrates the internal structure of the multi-agent system across its two tiers, the agents composing each tier, and the interaction between them.

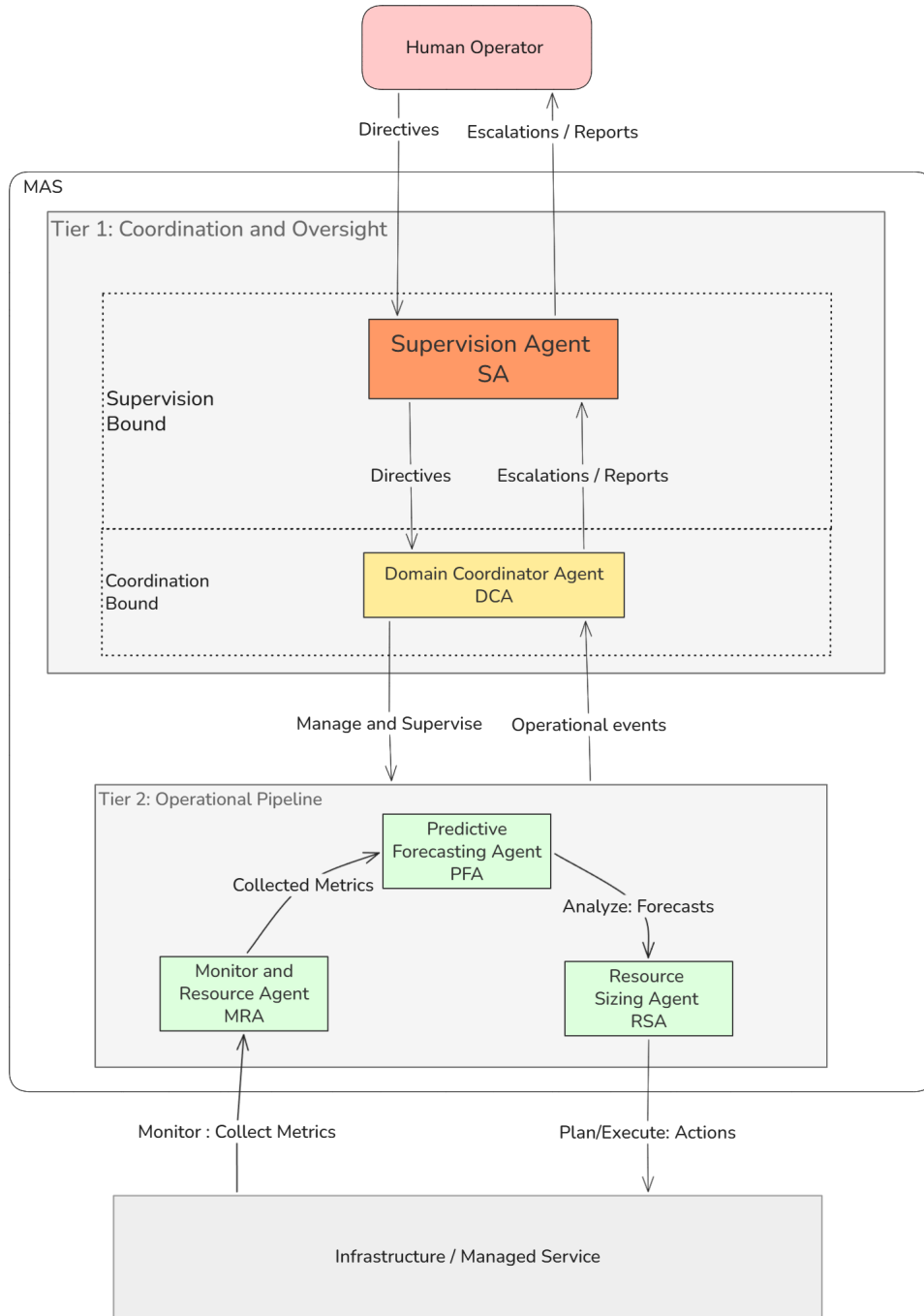


Figure 2.2: MAS Architecture

2.5.1 Tier 2: The Operational Pipeline

At the core of the system lies the operational tier, composed of a dedicated triads of specialized agents that form the primary resource management cycle. This tier executes its cycle without relying on Tier 1, which retains supervisory authority but does not intervene

in operational execution.

- **Monitor and Resource Agent (MRA):** Continuously collects telemetry from the infrastructure. It structures the collected metrics and shares them with the appropriate agents at fixed time intervals. Encapsulating monitoring within a dedicated agent ensures that infrastructure state and performance metrics are produced through a single, consistent, and validated source focused on a distinct and well-defined concern.
- **Predictive Forecasting Agent (PFA):** Consumes historical and current metric streams to forecast future resource demands before they materialize. A dedicated forecasting agent makes the system proactive by giving it sufficient lead time to adapt before resource pressure occurs. Without it, the system would be limited to reacting only after threshold breaches or performance degradation have already occurred.
- **Resource Sizing Agent (RSA):** Serves as the sizing engine and provisioner of the system. It evaluates the PFA's forecasts to compute optimal resource allocation and determine whether subsequent actions are required. Regrouping sizing and provisioning within a single agent ensures atomic response decisions, a single execution point, and reduced action latency by eliminating inter-agent coordination delays.

2.5.2 Tier 1: Coordination and Oversight

While the operational tier executes the resource management cycle, the coordination tier provides governance, failure recovery, and an escalation pathway to human operators. Internally, it is organized into two bounds: a lower coordination bound and a higher supervision bound, each corresponding to a distinct level of responsibility within the tier.

- **Domain Coordinator Agent (DCA):** The DCA occupies the lower coordination bound and acts as the intermediary between the operational tier and the higher supervision bound. It monitors the liveness of Tier 2 agents, orchestrates recovery, aggregates system state, all to ensure local stability and consistency. It also produces status summaries and filters operational events so that only relevant and context-enriched information is escalated to the SA, resulting in reduced noise and an effective oversight level.
- **Supervision Agent (SA):** Positioned within the upper supervision bound above the DCA, the SA serves as the bidirectional bridge between the human operator and the multi-agent system, enforcing the operator's intent within the MAS and providing structured, relevant, and aggregated escalations to the operator. The SA ensures that the operator retains full authority over the system without being exposed to escalations that the agents can resolve autonomously.

2.5.3 Agent Distribution

The strategy behind the upcoming agent distribution was defined after taking into consideration what each agent offers, how it is best utilized, and finally the design objectives that we intend to achieve with this architecture.

These considerations lead us to a distribution that positions the operational agents at the service level, where direct interaction with service-specific state is available, while the coordination and supervision agents are positioned at broader administrative scopes.

This distribution limits impact of failures, preserves service level isolation, and aligns each agent with the optimal scope of its abilities.

- **MRA – One instance per managed service:** The MRA is the exclusive source of telemetry for a service. Deploying one instance per managed service ensures that each agent collects only the metrics of its assigned service. These metrics in their turn provide a precise and isolated view of the service’s operational state. This prevents cross-service metric contamination, eliminates the need for additional filtering or partitioning, and localizes failures to the affected service.
- **PFA – One instance per managed service:** The PFA forecasts the future demands of the service based on its historical consumption. Sharing a single PFA across multiple services would mix heterogeneous workloads, reducing forecast accuracy, which in consequence amplifies error propagation to subsequent scaling decisions. Assigning a dedicated PFA to each service preserves data isolation, enables the agent to capture service-specific seasonality and non-stationary behavior, mitigates workload heterogeneity and global model rigidity, avoids centralized bottlenecks, and eliminates a system wide single point of failure by confining failures to individual services.
- **RSA – One instance per managed service:** The RSA performs sizing and provisioning for its associated service. One RSA instance per service eliminates cross-service queuing and sequencing delays, allowing scaling actions to be executed instantly. This reduces reaction latency during critical conditions, ensures that every service has a dedicated and uncontested scaling path, it also avoids centralized bottlenecks, and finally confines failures to only the affected service. One instance per service mirrors the PFA and RSA distribution decisions to form a unified per service operational flow.

These three agents together form the resource management pipeline for each managed service.

- **DCA – One instance per administrative domain:** An administrative domain contains multiple managed services, each represented by its own MRA-PFA-RSA triad. The DCA is specifically designed to coordinate and stabilize these multiple service-level pipelines. One DCA deployed per service would introduce unnecessary redundancy and supervisory noise instead of actual value. On the other hand, a single DCA for the entire system would create a major bottleneck and a system-wide single point of failure. The choice of one DCA per administrative domain provides an appropriate coordination scope, contains failures within the affected domain, and leaves other domains unaffected.
- **SA – One instance per DCA group:** An SA supervises a group of DCAs, and each DCA represents a separate administrative domain. A single SA for all DCAs would result in both a bottleneck and a system-wide single point of failure, while one SA per DCA would only create unnecessary overhead with no added benefit. The selected position of one SA per group defines a well balanced and manageable oversight over the system, while also allowing the system to limit the failure impact of any SA to its respectable group. An SA supervising a group of DCAs will be referred to as a partition in subsequent sections.

2.5.4 Topology

Both the deployment strategy and architecture we defined in the previous sections result in a topology with three levels. Each level carries a well-defined responsibility.

- Level 1: one operational triad (MRA-PFA-RSA) per managed service at the base.
- Level 2: one DCA per administrative domain above level 1.
- Level 3: one SA per DCA group at the very top.

The resulting topology is illustrated in Figure 2.3 and highlights the organization of the proposed architecture and the adopted distribution strategy across three operational levels.

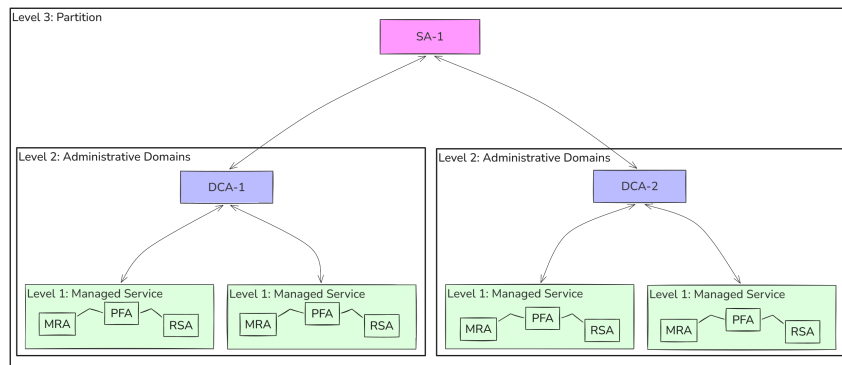


Figure 2.3: MAS Topology

This topology scales progressively with the number of managed services through an alternating pattern of horizontal expansion between levels. The expansion starts at level 1, where each new managed service adds one operational agent triad to the current administrative domain. When the maximum number of triads the DCA can manage is reached, the topology shifts to level 2 and expands it by adding a new DCA, then shifts back to level 1 and resumes expansion in the administrative domain of the new DCA.

Once this is repeated enough times and the maximum number of DCAs that a single SA can supervise is reached, the topology shifts to level 3 and adds a new SA, which creates a new partition initially containing that new SA and one DCA and one operational triad. This same pattern of shifting back and forth between levels and horizontal expansion within one repeats as new services are added until the available hardware capacity is reached.

Figure 2.4 illustrates how the proposed topology expands as new managed services are introduced and the capacity limits of each level are reached.

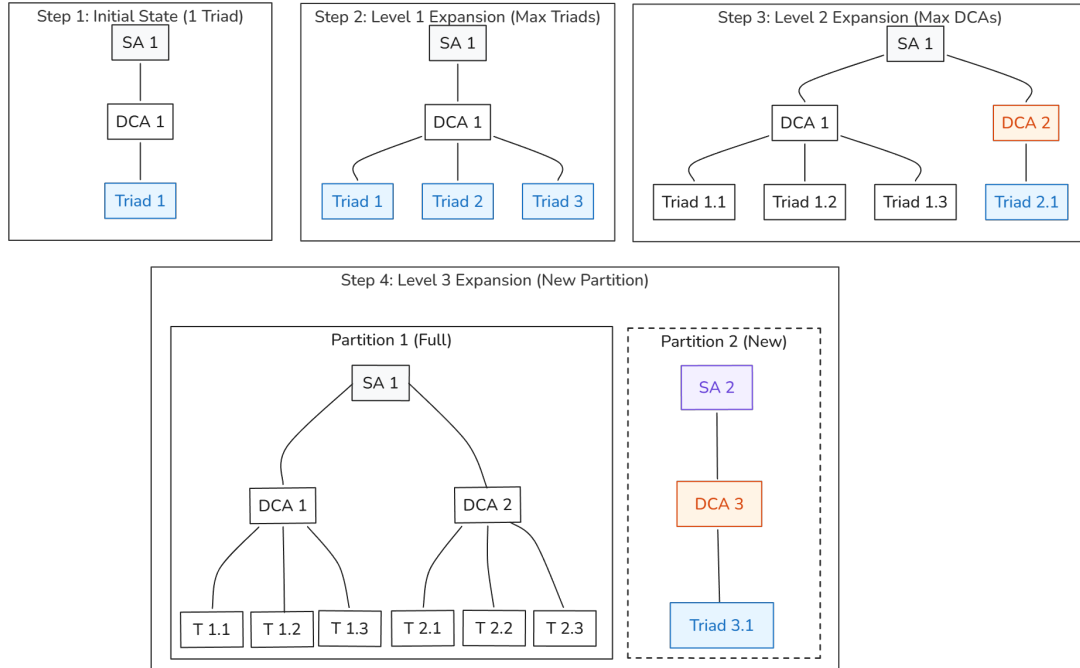


Figure 2.4: MAS Topology Expansion

2.5.5 Operational Flow

During execution, the system operates two concurrent paths that reflect its structural decomposition.

The first path executes resource management for each managed service through an independent agent triad. Within this triad, the MRA extracts real-time infrastructure metrics, then the PFA utilizes these metrics to forecast future resource demand, and at last the RSA evaluates these forecasts to execute appropriate provisioning actions.

In parallel, the second path provides coordination and supervision over the operating triads. The DCA continuously monitors agent health, aggregates the state of its domain, and forwards to the SA only escalations that require human attention or exceed its autonomous resolution capabilities. The SA, on the other hand, tracks the liveness of its DCAs and mediates bidirectional interaction between the human operator and the MAS by providing system oversight and the authority to intervene.

2.5.6 Agent Behavioral Architecture:

Proactive supervision and resource management require agents capable of anticipatory reasoning. This means the ability to form expectations about future system states and commit to actions before demand materializes. Purely reactive agents carry no internal state and respond only to immediate stimuli, thus cannot satisfy this requirement by construction.

The BDI model, grounded in Bratman’s theory of practical reasoning [49] and formalized by Rao and Georgeff [50], [51], provides this capability natively through its explicit representation of beliefs, desires, and intentions. However, pure BDI is deemed insufficient for time critical and safety sensitive decisions [52], [53], [54], [55]. For example, a deliberation cycle processing a long forecasting horizon must not delay the response during an elevated pressure surge. For this reason, the system adopts a BDI approach applied asymmetrically based on each agent’s role.

The SA and DCA strictly operate on the oversight and coordination tier and do not face millisecond level infrastructure constraints. They are modeled as pure BDI agents because their decision space is complex but full deliberation is both feasible and appropriate for producing optimal decisions.

The MRA and PFA are also modeled as BDI agents. They maintain beliefs and pursue persistent desires, but their intentions are restricted and kept simple and clear within fixed pipelines, in order to respect the time constraints that guarantee a deterministic publication cadence that downstream deliberative agents depend on.

The RSA is the only agent authorized to issue sizing and provisioning decisions that adjust the resource capacity of a managed service. For that specific reason, it is designed as a hybrid agent BDI. It has a deliberative layer (BDI) that governs proactive resource expansion and a reactive layer that is responsible for two functions: conservative resource capacity reduction and fallback emergency resource expansion.

2.5.7 Policy Definition

From an operational point of view, the policy is a blueprint with different sets of thresholds, limits, parameters, and timings that influence autonomous behavior across the MAS functions. In contrast, from an architectural perspective, it represents a supervision

concern that is specified and validated at the supervision layer, then propagated to the lower ones before execution begins.

It is defined and maintained by the human operator based on the nature of the service and its associated service level agreement, in addition to the service level objectives of the infrastructure. SLAs and SLOs are used to introduce a sense of awareness regarding performance expectations and standards within the agents. The reason behind this is to help better uphold SLAs and SLOs, which in its turn increases the value and efficiency of the system. The policy can be revised and updated whenever priorities shift, conditions change, or new requirements arise. The policy being based on the SLA associated with each service enables the MAS to manage each service under a policy that is tailored to its requirements while also remaining aligned with the global infrastructure objectives.

The policy plays an important role in shaping agent behavior and preserving a consistent decision logic across distributed agents; that is why it needs to be defined with internal consistency and operational precision and also meticulously validated before deployment to avoid inconsistent or contradictory constraints infiltrating agent's decision processes.

2.5.8 Human Oversight Model

The architecture adopts a structured human in the loop model in which the operator retains ultimate authority while agents handle all routine and repetitive decisions autonomously.

This positions the system at an intermediate autonomy level three or four on the TM-Forum Autonomous Networks scale [56][57]. The agents act autonomously within defined boundaries and escalate to the operator for situations that exceed their authorization.

The human operator retains four control mechanisms : policy definition, action on escalated decisions, forced direct action, rollback actions and emergency stop. These mechanisms ensure human authority at any given time as safety and emergency measures for extreme and rare scenarios.

2.6 Communication

2.6.1 Communication Model

Communication Requirements: The communication model is a foundational component in the global architecture of the system. It is considered to be the backbone of the MAS because it handles all inter-agent communication. For this reason it needs to be compatible with the nature and behavior of these agents and complement it. The agents within the MAS are distributed across three different layers (service, administrative domain, and partition), where each agent is driven by its internal state and operates on its own independent cycle.

This results in an elevated difficulty to determine the readiness of an agent to receive a message at the same moment another needs to transmit, and it is why a synchronous communication model falls instantly from the picture. A communication model that requires the sender to wait for receiver acknowledgment or constrains message delivery on receiver readiness would introduce a complex coordination overhead that would only turn into unwanted latency and reduced efficiency in a system like ours where timely execution and smooth operational flow are critical. Such a model would also introduce implicit temporal coupling between agents which would be a violation of a foundational design requirement. As a result the chosen communication model must remain structurally aligned with foundational architectural decisions and the characteristics that define the agents.

Selected Model: The system adopts an asynchronous broker-mediated publish-subscribe model. This model consists of three components: producers, consumers, and a broker. Producing agents publish structured message types to a central broker without addressing any specific receivers. Consuming agents subscribe to the message types relevant to their role and process them according to their own execution cycle. The broker is not an agent itself, but rather a platform that helps to decouple producers and consumers, centralize routing, and eliminate the need for any producer to maintain awareness of its consumers or manage multiple targeted transmissions. The semantics of the mentioned message types are addressed in subsection 2.6.2.

Architectural alignment: This model choice directly reflects the requirements imposed by the architecture. First, based on the architectural design, the tier 2 must remain structurally independent of tier 1 availability. This means that communication must not create any implicit or explicit dependency between the two tiers, because a dependency of this kind could spread latency or unavailability from tier 1 downward and affect operational execution. This is exactly the unwanted behavior that a synchronous model would introduce.

Second, agent coordination requires in many cases a single message produced by one agent to reach multiple consumers at once, which introduces fan-out requirements. A point-to-point model would place a routing burden on the producer and also increase coupling to its consumers, which directly shifts focus away from its primary responsibilities. So, broker mediation is well justified in this case because it absorbs this responsibility entirely and frees producing agents from recipient management liability.

Finally, with a topology that consists of multiple triads per domain and multiple domains per partition, the number of concurrent agents and inter-agent interactions grows substantially. The combination of asynchronous delivery, broker-mediated routing, and subscription-based consumption ensures that this growth is handled efficiently. Each agent

publishes on its own cycle without waiting on receivers, the broker handles all routing and fan-out without producer involvement, and each consumer independently receives only what it needs. This allows the system to scale reliably without agents blocking each other or adding coordination overhead to individual agents.

Model Properties: The model guarantees three properties that matter directly to the system. The first two properties, loose coupling and non-blocking execution, are ensured because producers and consumers have no awareness of each other's identity, internal state, or cycle, and no agent is stalled by another's delay or unavailability. The third and last property is the fan-out property. It is ensured by design because a single publication reaches all relevant subscribers without any added complexity on the producer side.

2.6.2 Communication Semantics

Semantic scope

The communication model defined in the previous subsection specifies how inter-agent communication is carried out through a broker-mediated publish-subscribe model. However, a transport mechanism alone is not sufficient for reliable agent interactions. In our defined system, as it stands, agents must share a common understanding of what a message means, when it must be sent, and what its absence implies. For this reason we formalize our inter-agent exchanges within a custom communication language made to provide a common understanding between agents and enhance their interactions.

The language is inspired by and draws from FIPA-ACL [58] and KQML[59], [60], which are two established languages that were designed for general purpose use. Unlike them, our custom language is intentionally designed to be simpler and lightweight, and only includes what is considered essential for the system's design to avoid unnecessary complexity. It addresses three concerns: the communicative intent of every message, the exchanged message types, and the interaction rules.

Performatives

Every message in the language carries exactly one performative, which represents the communicative act performed by the sender. In other words, it declares the intent of the sender. Six performatives are defined as follows:

- **INFORM:** The sender shares a very lightweight, informative, and routine signal with receivers. No response is required.
- **REPORT:** The sender publishes a routine structured summary of an observed or derived state. No response is required, but the information is operationally significant.

- **ALERT:** The sender signals an abnormal or degraded condition that requires the receiver to evaluate and potentially act.
- **NOTIFY:** The sender communicates an event that must be acknowledged internally by the receiver and acted upon.
- **ESCALATE:** The sender delegates an unresolved or out-of-scope condition to a higher authority. The receiving agent is expected to assume responsibility for resolution or further delegation.
- **DIRECTIVE:** The sender shares an instruction that the receiver must enforce. Used exclusively by agents of higher authority, or by the human operator.

Message types

All messages carry one performative. The table below summarizes system message types.

Performative	Senders	Message	Receivers
INFORM	MRA	Liveness Signal	DCA
	PFA	Liveness Signal	DCA
	RSA	Liveness Signal	DCA
	DCA	Liveness Signal	SA
	SA	Liveness Signal	Human Operator
REPORT	MRA	Service State Update	PFA, RSA, DCA
	PFA	Forecast Output	RSA, DCA
	RSA	Resource Adjustment	DCA
	DCA	Domain State Report	SA
	SA	Partition Health Overview	Human Operator
ALERT	MRA	Data Collection Failure	PFA, RSA, DCA
		Validation Error	PFA, RSA, DCA
		Configuration Fault	PFA, RSA, DCA
	PFA	Computation Failure	DCA, RSA
		Degraded Input Quality	DCA, RSA
		Operational Unreadiness	DCA, RSA
		Unavailability Status Report	DCA
	RSA	Capacity Alert	DCA
		Unavailability Status Report	DCA
NOTIFY	DCA	Policy Notification	MRA, PFA, RSA
		Confirmed Agent Failure	MRA, PFA, RSA
	SA	Policy Signal	DCA
ESCALATE	DCA	Escalation Signal	SA
	SA	Escalation Presentation	Human Operator
DIRECTIVE	DCA	Relayed Control Signal	MRA, PFA, RSA
	SA	Control Signal	DCA

Table 2.1: Global table of message types

Interaction rules

The language defines two categories of rules :

1. **Publication rules:** they guide when certain messages must or must not be sent:

- Liveness signals of all agents must be published at fixed policy-defined intervals.
- Service state update is published at every monitoring cycle. If an unexpected event occurs, a corresponding status signal is published instead.

- Forecast output is only published after a successful forecasting cycle. A status signal is published instead if data quantity and quality fall below acceptable standards or if a computation failure occurs.
- Resource adjustment is published only when a capacity change is actually executed. A "no action" decision does not produce this message.
- Domain state report and Partition health overview are published at a policy-defined frequency.

2. **Absence interpretation rules:** they define what the sustained absence of a certain messages imply:

- Sustained absence of either Service state update or Forecast output for a policy-defined number of cycles, without a corresponding status signal, is interpreted by peer agents as a probable MRA or PFA failure, respectively. An Unavailability status report must be published.
- Sustained absence of any agent's Liveness signal beyond a policy-defined number of intervals is interpreted by the supervising agent as a probable failure, triggering the failure detection confirmation procedure.
- Absence is only conclusive when cross-validated against peer observations. No single evidence source alone is sufficient to confirm failure.

2.7 Knowledge Base

2.7.1 Conceptual Definition

The knowledge base is the shared, persistent, and structured storage space that all agents use as their common ground to read from and write to according to specifically defined access policies. It is one of four architectural components that make up the global system, and it is responsible for maintaining a shared and durable state across the agents of the system. Due to the distributed and autonomous nature of the agents, this knowledge base is necessary to preserve the continuity of context across restarts, failures, and deliberation cycles.

Before continuing any further, there is a distinction that needs to be made between the knowledge base and individual agent beliefs. Agent beliefs represent what a specific agent currently knows about its immediate environment. This knowledge lives in the agent's memory, and is rebuilt from scratch on restart unless a backup mechanism is implemented. These particular beliefs are private, local, and volatile. The knowledge base itself is shared, global, and persistent. It represents system-wide facts that multiple agents need to access, that must survive agent restarts, and that constitute the authoritative record of system state and history.

2.7.2 Knowledge Base Domains

The knowledge base is not explicitly a database in the traditional sense. It is organized into five functional domains where each one is reserved for a specific system concern:

- **Policy Domain:** stores all the policies implemented in the system. These policies are defined by the human operator and they contain various operational parameters and constraints. The SA writes the policies after validation and each agent reads its associated policy in order to load and implement it. This allows agents to stay internally consistent and compliant with their associated policies.
- **Topology Domain:** holds the entire structure and topology of the system. It contains structural mappings that span from the SAs at the top to the managed services at the bottom, including all the partitions, DCAs, administrative domains, and operational triads of the system. These mappings describe all decomposition levels of the system, their affiliated entities and their managers. This domain grants global visibility of the system and supports traceability and accountability. The SA uses it to acquire necessary structural context, and operational agents read from it to verify service existence and assignment before entering active operation.
- **Historical State Domain:** stores monitoring records (time series) produced by the MRA at every observation cycle. It allows the system to preserve historical data that can be used later for model training and fine-tuning or system analysis and auditing. The PFA reads from this domain on restart to reconstruct its forecasting context, allowing it to resume operation without having to rebuild it from the ground up.
- **Audit Log Domain:** represents an operational ledger that stores diagnostic events, coordination decisions, resource adjustment outcomes, and operator interventions, and other significant outcomes. It acts as both a retrospective diagnostic trail for analyzing past behavior and a reference for assessing the current system state. Every agent writes to it to ensure that behavior remains observable and traceable over time.
- **Coordination Domain:** used by Domain Coordinator Agents to maintain coordination states by tracking assigned agent registration records and their liveness status, ongoing coordination operations like active escalations, and current control directives relayed by the SA like the operational mode of the system. This domain is exclusively managed by the DCA, and it is read from or written to across interruptions and recoveries to preserve continuity of global directives and coordination operations.

2.7.3 Concurrency Control

The knowledge base is a central component that is accessed simultaneously by multiple agents of the system. For this reason, preventing race conditions and data corruption becomes necessary to preserve the integrity and coherence of the data. To address the issue, we decided to adopt a structural strict role-based isolation solution instead of a separate coordination mechanism such as distributed locking, which can be complex and may introduce unwanted latency and deadlocks into the system. This solution implies each agent only has access to domains that match its role within the MAS.

As a result, each domain has exactly one agent-type manager that is authorized to write to it, while other agents are read-only consumers: the SA manages Policy Domain, the DCA manages Coordination Domain, and the MRA manages Historical State Domain. This design eliminates write-write conflicts between agents. However, two exceptions represented by Topology Domain and Audit Log Domain arise in this solution. The Topology Domain is managed by the human operator, and the agents interacting with it only read from it, which eliminates any conflicts. As for the Audit Log Domain, all agents without exception require write access to it, and there is no way around that. For this specific reason, interactions are restricted to an append-only pattern, where each agent writes immutable records to the domain without modifying or invalidating existing entries. While this is not an optimal solution by any means, it still helps circumvent contention for this domain.

For better clarity, the following table summarizes the access rights of agents across each domain:

Domain	MRA	PFA	RSA	DCA	SA
Policy	R	R	R	R	RW
Topology	R	R	R	-	R
Historical State	W	R	-	-	-
Audit Log	W	W	W	W	W
Coordination	-	-	-	RW	-

R = Read, W = Write, RW = Read/Write, - = No access.

Table 2.2: Domain Access Across Agents

As a final note, the proposed solution addresses the problem from an agent-type standpoint. Nonetheless, each agent type has multiple instances running within the system. To account for this, we clarify that each domain is internally segmented and structured to ensure strict separation between instances. This approach ensures that each agent is responsible for a unique and specific record associated with it.

2.8 Agent Internal Design

2.8.1 Monitor and Resource Agent (MRA)

Role and Positioning

The Monitor and Resource Agent is the observational foundation of the MAS. One MRA instance is deployed per managed service, and it constitutes the system's sole point of contact with the target infrastructure. Every other agent derives its knowledge of the service state exclusively from MRA publications. This strict boundary ensures that infrastructure monitoring is centralized within the triad, consistent, and independently governable.

The MRA's scope is deliberately narrow. It performs no provisioning decisions, no predictive reasoning, and no coordination actions. Its responsibility is confined to a fixed observation cycle in which it collects, normalizes, and validates telemetry, next it performs a resource pressure assessment, then publishes a structured service state representation, in the end it stores the collected telemetry in the Knowledge Base.

BDI Model

The MRA is modeled as a BDI agent. It maintains beliefs and pursues a persistent desire, but its intentions are restricted to a periodic and fixed pipeline with no complex branching.

Beliefs: The MRA's world model consists of the current state of its assigned service, including observed resource utilization, allocated capacity, current service scale, and the resulting pressure assessment. These beliefs are refreshed at every cycle and immediately replace prior beliefs.

Desires: The MRA's persistent goal is to maintain an accurate and timely representation of the service state for the other agents. This is implemented as an unconditional commitment to publication at every cycle. Even in the presence of data collection failures, validation errors, or supplemented observations, an appropriate message is always emitted.

Intentions: At each cycle, the MRA commits to executing its monitoring cycle: collect, normalize, validate, assess, and publish. Conditional branching is limited to data quality outcomes, determining whether incomplete observations are rejected, completed with explicit uncertainty markings, or propagated as valid state updates. The MRA does not deliberate over whether to publish; publication is a non-negotiable commitment.

Internal Logic

The MRA executes a fixed, sequential pipeline on every monitoring cycle. Each stage has a single responsibility and a clearly defined failure behavior.

Metric collection and aggregation: Raw metrics are collected from the infrastructure at the lowest observable execution unit level, capturing resource usage, resource allocation, and current service scale. These values are then aggregated into service level observations. The resulting metrics fall into two categories:

- **Forecast metrics:** include the service resource usage that supports the PFA’s forecasting context.
- **Context metrics:** include service resource allocation and scale state that support the RSA’s sizing and provisioning decisions and the DCA’s domain state tracking.

Preprocessing and validation: Before belief construction, raw metric values are normalized into a consistent internal representation and infrastructure identifiers are standardized to ensure consistency and provide audit traceability across the system.

Three validation rules are then applied in sequence. First, a validity check ensures that utilization values remain within plausible bounds. Then a temporal consistency check rejects duplicate or out-of-order observations. The completeness check handles cycles where one or more metrics are unavailable: Instead of silently dropping the observation, the MRA completes the missing values using the most recent valid observation, preserving the fixed temporal cadence that the other agents rely upon.

A special condition arises when a managed service has no resource allocation defined. The range check cannot be applied and a structured configuration error is emitted instead of a belief update for that cycle.

Pressure assessment: The MRA classifies resource pressure using predefined policy thresholds for CPU and memory relative to allocated capacity. For each resource, utilization is mapped to four states (low, normal, warning, and critical) based on usage intensity. The overall service pressure level is defined as the maximum of the two classifications. This level drives downstream behavior directly: low may trigger the RSA’s reactive resource recovery path once sustained under utilization is established, warning triggers proactive forecasting by the PFA, and critical activates the RSA’s emergency allocation path once the condition is confirmed.

Publication behavior: The MRA publishes a structured service state update at every monitoring cycle regardless of whether values have changed. When collection and validation succeed, a complete state update is published; otherwise, a structured error signal is

emitted. This unconditional and fixed publication cadence ensures that the PFA receives a temporally consistent observation stream and that sustained publication absence is interpretable by peer agents as a meaningful failure signal.

Inputs and Outputs

Inputs: The MRA is not dependent on any agent for its operation and relies only on the infrastructure that it collects observations from. For this reason, it receives only a minimal set of inputs that are required for its operations. These inputs consist of a policy notification signals, relayed control signals, and confirmed agent failure notifications all from the DCA. The policy notification signal informs the MRA a new version of its associated policy is available and it should update its current one, while the relayed control signals represent directives that must be enforced and applied. As for the confirmed agent failure notification, it is received passively, since it does not affect the MRA’s behavior; the agent simply records the event and continues its monitoring task.

Outputs: The MRA publishes six distinct event types that cover both its primary belief stream and its diagnostic status signals.

Output	Consumers	Description
Periodic liveness signal	DCA	Confirms the MRA is operational, published periodically within a fixed time interval.
Service state update publication	PFA, RSA, DCA	Structured view of the current service state, published for the other agents each cycle.
Data collection failure signal	DCA, PFA, RSA	Indicates that telemetry collection was unavailable for the current cycle.
Validation error signal	DCA, PFA, RSA	The observation was rejected because it violated validity, temporal consistency or completeness constraints.
Configuration fault signal	DCA, PFA, RSA	Required resource allocation information is absent.

Table 2.3: MRA Publications

Knowledge Base Access

The MRA interacts with the Knowledge Base at every cycle by writing operational records continuously throughout its execution. It also keeps its policy up to date through the policy domain.

Domain	Access	Purpose
Policy Domain	Read	Retrieves the policy thresholds required for pressure assessment.
Topology Domain	Read	Confirms that the assigned managed service exists in the system topology before monitoring begins.
Historical State Domain	Write	Stores historical monitoring records to preserve temporal continuity and support state recovery.
Audit Log Domain	Write	Stores diagnostic records related to data quality and availability issues.

Table 2.4: MRA Knowledge Base Access

Interaction and Coordination :

The MRA sits at the base of the MAS dependency chain. Every other agent operational capability is conditioned on its output, yet the MRA itself has no runtime dependency on any peer agent.

With the PFA: The MRA is the PFA’s only source of service telemetry in the MAS, which makes the PFA entirely dependent on its output stream. Every service state update extends the maintained forecasting context, it also influences when the PFA performs forecasting allowing it to react more aggressively under elevated pressure while remaining periodic under stable conditions. The PFA interprets the state as an indicator of input quality, which is reflected in the uncertainty assessment. The presence of service state publication is monitored by the PFA, and an unavailability status report is published to the DCA when state absence persists.

With the RSA: The service state publication supplies the RSA with the pressure assessment and with the context metrics required for both provisioning and sizing decisions. Confirmed critical pressure activates the RSA’s emergency path, while sustained low pressure activates the reactive resource recovery path. The RSA also, like the PFA, monitors the presence of service state publication as a liveness indicator; it reports an unavailability status to the DCA when absence is sustained.

With the DCA: The MRA provides the DCA with the service state information needed to maintain the domain pressure view and to supervise operational conditions within the domain. It also sends periodic liveness signals for availability supervision, and status signals that inform the DCA of any monitoring related errors. In return, the DCA evaluates if status signal conditions can be resolved locally or require further escalation. In addition,

it keeps the MRA informed of new policy versions with policy notification signals and relays control signals from the SA. The DCA provides agent failure notifications to the MRA and if the MRA itself fails, the DCA informs the dependent agents and coordinates the appropriate recovery response.

2.8.2 Predictive Forecasting Agent (PFA)

Role and Positioning

The Predictive Forecasting Agent (PFA) is the intelligence layer of the Multi-Agent System (MAS), positioned at the boundary between service state observation and autonomous sizing and provisioning decisions. It transforms the system from a reactive architecture into a proactive one by anticipating resource demand before thresholds are crossed and critical levels are reached. This provides the RSA with sufficient lead time for preventive intervention. One PFA instance is deployed per managed service, mirroring the granularity of the MRA. It consumes structured service state updates from the MRA, maintains a forecasting context, performs predictive analysis over the this context, and produces actionable forecast outputs for subsequent decision making. The PFA role is strictly bounded to demand anticipation and uncertainty assessment of its forecasts. This positioning makes the PFA the critical enabler of the system's proactive capability. Its absence reduces the system to emergency-only reactive behavior, while its correct operation determines the quality and timeliness of preventive sizing and provisioning decisions.

BDI Model

The PFA is modeled as a BDI agent whose beliefs reflect both service state and forecast quality. Instead of open ended deliberation, its intentions are constrained by the periodic forecasting interval and elevated pressure conditions, as well as data sufficiency, availability, and quality.

Beliefs: The PFA's world model includes the contents of the maintained forecasting context, the most recent pressure assessment, and indicators of input quality within the context. These beliefs are continuously updated from incoming service state publications, and they both inform the decision to perform a forecast and the reliability of its output (certainty). The forecasting context refers to the accumulated resource usage history maintained by the PFA, on which future demand estimation is based.

Desires: The PFA pursues two persistent goals. Its primary desire is to maintain an accurate and timely forecast of future demand so that the RSA has sufficient lead time for preventive action and can take optimal sizing decisions. Its secondary desire is to preserve integrity by informing other agents of its degraded state when inference is suspended

because input quality is below acceptable standards or when produced outputs are invalid and therefore withheld.

Intentions: Three intention patterns guide the PFA’s behavior. It commits to executing a forecasting cycle when elevated resource pressure is detected in an incoming service state update or when the periodic forecasting interval elapses, and then publishes the resulting forecast output. It commits to suspending forecasting and signaling a degraded mode when data or its quality is insufficient. This protects dependent agents from unreliable forecasts. Finally, it commits to reporting structured failure conditions to the DCA when the forecast cannot be completed successfully or when MRA publications are absent for a sustained period of time.

Internal Logic

Forecasting activation: The PFA updates its forecasting context on every incoming service state update regardless of whether forecasting is triggered or not. Forecasting is initiated by two triggers: a pressure driven trigger activated by elevated resource pressure and a periodic trigger that ensures regular reassessment even in stable conditions. The former takes precedence over the latter to ensure that the agent can respond immediately to emerging pressure. The periodic trigger resets each time the pressure driven trigger initiates forecasting. To avoid excessive forecasting activity during sustained pressure conditions, the PFA enforces a policy defined cooldown period between consecutive pressure driven forecasting cycles, while the periodic trigger remains independent of this cooldown.

Forecasting context management: The PFA needs a sufficient and clean history of recent observations before it can produce reliable forecasts. For this specific reason, it maintains a fixed forecasting context for relevant resources that is continuously updated from incoming service state updates. This context needs to accumulate a sufficient amount of data before forecasting can be performed reliably, and its size needs to be aligned correctly with the used model input requirements and properties to preserve integrity and avoid any artificial reshaping. The PFA keeps track of supplemented observations as indicators of input quality when they enter the forecasting context and suspends forecasting if they become too prevalent over time. This prevents the agent from producing unreliable forecast outputs that degrade the system’s performance.

Forecasting model: The PFA relies on a transformer based forecasting model to produce future demand estimates based on the maintained forecasting context. The reasoning behind this choice is that transformer-based models are widely used in time-series forecasting due to their effectiveness in capturing complex patterns and long-term dependencies within data and their parallel processing capabilities. Although they were

originally developed for natural language processing tasks, they have since become a major approach in time-series forecasting, with many improvements and architectural adaptations proposed and applied to better suit the domain and unlock their full potential. In addition, transformer-based models have outperformed many classical baseline models, which showcases strong empirical evidence supporting their effectiveness.[61] [62] [63]

Uncertainty quantification: After each successful forecasting cycle, the PFA computes an uncertainty measure that expresses the degree of confidence associated with the anticipated demand. The reason behind this measure is to prevent the system from treating all forecasts with the same level of reliability because forecast quality can be affected by changes in service conditions or diminishing quality of input data. So, the solution was to attach an explicit confidence indicator to each produced forecast that enables downstream decision making to remain risk aware instead of blindly relying on predictions. As a result, the RSA can now interpret forecast confidence and adapt its behavior accordingly: the lower confidence drops, the more cautious and conservative the RSA is, and the higher confidence rises, the more precise and aggressive it is. This makes proactive resource adjustments safer but less accurate during uncertain conditions, which represents an acceptable tradeoff in this context.

Time to breach derivation: Time to breach is a mechanism that gives the PFA the ability to derive a temporal interpretation from its forecast by estimating when anticipated demand is likely to violate policy-defined operational thresholds. This helps to determine if a future breach is possible and, if so, how much lead time would be remaining before a preventive action may be necessary. To accompany the time to breach, the PFA also derives a breach window that constitutes a bounded timing interval that captures the possible variations in when the breach might happen.

This temporal view supports the proactive nature of the architecture because when the anticipated breach is close and reliable enough, it is the main influence that fuels the proactive path of the RSA. In contrast, when the breach is less certain or more distant, it supports continued monitoring and earlier reassessment without forcing immediate action.

Forecast accuracy tracking: The PFA tracks the accuracy of its forecasts by measuring how close the previous demand predictions were to the actual recorded ones. This mechanism enables the MAS to evaluate its proactive performance and to see how reliable it was under various operating conditions. Instead of directly influencing the current forecast, the purpose behind this tracking is to preserve an explicit record of the forecast quality over time alongside the associated evaluation context. This record ensures consistent forecast quality evaluation, enforces system traceability, and supports later analysis and audit.

Publication behavior : Under normal conditions, the PFA publishes structured forecast outputs after each successful forecasting cycle. These outputs contain all the necessary information for downstream decision-making. Additionally, it emits periodic liveness signals to support supervision.

Under unfavorable conditions, where forecasting cannot be performed reliably, the PFA does not publish normal forecast outputs and instead issues status signals that indicate computation failure, degraded input quality, or operational unreadiness. Lastly, the PFA contributes to failure detection by publishing unavailability status signals when the functional publications of peer agents are absent beyond an acceptable period of time.

Inputs and Outputs

Inputs: The PFA receives its primary operational input from the service state publications and status signals of the monitoring agent. Together, they provide the historical context required for forecasting and they enable the PFA to make the distinction between temporary monitoring issues and actual agent failure. It also receives policy notification and relayed control signals from the DCA, which respectively inform the DCA of policy changes and provide directives that must be applied and enforced. In addition, it passively receives confirmed agent failure notifications for contextual awareness, although these do not directly alter its forecasting role.

Outputs: The PFA produces a well defined set of outputs covering forecast delivery, operational status, and peer agent failure detection. Under normal conditions, it emits regular liveness signals, communicates when it is still building sufficient context, and publishes structured forecast outputs. The structured forecast output publication is its main output and it contains the forecast itself, timing indicators, uncertainty measures, the state context on which the forecast was based on, among other information. When forecasting cannot be performed reliably, the PFA either reports a computation failure, a degraded input quality, or operational unreadiness signals. The PFA also acts as a peer agent failure detector and publishes unavailability status signals to the DCA after sustained publication absence of the tracked agents.

Knowledge Base Access

The PFA uses the Knowledge Base as a persistent support layer for policy retrieval, context restoration, and operational traceability.

Domain	Access	Purpose
Policy Domain	Read	Retrieves the operational parameters required to evaluate forecasted demand.
Topology Domain	Read	Verifies that the assigned service exists and is valid before entering active operation.
Historical State Domain	Read	Restores prior observation records to reconstruct the forecasting context after a restart.
Audit Log Domain	Write	Records forecasting activity and evaluation outcomes to ensure that agent behavior remains observable and traceable over time.

Table 2.5: PFA Knowledge Base Access

Interaction and Coordination

With the MRA: The PFA is entirely dependent on the MRA’s state update publications. They are its only source of service state publications and the foundation of its forecasting context. Every service state update also carries the pressure assessment that influences when the PFA performs forecasting, making the agent more vigilant under high pressure conditions while retaining periodic forecasting during stable ones. The PFA treats the presence of state update publication as a liveness indicator, but it also keeps track of the status signals to distinguish transient MRA errors from actual agent failures. When state update publications are absent for a sustained period, the agent reports an MRA unavailability status to the DCA and suspends forecasting until service state publication resumes.

With the RSA: The PFA provides the RSA with forecast outputs that carry the forecast itself and other necessary information. The RSA uses these outputs for proactive resource adjustment, where it interprets future demand and prepares the appropriate sizing and provisioning decisions. A credible upcoming breach detected by the forecast initiates proactive deliberation within the RSA, while the forecast confidence influences how cautious or aggressive the resulting sizing decision should be. In addition, the PFA informs both the DCA and RSA when it enters a degraded forecasting state and cannot produce forecast outputs reliably. This drives the RSA to suspend its proactive path and rely on its reactive one until the degraded state ends and publications resume. The RSA tracks the presence of PFA publications as part of peer agent monitoring and reports sustained absence to the DCA as a probable failure.

With the DCA: The DCA depends on the PFA’s publications to maintain an up-to-date view of the domain forecast risk state and to supervise forecasting conditions within its

domain. It uses the publications in three ways: the first one is to track forecasting activity, the second is to detect degraded forecasting states, and finally the third is to identify conditions that may require intervention. The identified conditions are later evaluated to determine if the DCA would be capable of handling them or an escalation to the higher supervision layer is required. The DCA relays control signals that contain directives from the SA, issues agent failure notification signals when an agent fails, policy notifications when policy is changed, and informs other agents and coordinates recovery in case of PFA failure.

2.8.3 Resource Sizing Agent (RSA)

Role and Positioning

The Resource Sizing Agent is the sizing and provisioning engine of the MAS. It represents the action point of the operational pipeline because it converts service state and forecast outputs into sizing decisions. In turn, these decisions inform and guide the provisioning actions.

One RSA instance is assigned per managed service, mirroring the granularity of both the MRA and PFA. While the MRA provides the current service state and the PFA anticipates future demand, the RSA acts on these inputs to keep the managed service within safe operational bounds.

The RSA holds the exclusive authority to adjust resource allocation for its assigned service. This exclusivity preserves control stability and prevents conflicts.

The RSA's operational model is deliberately asymmetric and supports three action paths: a proactive resource allocation path triggered by the PFA's anticipated increase in demand, a reactive resource recovery path triggered by sustained under-utilization of allocated resources (low pressure), and an emergency allocation path triggered by unforeseen pressure peak that bypasses deliberation entirely.

BDI Model

The RSA is modeled as a hybrid BDI agent. Its deliberative layer manages the resource allocation path, while its reactive layer guides the resource recovery and emergency allocation paths.

Beliefs: The RSA's beliefs constitute its world model, and they are maintained from two authoritative sources. Service state publications from the MRA capture current resource usage, pressure assessment, and service scale. Forecast outputs from the PFA capture anticipated resource demand, timing indicators, and forecast uncertainty measures. Together, these beliefs give the RSA a coherent and persistent view of both the present operational state and the near future demand outlook.

Desires: The RSA pursues persistent operational goals that reflect its role as the action point of the operational pipeline. These goals are to prevent resource threshold breaches before they occur, respond immediately when unforeseen pressure peaks materialize, recover excess capacity when under-utilization is sustained, and finally, preserve stability and consistency by avoiding oscillation between consecutive actions. All these goals are pursued simultaneously and are mediated by the asymmetric three path operational model.

Intentions: The active commitments of the RSA are adopted in response to its updated beliefs. When a forecast indicates an upcoming breach, the RSA commits to the resource allocation path and, if required, performs a provisioning action. When resources are underutilized for a sustained period of time, the RSA commits to the resource recovery path. In the case of confirmed critical pressure, it commits to the emergency allocation path. Finally, when none of these conditions justify action, the agent explicitly commits to a "do nothing" decision as a deliberate and auditable intention rather than the absence of one.

As a final commitment, the RSA commits to reporting structured failure conditions to the DCA when MRA or PFA publications are absent for a sustained period of time.

Internal Logic

Trigger logic: The RSA accepts exactly three trigger types, and each type corresponds to one of its three operational paths. The first one corresponds to the resource allocation path and is the result of a forecast indicating an upcoming threshold breach within the forecast horizon. The second corresponds to the emergency allocation path and is a response to service state reports confirming critical pressure. The third and final one matches the resource recovery path and is a reaction to service over provisioning, which means that the allocated capacity exceeds the service's needs and resources are being wasted. All other received inputs are used for contextual awareness and coordination, but do not trigger any operational path.

Proactive Resource Allocation Path: When triggered by an anticipated future breach, the RSA performs a structured deliberation process that translates anticipated demand into a target service capacity. The first step of the process is to select the most appropriate forecast projection to use for the current pressure assessment. The next step is to adapt this projection by applying safety margins influenced by forecast uncertainty so that lower confidence leads to a more cautious sizing. The resulting projection is compared against the current allocated capacity across the relevant resources, then the agent decides if the projected gap is large enough to justify a provisioning action. If it is, the RSA computes a target capacity that preserves sufficient headroom across all

considered resources and allocates it after being confined within policy defined operational bounds. Otherwise, if the gap is insufficient, it opts towards a "no action" outcome.

Reactive Resource Recovery Path: This path is activate when a service is overprovisioned for a sustained period of time. In other words, the path triggers when the RSA detects that its assigned service underutilizes its allocated resources, indicating that it holds more capacity than it actually requires. In this path, the agent follows a defined recovery process that derives a reduced capacity target from the current service state. It computes the minimum capacity required to satisfy current resource consumption while maintaining utilization at a defined target threshold for each relevant resource. Then it selects the minimum capacity that satisfies all the considered resources simultaneously. As a final step, the RSA applies policy defined capacity bounds and issues a provisioning action that frees unused resource capacity. The utilization target threshold must preserve sufficient safety headroom without making the recovered capacity negligible.

Reactive Emergency Allocation Path: This path is designed as a final and last resort that the agent takes if the proactive forecasting and allocation mechanisms fail to anticipate, detect, and properly address the pressure surge, whether it materialized in a fast way or accumulated over time. A critical pressure level indicates that the service is on the verge of operational failure. This level is intended to remain unreachable under normal operation because preventing this scenario is the primary objective of the multi-agent system.

As a result, if a service reaches this level, it suggests that the predictive and corrective mechanisms of the MAS were unable to manage the surging resource demand effectively. In this case, the path serves as an emergency safety layer.

Action selection and asymmetric thresholds: The RSA applies asymmetric decision thresholds to reflect the uneven consequences of under-provisioning (capacity shortage) versus over-provisioning (excess allocation).

The resource allocation path follows a comparatively lower decision threshold that expresses a deliberate bias toward early intervention because under-provisioning has a direct and instant impact on service performance, while over-provisioning primarily results in resource inefficiencies and financial cost.

Reducing the allocated resource capacity of the assigned service is restricted to the reactive recovery path and is activated only when resource under-utilization is both apparent and sustained over time and not in conflict with the stability constraint of the agent.

This asymmetry ensures that capacity expansion is proactive and capacity reduction remains deliberately conservative and reactive in order to avoid premature contraction

and oscillatory behavior.

Stability Constraint: To preserve control stability between consecutive scaling decisions, a cooldown mechanism is introduced. Once a capacity expansion action is executed, the agent temporarily suppresses any further non-emergency adjustment to allow time for the executed action to take effect and the service state to settle before another decision is made. This prevents oscillatory behavior during volatile service states.

This mechanism does not concern the emergency allocation path nor is affected by it, because emergency intervention must remain immediate and ready to address any unforeseen and critical pressure surges. Similarly, a "no action" outcome does not interact with the cooldown, because no service capacity change has occurred.

Escalation to the DCA: Escalation is reserved for situations that exceed the handling capacity of the agent. The first situation is when the computed target capacity for a service surpasses the policy defined limit. In this particular situation the agent allocates the allowed capacity and simultaneously emits a structured escalation to the DCA so that the condition is propagated to the supervisory layer (SA) with the necessary context. The second situation represents the scenario where an elevated critical pressure level persists even after the service has already been expanded to its policy defined admissible limit capacity after an emergency path allocation. This indicates that local capacity expansion can no longer resolve the condition; therefore, an escalation to a higher authority is required.

In both situations, escalation is not a request for authorization, but a mechanism for delegating unresolved or structurally constrained situations that lie beyond the RSA's autonomous scope to system layers with a higher authority and broader handling capabilities.

Inputs and Outputs

Inputs: The RSA receives four categories of operational inputs. Its primary operational input includes structured service state updates from the MRA and forecast output publications from the PFA. These inputs provide the necessary information required to execute the three action paths and they are treated as liveness indicators of source agents. As for its secondary operational input, it combines monitoring and forecasting status signals to distinguish temporary faults from actual MRA and PFA failures respectively. In addition, its coordination input from the DCA includes policy notifications and relayed control signals. Finally, its agent failures input consists of confirmed agent failure notification from the DCA, which provides failure awareness and determines the appropriate response action the RSA takes based on which agent failed.

Outputs: The RSA publishes a set of outputs that include: liveness signals, resource adjustment signals, status signals, and peer agent failure reports. Under normal conditions, it emits a periodic liveness signal for supervision and publishes structured resource adjustment signals whenever one of its action paths results in a capacity adjustment. When the RSA faces situations that are outside of its autonomous handling capabilities, it publishes status signals to the DCA. In parallel, it contributes to failure detection as a peer agent by publishing unavailability status signals when the functional publications of either the MRA or the PFA are absent beyond a tolerable period.

Knowledge Base Access

The RSA uses the Knowledge Base as a persistent support layer for policy retrieval and operational traceability.

Domain	Access	Purpose
Policy Domain	Read	Retrieves the operational parameters required to guide sizing and provisioning decisions.
Topology Domain	Read	Verifies that the assigned managed service exists and is valid before entering active operation.
Audit Log Domain	Write	Records the outcome of every operational path, including action and no-action decisions. Ensures that agent behavior remains auditable and traceable over time.

Table 2.6: RSA Knowledge Base Access

Interaction and Coordination

With the MRA: The RSA depends on the MRA’s service state publications to stay updated on the current performance of the service. In addition, these publications are the primary drivers of the reactive paths because every service state publication carries the pressure assessment that influences both the emergency allocation and resource recovery paths.

The RSA interprets the presence of service state publications as a liveness indicator, and it also tracks the status signals to distinguish transient MRA errors from actual agent failures. When the absence of state update publications persists for a sustained period, the agent reports an MRA unavailability status to the DCA and suspends all paths until state publication resumes.

With the PFA: Each forecast output publication provides the estimated future demand, timing indicators, uncertainty measures, and the associated state context that enables the RSA to deliberate without requiring additional cross-referencing. A forecasted breach

within the horizon triggers the proactive allocation path, while the forecast uncertainty influences the size of safety margins. When the PFA enters a degraded state, the RSA suspends the proactive path until forecast publication resumes.

The same failure detection behavior applied to the MRA is also applied to the PFA, where the RSA considers forecast output publication as a liveness indicator, tracks status signals to avoid false alarms, and finally reports a PFA unavailability status to the DCA. The only distinction is that upon detection it suspends the proactive path until publication restarts.

With the DCA: The DCA and RSA interact based on three exchanges. In the first, the RSA receives coordination and control signals. These signals include confirmed agent failure notifications, policy notification signals, and relayed control signals. Second, the RSA issues state signals escalation concerning faults or incidents. Upon inspection, the DCA decides if it can be handled locally or escalation to the SA is required. Third, the RSA contributes to peer agent monitoring by reporting on its peer agents unavailability status. If the RSA fails the DCA informs the dependent agents and coordinates the appropriate recovery response.

2.8.4 Domain Coordinator Agent (DCA)

Role and Positioning

The DCA is a supervision and coordination agent that occupies the lower bound of the coordination and oversight tier, which makes it the intermediary interface between the operational triads within its managed domain and its managing SA.

The DCA's role is to preserve domain coherence and mediate between the operational tier and the higher level supervision bound. These two roles imply two sets of responsibilities: one directed toward the triads of the domain and the other toward the SA. In regard to the triads, the DCA maintains the liveness, coordinates the recovery, and manages the escalations of the agents within these triads. On the other hand, regarding the SA, the DCA relays signals from and to the SA and helps the SA build and sustain its partition wide view by providing periodic domain state reports. The DCA does not participate in the operational decision making and execution. Instead, it operates at a higher architectural level that enables it to fulfill its roles efficiently.

A key property that characterizes the DCA's position is that the triads must remain independent of DCA availability. This enforces the separation between Tier 1 and Tier 2 agents as described in Section 2.5.

BDI Model

The DCA is modeled as a pure BDI agent. It continuously revises its beliefs in response to incoming agent signals, then pursues its desires by adopting intentions appropriate to the received signals. Beliefs are updated synchronously upon signal receipt, before any intention is adopted, ensuring that every decision is grounded in the most current domain state.

Beliefs: The DCA maintains a live domain model that combines the operational status of the agents, current domain pressure, current forecast risk state, recent escalation record, and finally the domain's operation mode. These five structures form the beliefs of the agent and depict an up to date and coherent state of the administrative domain. This depiction constitutes the foundation for all future decisions of the agent.

Desires: This agent pursues two clear and persistent desires. The first one is domain coherence and it means that all the agents are alive and functional, the operational pipeline is uninterrupted, and all incoming signals are handled properly. The second is efficient mediation and it reflects that the DCA relays SA desires to the domain and domain requests to the SA. The only difference is that escalated domain requests are enriched with full domain context to allow better decision making and comprehension while the requests that are duplicates or can be handled locally are filtered and suppressed to ensure that the SA is only contacted when local coordination can no longer resolve the situation appropriately and intervention from a higher authority is necessary.

Intentions: The DCA adopts intentions based on the many signals that originate from its domain. Each signal triggers one specific handling mode, then the DCA commits to the mode's execution until it completes. The DCA possesses four handling modes presented as follows: domain model update, passive monitoring, autonomous action, and mediation. This gives the agent a clear and flexible deliberative structure. Every handling decision produces an audit record regardless of the outcome.

Internal Logic

Core decision cycle: The DCA follows a sequential event driven cycle that handles incoming domain signals one at a time. This preserves the coherence of the domain model by ensuring that two or more updates can not occur at the same time.

For each signal, the DCA starts by updating its beliefs (domain model); then it determines the appropriate handling mode for the signal and commits to it until execution concludes. The decision loop combines four handling modes: domain model update, passive monitoring, autonomous action, and mediation. The domain model update mode

is initiated when the received signal is of the informative type, which means that it reflects normal state updates like the state update publications by the MRA or the forecast output by the PFA. In this case, the DCA just updates its beliefs, and no further actions are required. As for the passive monitoring mode, it is initiated if the signal indicates a low severity or a transient condition. In this scenario the DCA monitors the situation in case of further developments. Regarding the autonomous action mode, it activates when the signal carries a disruption that the DCA recognizes it can resolve. The DCA begins a recovery and coordination attempt to solve the detected disruption. If the agent succeeds, it logs the attempt and moves on to the next signal; otherwise, it escalates the signal with all the necessary information. For critical failures that threaten operational continuity, recovery and escalation may be initiated simultaneously. Lastly, the mediation mode is triggered under two conditions. First, when the signal corresponds to an unresolvable fault or a condition outside local authority. The DCA enriches it with domain context and then escalates it. Second, when the signal originates from the SA. In this situation, the signal is either treated by the DCA or routed to its destination within the domain.

At the end of every cycle, the DCA generates and stores an audit record regardless of whether any external action was taken.

Failure detection: The DCA uses an asymmetric failure detection mechanism to identify agent failures in a reliable way without interfering with the structural independence of the operational pipeline. This mechanism incorporates a dual-layered approach that leverages coordination between two agent layers to prevent false positives and ensure that actual failures are not missed either. The approach is based on two independent yet complementary forms of evidence, one for each layer. The DCA utilizes the continued presence of liveness signals as evidence, while the triads rely on peer agent observations of deterministic operational outputs as their source of evidence.

A set of confirmation rules is applied to both sources by cross validating them against each other to determine whether enough evidence has been corroborated to confirm the failure of the monitored agent. Agents have variable roles and assume different responsibilities, which makes them differ in how observable their behavior is to others. For this specific reason, the DCA does not apply identical confirmation rules to all of them. This represents the asymmetry that was pointed out at the beginning of this section.

Escalation enrichment and filtering: The DCA filters escalations before forwarding any of them to prevent repeated reports of the same ongoing condition from flooding the higher supervision layer. This mechanism suppresses duplicate escalations over a policy defined interval based on the associated service and event. The DCA logs the process for auditing and traceability purposes. Some escalations related to critical conditions are exempted from this mechanism and are always escalated immediately.

The forwarded escalations are enriched with the relevant domain context to ensure that the SA receives actionable and insightful information instead of raw operational signals.

Periodic domain state reports: On top of forwarding escalations, the DCA also provides the SA with a solid view of the administrative domain by summarizing its overall health, coordination state, and the status of the operational triads into a compact periodic domain state report. These reports contribute to sustaining the SA's stable partition wide view during all conditions. The frequency of the reports is defined by policy.

Inputs and Outputs

Inputs:

From MRA: The DCA receives the required signals to keep its domain model up to date and to continue its supervision and coordination responsibilities. The service state update reflects the current operational state of the managed services and updates the DCA's domain pressure view, while the periodic liveness signals provide direct supervision of MRA availability. As for the data collection failure, validation error, and configuration fault signals, they supply the DCA with the necessary information to distinguish transient inconsistencies from sustained faults and conditions that require escalation. All these inputs combined enable the DCA to maintain its domain model, availability of the triads, and finally to determine the appropriate action for each situation.

From PFA : The DCA receives from the PFA forecast output publications to maintain its view of the forecast risk state, periodic liveness signals to supervise PFA availability, and unavailability status reports related to peer agent monitoring. In addition, the forecasting status signals inform the DCA of any degraded forecasting input quality, operational unreadiness, or forecasting computation faults so it can act accordingly. Together, these inputs allow the DCA to preserve forecasting awareness at the domain level and determine how each condition is handled.

From RSA : The DCA receives periodic liveness signals to supervise RSA availability, resource adjustment signals to observe resource adjustment activity, peer agent unavailability status reports, and status signals that are treated locally if possible, otherwise escalated to the SA. These inputs allow the DCA to maintain awareness of sizing and provisioning activities and ensure that each condition is handled appropriately.

From SA: The SA supplies the DCA with policy and control signals. The policy signals inform the DCA if a policy is ready, under revision, or suspended, and specify how the agent should adapt accordingly. On the other hand, the control signals carry instructions that

the DCA must comply with; for example, the instructions may enforce the suspension or resumption of autonomous activities. These two inputs establish the supervisory authority of the SA over the domain and ensure that the DCA and its managed triads remain aligned with current policies and the operational mode.

Outputs:

The DCA produces different outputs that support different system functions. Coordination is supported by policy notifications, confirmed agent failure notifications, and relayed control signals. As for supervision, it includes periodic liveness signals and periodic domain state reports for the SA. The DCA also sends structured and enriched escalation signals to the SA regarding conditions that cannot be handled by the DCA or critical conditions that threaten the system.

Knowledge Base Access

The DCA uses the Knowledge Base as a persistent support layer for policy retrieval, coordination consistency, and decision traceability.

Domain	Access	Purpose
Policy Domain	Read	Retrieves the coordination rules and supervisory constraints that guide DCA behavior
Audit Log Domain	Read	Records every coordination decision to ensure agent behavior remains auditable and traceable over time.
Coordination Domain	Read Write	Maintains persistent coordination state, including agent registration and operational mode, to support continuity across interruptions and recovery.

Table 2.7: DCA Knowledge Base Access

Interaction and Coordination

With the MRA: The DCA relies on the MRA's service state publications as its primary source of service state information within the domain. These publications allow it to maintain a current view of domain pressure and overall operational conditions. The MRA is subjected to the DCA failure detection mechanism where the DCA uses periodic liveness signals as its failure evidence and the peer agents use the presence of publications as theirs. The two sources of evidence are then cross validated against each other based on confirmation rules to determine whether the MRA has failed or not. If the MRA failure is confirmed, the DCA informs the affected peer agents and takes the appropriate action based on the context surrounding the failure. Once MRA operations resume, the restored

service state publications enable a coordinated return to normal operation across the dependent agents.

With the PFA: The DCA is dependent on the PFA’s publications to maintain an up to date view of its domain forecast risk state. These publications provide continuous information about forecast conditions that enable the DCA to track the health of the PFA and detect any degraded forecasting states that may affect normal operation and require its attention. In addition, the PFA is supervised through the same failure detection mechanism applied to the MRA.

With the RSA: The RSA provides the DCA with insight into sizing and provisioning activities. The RSA’s publications enable the DCA to supervise RSA availability, observe resource adjustment activities, and receive relevant signals related to peer agent monitoring tasks, or unresolved operational conditions. When such conditions arise, the DCA determines whether they can be handled locally within the domain or must be escalated to the higher supervision layer. The same failure detection mechanism applied to the MRA and PFA is also applied to detect RSA failures.

With the SA: The DCA serves as the mediator between its domain and its supervising SA. This role allows the SA to maintain a partition wide view through the periodic domain state reports and context enriched escalations provided by the DCA. The DCA receives policy signals and relays SA directives as control signals to the operational layer and ensures that they are enforced. In turn, the SA monitors the DCA’s periodic liveness signals to detect when failure occurs.

2.8.5 Supervision Agent (SA)

Role and Positioning

The Supervision Agent is the direct link between the human operator and the MAS, and it is the highest authority in the agent hierarchy. It operates at the upper bound of the coordination tier, governing a fixed group of assigned Domain Coordinator Agents. One assigned SA instance per group forms a partitioned model that reflects the broader decomposition logic of the architecture where each SA governs its own distinct partition of the cluster, holds exclusive policy authority within it, and operates independently of the other peer SAs.

The SA serves as the bidirectional bridge between the MAS and the human operator. Upward, it aggregates, prioritizes, and presents structured escalations from its assigned DCAs to ensure that operators receive actionable and contextual information instead of low level operational signals. Downward, it translates the intent of the operator (policy

definitions, control commands, and forced actions) into formal directives that propagate through the DCA layer to the operational tier. The SA does not communicate directly with operational agents and instead lets the DCA mediate all interaction between the two tiers. By design, the SA is an oversight and supervision agent, not an operational one.

BDI Model

The SA is modeled as a pure BDI agent. This framing suits its oversight role because the information that reaches the SA has already been enriched and filtered by the DCA. Consequently, the SA receives structured escalations with full domain context that warrant deliberation instead of purely reactive behavior. Full deliberation is therefore both feasible and appropriate for the SA's decision space.

Beliefs: The SA maintains a partition state model assembled from its assigned DCAs. This includes the operational status of each DCA within its partition group, a prioritized escalation queue, and an aggregate pressure and risk view across the supervised domains. The SA also maintains the current active policies as defined by the human operator and preserved in the shared knowledge base.

Desires: The SA pursues two persistent goals. Its primary desire is operator awareness: the agent ensures that the human operator has an accurate, timely, and prioritized view of the supervised domains without being overwhelmed by redundant or irrelevant information. Its secondary desire is policy integrity: the SA makes sure that all the policies defined by the operator are correctly formalized, validated for internal consistency, and propagated to all assigned DCAs before any agent acts on them.

Intentions: The SA's active commitments fall into two categories. When an escalation is received from a DCA the SA classifies it by severity, then adds it to the escalation queue. After that, it derives a set of suggested actions based on the escalation type, domain context, and historical interactions. Finally, the SA presents the escalation to the operator along with the associated context and recommended actions. When the SA receives an operator command, it first validates the command against safety constraints. After validation, the command is routed to the relevant DCA to handle it. The SA does not proactively initiate actions or query agents.

Internal Logic

Escalation handling: The escalations the SA receives from its assigned DCAs are already filtered and enriched before being forwarded. So, the SA's role is triage rather than additional enrichment or filtering. These received escalations are categorized based

on severity, tagged with the SA's recommended actions and queued for presentation based on the assigned priority. The SA assigns the highest priority to critical escalations that can significantly impact the system and then immediately forwards them to the operator. The operator is provided with all relevant information and all available response actions to make an informed decision.

Operator command relay: The SA relays or executes operator directives to translate his intent to the system. The SA supports 4 types of operator directives: The first is policy management, and it allows the operator to add new policies or revise active ones. The second is escalation management, which lets the operator address and handle each escalated signal accordingly. The third one is control action, and it enables the operator to directly impose or reverse an action when conditions require intervention outside the normal autonomous operation. The fourth and final directive is operational mode, which is designed for extreme situations and acts as a system fail-safe where the operator can stop the proactive behavior of the system and enter a purely reactive mode. This mode can be enabled or disabled and it is reserved for unique and rare scenarios. In all cases, the SA validates the requested command against safety constraints before executing and routing it through the DCA layer for enforcement.

DCA liveness supervision: The SA monitors the liveness of each assigned DCA through periodic liveness signals. When this signal is present and received at the expected time intervals, the SA considers the related DCA alive and properly functional. In contrast, if the signal is absent for a sustained period of time, the SA considers that the DCA is in a failed state, marks the associated administrative domain as operating in a degraded supervision conditions, and then notifies the operator. If this situation persists, the SA activates a fallback supervision mode. During this situation, the operational tier continues to function independently without DCA supervision and coordination until the SA fallback mode is established. This fallback only maintains visibility over the domain and only critical faults are presented to the operator. Once the DCA recovers autonomously or through human intervention, normal DCA mediated supervision is restored, and the system returns to its normal operating state.

Cross domain aggregation: The SA receives periodic domain status reports and different kinds of escalations from its assigned DCAs. It aggregates these reports and maps them to their associated escalations in order to form a higher level understanding of pressure, risk, and operational conditions across the supervised domains. This enables the SA to prioritize escalations more effectively, identify broader patterns that extend beyond a single domain, and present the operator with an accurate representative view of the entire partition.

Policy authority: Policies are defined and maintained by the human operator. The SA acts as the MAS gatekeeper before policies are validated and as their enforcer inside the MAS once they are. After each policy addition or revision, the SA validates these policies for internal consistency before saving them in the shared knowledge base and sharing them with its DCAs. If a policy fails validation, the SA rejects it and reports back to the operator with the reasons behind the rejection. This ensures that all lower layers operate under coherent and consistent policies with zero inconsistencies or contradictions.

SA failure and remediation: The SA is a partition level single point of failure, but its impact is constrained by the partitioned architecture and mitigated to some extent through continuity measures built into the system. If the SA fails, its assigned DCAs and their respective domains continue autonomous operation using the latest validated policies. This is acceptable because policy revision is not a frequent task during normal operation.

During SA unavailability, the operator can temporarily interact with the affected partition through its DCAs. In this degraded scenario, the DCAs become the highest active supervision layer and continue to provide escalations and domain state directly to the operator. Although this measure is less filtered and less structured than normal SA mediated supervision, it preserves human intervention capability and supervision continuity until the SA is restored.

Inputs and Outputs

Inputs: The SA receives supervision input from its assigned DCAs and control input from the operator. The supervision input consists of periodic liveness signals, periodic domain state reports, and structured and enriched escalation signals. As for the control input, it includes policy management directives, escalation management directives, control action directives, and operational mode directives that are provided by the operator.

Outputs: In return, the SA provides its assigned DCAs with policy signals that inform them of the policy state and control signals that relay operator escalation, control action, and operational mode directives. Both types of signals serve propagation and enforcement purposes. The SA also sends periodic liveness signals to the operator for direct supervision, provides partition health overviews, and delivers structured and prioritized escalations accompanied by recommended actions.

Knowledge Base Access

The SA uses the Knowledge Base as a persistent support layer for policy management, structural context, and decision traceability.

Domain	Access	Purpose
Policy Domain	Read Write	Keeps the active policies under authority. Enables the SA to retrieve a current policy state and commit validated policy revisions.
Topology Domain	Read	Provides the structural context required to interpret escalations and operator issued directives.
Audit log Domain	Write	Records policy revisions, operator interventions, supervision-mode changes, and significant events to preserve traceability and accountability.

Table 2.8: SA Knowledge Base Access

Interaction and Coordination

With the DCAs: The assigned DCAs represent an interface to the supervised domains. The SA uses this interface to maintain its partition wide view through the reception of periodic domain status reports and enriched escalations from its DCAs. The SA also receives periodic liveness signals from each assigned DCA to supervise their availability and to prevent or adapt to down periods accordingly. In the opposite direction, the SA provides the DCAs with policy state update signals and relays operator directives, which the DCAs are then responsible for enforcing within their corresponding domains. This preserves the separation between supervision and the operational tier by ensuring that the SA can govern the partition without interacting directly with the operational agents.

With the operator: The SA represents the contact point of the operator with the MAS. It presents a structured, prioritized view of partition health to the operator. The SA leverages its DCAs to provide only enriched and filtered escalations that require human judgment to the operator. This operator can interact with the system by sending directives through the SA to express his intents and see them implemented within the system. This relationship preserves human authority and oversight over the system while shielding the operator from low level operational noise and routine autonomous decisions.

With operational agents (Tier 2): The SA has no direct interaction with the operational tier. Instead, it transfers this responsibility to the DCA layer that mediates all supervision over this tier. The DCA acts as the sole coordination interface between higher level supervision and service level operations. This makes sure the SA remains focused on oversight, policy authority, and operator mediation rather than direct operational control, which stays true to the architectural separation between supervision and operation referenced in section 2.5.

2.9 Conclusion

In this chapter, we presented the full conceptual design of the proposed multi-agent system that translates the structural argument of Chapter 1 into a concrete system. Throughout it we showcased our thought process, reasoning, and how we intend to address the identified limitations.

The system is organized into four non-overlapping components, and follows seven key design choices. First, the MAS is distributed across five specialized agents organized in two tiers. Tier 2 forms the operational core through a per-service agent triad (MRA–PFA–RSA) that handles monitoring, forecasting, and provisioning independently of any higher-tier availability. Tier 1 provides coordination and supervision through the DCA and SA without interfering with operational execution. This separation is one of the central structural choices of the design, as it preserves operational continuity even under partial coordination failure.

Second, deploying an operational triad per managed service confines failures, preserves service-level isolation, and aligns each agent with the optimal scope of its abilities. The DCA and SA are scoped to administrative domains and DCA groups to avoid bottlenecks, single points of failure, and unnecessary redundancy. The resulting three-level topology scales horizontally through an alternating expansion pattern that progressively adds agents when needed without restructuring the existing deployment.

Third, the asynchronous broker-mediated publish-subscribe model was selected specifically to preserve the structural independence between tiers, accommodate fan-out requirements, and eliminate temporal coupling between agents operating on independent cycles. A custom lightweight communication language formalizes the semantics of all inter-agent exchanges through six performatives, structured message types, and publication and absence interpretation rules that make agent behavior well-defined and observable.

Fourth, the knowledge base provides durable shared state across five functional domains, each governed by strict role-based access policies that eliminate write conflicts without requiring distributed locking. This persistent layer ensures that agents can recover their operational context after failures and that system behavior remains traceable and auditable throughout.

Fifth, policies are blueprints that guide autonomous behavior across all agents. They are defined by the human operator in alignment with the SLA of each managed service, giving each service a tailored yet globally consistent operational contract. A policy is specified and validated at the supervision layer before being propagated downward to all agents that act on it. This ensures that no agent operates under unvalidated or contradictory constraints, and that behavioral consistency is preserved across all distributed instances without individual reconfiguration.

Sixth, the architecture adopts a human-in-the-loop model where the operator supervises

and manages the system with the minimal intervention necessary, retaining ultimate authority while agents handle all routine decisions autonomously. Intervention is reserved for conditions that exceed agent capabilities, where the operator takes critical decisions based on structured, enriched, and fully contextualized escalations and reports delivered by the SA. These escalations are deliberately designed to be actionable, presenting the operator with the relevant context and recommended actions to facilitate fast and optimal decisions. Four control mechanisms are defined to ensure that human authority remains exercisable at any point and under any condition.

Finally, the behavioral architecture of each agent was defined to reflect its specific operational constraints. The MRA and PFA operate as BDI agents with fixed intention pipelines that preserve deterministic publication cadences. The RSA is designed as a hybrid BDI agent whose deliberative layer governs proactive allocation and whose reactive layer handles emergency responses and resource recovery. The DCA and SA operate as pure BDI agents, where full deliberation is both feasible and appropriate given their position in the system and the enriched information they process.

Combined together, these seven design choices form an architecture whose properties are directly traceable back to the limitations that motivated this work. The conceptual design presented here is, however, only the first step. The following chapter transitions from design to realization, detailing how each architectural component is implemented within a controlled Kubernetes-based environment, covering the cluster infrastructure, agent deployment strategy, communication and storage mechanisms, forecasting pipeline, and the operational parameters governing agent behavior at runtime.

Chapter 3

Implementation and Deployment

Chapter 3

Implementation and Deployment

3.1 Introduction

This chapter focuses on the practical realization of the proposed multi-agent supervision architecture. After presenting the theoretical foundations of proactive resource management and defining the conceptual organization of the MAS in the previous chapters, the objective here is to implement the architecture within a Kubernetes cloud environment.

This chapter presents the experimental environment that was created to support deployment of the architecture, including the Kubernetes cluster topology, workload configuration, monitoring stack, messaging infrastructure, persistent storage mechanisms, and agent deployment strategy. The implementation technologies and runtime policies governing forecasting, scaling, and coordination behavior are then detailed.

By presenting the deployment architecture and operational configuration of the system, this chapter provides the foundation necessary for the experimental evaluation and validation activities presented in Chapter 4.

3.2 Experimental Environment and Infrastructure

The architecture proposed in this thesis is evaluated in a controlled, locally deployed cloud environment designed to reproduce the essential characteristics of an on-premises data center at a reduced scale. A single-host deployment platform was chosen instead of a public cloud deployment for several reasons: it provides full environmental control, eliminates the cost and non-determinism of shared public infrastructure, and provides the reproducible conditions needed to compare different resource management strategies.

3.2.1 Cluster Infrastructure

The experimental cluster is hosted on a single physical machine using a Xubuntu 24.04 LTS virtual machine managed by VMware Workstation. In this environment, the Kubernetes

infrastructure is deployed using k3d v5.8.3, where k3s nodes operate as Docker containers and cluster resources are managed through Kubernetes' declarative configuration model.

k3d was selected because it preserves compatibility with standard Kubernetes deployment workflows while providing a lightweight and reproducible experimental platform. By deploying a k3s cluster using containerized nodes, the environment remains compatible with native Kubernetes APIs, scheduling behavior, Helm-based deployment mechanisms, and RBAC policies. This allows the MAS architecture to be evaluated under conditions that remain structurally close to real Kubernetes production environments. Additionally, declarative cluster configuration ensures that the same cluster setup can be recreated across experimental runs, including node topology, scheduling rules, persistent storage allocation, and exposed services. This improves reproducibility and ensures a consistent infrastructure environment throughout the experiment period.

Another reason for choosing k3d is the resource limitations of the test environment. The available system memory, shared between the Windows host, the VMware hypervisor, and the Xubuntu guest environment, was insufficient to support a typical multi-VM Kubernetes topology, while also supporting the monitoring framework, MAS agents, and prediction tasks. By using k3d's containerized node model, the infrastructure overhead required for the experimental platform was significantly reduced while still preserving the operational behavior necessary for the evaluation process.

On top of this virtualization layer, a k3d cluster running k3s v1.31.5 is deployed across five containerized nodes. The role assignment follows a strict policy of control-plane and data-plane separation: One container runs as a Kubernetes control-plane node, three containers run as worker nodes that contain workloads and MAS agents, and a separate HAProxy loadbalancer container handles port forwarding between the host machine and cluster services. The control plane is isolated from application workloads by applying a `NoSchedule` taint to `server-0` and preventing any workloads from being scheduled on the control plane node. All workloads and MAS agents additionally declare a `requiredDuringSchedulingIgnoredDuringExecution` node affinity constraint targeting the `role=worker` label assigned to the worker nodes. This placement strategy helps maintain a clean separation between orchestration infrastructure and supervised workloads. Figure 3.1 summarizes the resulting topology of the cluster and the placement of its components.

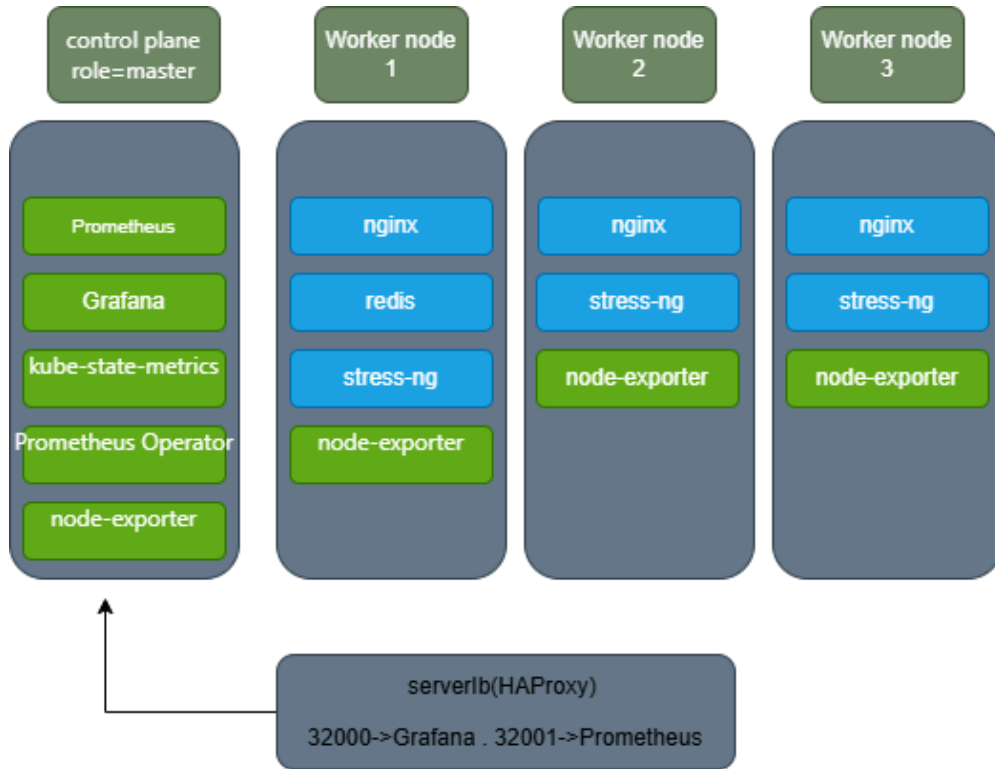


Figure 3.1: k3d Cluster Roles and Components

3.2.2 Workload Profile

The cluster hosts three categories of containerized workload, each selected to exercise a distinct resource consumption pattern and expose the behavioral diversity that the supervision architecture must manage. Together, these workloads introduce variation across resource dimensions, scaling behavior, and temporal utilization patterns, providing a heterogeneous experimental environment for evaluating both reactive and proactive supervision strategies. Table 3.1 summarizes the container image, replica count, primary resource dimension, and resource allocation associated with each workload category.

Workload	Image	Repli- cas	Primary Re- source Dimen- sion	CPU Re- quest / Limit	Memory Request / Limit
Nginx	nginx:stable-alpine	3	CPU-bound, request-driven	50 m / 200 m	32 MiB / 64 MiB
Redis	redis:7-alpine	1	Memory- resident, data- volume driven	50 m / 200 m	32 MiB / 64 MiB
stress-ng	alexexiled/stress-ng:latest	2	Synthetic CPU + memory pressure	100 m / 500 m	96 MiB / 128 MiB

Table 3.1: Workload Configuration

The first workload is a stateless HTTP service, implemented as three replicas of Nginx distributed across the worker nodes. Nginx represents a request-driven, CPU-bound workload whose resource demand scales proportionally with incoming traffic volume. Under stable request rates, resource consumption remains relatively consistent, while burst traffic produces rapid utilization increases. This workload serves as the target for horizontal scaling experiments and for evaluating the reaction latency of the supervision pipeline under changing demand conditions.

The second type of workload will consist of a single instance of Redis running as a memory-based data store. Redis maintains most of the data necessary to provide its services in memory. In comparison to the Nginx workload, Redis will be affected by more data than CPU processing intensity. Thus, memory resource consumption is the primary operational limiting factor for the Redis workload. This distinction allows the forecasting and scaling pipeline to be evaluated across workloads exhibiting different dominant resource dimensions, rather than only CPU-centric utilization behavior.

The third workload consists of two replicas of stress-ng deployed across the worker nodes. During experiments, its load intensity can be adjusted to simulate burst conditions, sustained saturation, and sudden workload changes. Its main purpose in the experiment is to push the system's resource utilization close to the saturation levels, where the timing difference between the reactive alerts and proactive agent-based scaling can be measured.

3.2.3 Monitoring and Observability Layer

Using the Helm chart kube-prometheus stack, we have deployed a dedicated monitoring and observability layer, which includes Prometheus, the Prometheus Operator, Grafana, kube-state-metrics, and node-exporter as a single versioned deployment. This monitoring layer is continuously collecting node-level and pod-level resource metrics and storing them as time-series data that can be accessed by the proposed multi-agent system.

The monitoring components, with the exception of node-exporter, are pinned to the control-plane node (`server-0`), maintaining the administrative-plane isolation discussed in Section 1.1.1. Node-exporter is deployed as a `DaemonSet` without any scheduling constraints and guarantees that CPU, memory, and network metrics are collected from all four hardware nodes at the hardware level. The Grafana interface is available at NodePort 32000 (k3d HAProxy load-balancer container) as shown in Table 3.2.

Component	Deployment Type	Scheduled Node(s)	Scheduling Mechanism
Prometheus	StatefulSet	server-0 (control plane)	nodeSelector + NoSchedule toleration
Grafana	Deployment	server-0 (control plane)	nodeSelector + NoSchedule toleration
Prometheus Operator	Deployment	server-0 (control plane)	nodeSelector + NoSchedule toleration
kube-state-metrics	Deployment	server-0 (control plane)	nodeSelector + NoSchedule toleration
node-exporter	DaemonSet	All 4 nodes	No constraint — DaemonSet by design

Table 3.2: Monitoring component placement and scheduling mechanism

3.3 Implementation Technologies

The implementation of the proposed Multi-Agent System required the integration of multiple technologies spanning container orchestration, distributed monitoring, time-series forecasting, inter-agent communication, persistent storage, and operational visualization.

Technology selection was governed by four primary criteria: compatibility with Kubernetes-native deployment patterns; lightweight execution suitable for a resource-constrained local cluster; support for asynchronous, decoupled agent coordination; and reproducibility of experimental conditions across repeated evaluation runs. This section organizes these tools according to their functional role within the architecture, presenting each as an enabling mechanism for a specific set of architectural objectives.

3.3.1 Agent Runtime and Application Layer

Python is the main programming language for the five agents of the MAS (MRA, PFA, RSA, DCA, and SA) due to its robust ecosystem in machine learning, time-series analysis, distributed systems, and Kubernetes orchestration. Its rapid prototyping capabilities enabled iterative architectural refinement during development, while built-in concurrency mechanisms such as threading, events, and callback-driven execution supported the event-oriented coordination model adopted by the MAS. Additionally, Python’s compatibility with the official Kubernetes Python client and the `paho-mqtt` library allowed native integration with the orchestration infrastructure and the MQTT-based coordination layer.

The *Kubernetes Python client* creates a public-facing API for the RSA to communicate with the orchestration service, allowing it to perform patch operations associated with scaling in response to forecast-driven or pressure-driven decisions. It can trigger controlled pod resets during DCA recovery processes, ensuring that all infrastructure changes are auditable and maintain access control through the Kubernetes API server.

SQLite serves as the primary backend data source for both the Audit Log and Coordination Store in knowledge base domains, operating in WriteAhead Logging (WAL) mode for data persistence. The Audit Log records agent deliberation cycles and scaling events, while the Coordination Store tracks each agent’s cooldown state, heartbeat registrations, and escalation records.

Using *SQLite* eliminates the need for a separate database service, minimizing operational overhead. It effectively meets the moderate coordination and persistence needs of the experimental prototype. Additionally, the database files are mounted via a Kubernetes *PersistentVolume*, ensuring that the coordination state is preserved across pod restarts and supporting recovery mechanisms like the RSA cooldown resume logic.

3.3.2 Containerization and Cluster Orchestration

Docker was used to create isolated and reproducible execution environments for workloads and MAS agents. Each agent is packaged as a unique Docker image through a multi-stage building process that generates a minimal runtime image free of any unnecessary dependencies or development toolchains. This approach ensures that the execution environments for the agents are portable and can be reproduced across recreations of clusters, reducing reliance on the host operating system. Additionally, using separate images to define each agent type preserves the modularity of the architecture and allows for agents to be rebuilt, updated or redeployed independently.

Kubernetes serves as the orchestration layer in the experimental setup, fulfilling three main roles: First, it is the environment where workloads, monitoring components, and MAS agents are scheduled and executed. Second, it is the supervision target of the MAS, being monitored and forecasted in terms of its resource consumption. Finally, it enforces scaling decisions from the RSA by adjusting Kubernetes Deployment replica counts. This integration allows the MAS to operate within the Kubernetes environment rather than as a separate control system. Additionally, namespaces provide logical isolation for workloads and components, while Kubernetes Role-Based Access Control (RBAC) restricts API permissions to ensure that only the RSA can implement scaling changes.

Helm was used to deploy the monitoring and observability stack through the *kube-prometheus-stack* chart. Each of the Helm chart deployment configurations are maintained and controlled via a set of declarative values files that contain scheduling constraints, resource allocations, scrape settings, and component-specific overrides. This approach

allows the monitoring infrastructure to be redeployed consistently across cluster recreations while ensuring that the amount of manual configuration required does not vary between experiments.

3.3.3 Monitoring and Observability

Prometheus serves as the main telemetry collection and time-series storage system in the experimental setup, using a pull-based scrape model to collect container-level CPU and memory metrics through the kubelet’s cAdvisor endpoint and Kubernetes object state information via kube-state-metrics. It gathers data such as replica counts and resource requests/limits. The MRA queries Prometheus every 30 seconds, employing **PromQL** expressions to transform counter and state metrics into a normalized belief object for the MAS pipeline. Prometheus was chosen because of its native integration with Kubernetes and its capabilities in aggregation, rate computation, and label-based metric filtering through **PromQL**.

Grafana serves as the visualization layer in the observability stack, crucial for monitoring workload behavior, verifying metric collection, and observing agent activity during experiments. Custom dashboard panels display CPU and memory utilization trends, workload-pressure classifications, replica changes, and forecast accuracy from the Domain 4 audit log. During experimental validation, Grafana facilitated the identification of temporal correlations between workload-pressure events, PFA inference cycles, and RSA scaling actions, ensuring continuous visual verification of the MAS pipeline’s functionality.

3.3.4 Forecasting and Inter-Agent Coordination

TimesFM 2.5 is a pretrained time-series foundation model developed by Google Research, functioning as the forecasting engine for the PFA. This model uses a decoder-only transformer architecture with patch-based input tokenization to generate direct multi-step forecasts from historical univariate time series data. In addition to point forecasts, TimesFM offers quantile predictions at various confidence levels (P10, P50, P90) over a configurable prediction horizon..

TimesFM was selected primarily because of its zero-shot forecasting capability that eliminates the need for deployment-specific retraining, simplifying operational complexity in machine learning-based forecasting, which often require dataset preparation, retraining procedures, and periodic model maintenance. Within the experimental environment, the PFA uses TimesFM to forecast CPU and memory utilization over a fifteen-minute horizon at thirty-second resolution.

A second factor motivating the selection of TimesFM is its native support for direct multi-horizon forecasting. Unlike recursive autoregressive approaches that repeatedly feed prior predictions back into the model, TimesFM produces the entire prediction horizon

in a single inference step, reducing cumulative forecasting error over longer horizons. Its quantile outputs also estimate forecast uncertainty, helping in conservative scaling under uncertain workload conditions.

MQTT, implemented via the Eclipse Mosquitto broker, serves as the communication backbone for inter-agent coordination within the MAS. MQTT was selected primarily because its lightweight publish-subscribe communication model is well suited to distributed and event-driven systems as it allows asynchronous coordination without persistent connections, unlike synchronous protocols such as REST or gRPC. This decoupled communication model simplifies coordination between the MAS agents while allowing each component to operate independently within the supervision pipeline. The publish-subscribe architecture also supports flexible topic-based routing, enabling agents to subscribe selectively to deployment-specific or system-wide event streams without maintaining explicit knowledge of other participants in the system.

Additionally, MQTT supports configurable Quality-of-Service (QoS) levels. High-frequency, non-critical messages such as heartbeat signals can be transmitted using lower-overhead delivery guarantees, while coordination events associated with scaling actions, escalation notifications, or policy updates can use stronger delivery semantics when reliability is required. These features, along with the Mosquitto broker’s lightweight nature, make MQTT ideal for the resource-constrained experimental environment.

3.4 MAS Deployment Architecture

In Chapter Two, the definition of the multi-agent system described the conceptual architecture at the agent role, the protocol of interaction, and the coordination principles. This section describes how the conceptual architecture was realized within the Kubernetes cluster itself: it describes how the cluster is partitioned to provide the isolation required for agent execution, the method by which agents communicate at runtime, how agent state is preserved across failures, and how scheduling constraints maintain the structural separation of the architecture. Each of these deployment decisions is presented as a concrete realization of a design objective established in Chapter One, demonstrating how architectural requirements are translated into deployment choices.

3.4.1 Namespace Organization

The experimental cluster is divided into three Kubernetes namespaces, each representing a unique functional layer within the system architecture. The use of Kubernetes namespaces provides the boundaries of isolation that can be used to enforce Role-Based Access Control (RBAC), resource management and ownership, and service organization within the experimental environment, making them ideal for separating out the supervised workloads,

monitoring infrastructure and components involved in coordinating the activities of MAS agents.

The monitoring Kubernetes namespace contains a complete deployment of `kube-stack-prometheus`, which includes Prometheus, Grafana, kube-state-metrics, and node-exporter. By isolating the monitoring infrastructure from both the workloads being supervised and the MAS agents that are part of the experimental setup, observable components do not interfere with the supervision of the services being monitored throughout the course of the experiment.

The workloads namespace is composed of the supervised application services, that is nginx, redis, and stress-ng. These workloads create the patterns of resource utilization observed by the MRA, and these patterns are modified indirectly by RSA scaling actions.

The supervisory layer of the architecture is hosted within the MAS-system namespace. This namespace contains all MAS agents, the MQTT broker, and a shared `PersistentVolume` that is used by all Knowledge Base domains. The initialization job for creating SQLite database schemas before agent startup is also in this namespace. By grouping the supervisory components in a dedicated namespace, it simplifies lifecycle management of agents and coordination services, enabling them to be updated or restarted independently from monitored workloads and observability infrastructure. This confines the Kubernetes API permissions granted by namespace-scoped RBAC policies to a narrower operational scope. A summary of the namespaces and their functional roles within the deployment architecture is provided in Table 3.3.

Namespace	Contents	Architectural Role
monitoring	Prometheus, Grafana, kube-state-metrics, node-exporter, Prometheus Operator	Observability infrastructure: isolated from supervised workloads and supervisory agents
workloads	nginx, redis, stress-ng	Supervised infrastructure: the experimental target of the MAS scaling decisions
mas-system	MRA, PFA, RSA, DCA, SA, Mosquitto MQTT broker, shared PersistentVolume, Knowledge Base init Job	Supervisory intelligence layer: isolated from both workloads and monitoring infrastructure

Table 3.3: Kubernetes namespace organization and architectural role of each partition

Figure 3.2 provides a complete architectural view of the realized system, showing the placement of all components across the three namespaces, the full set of inter-agent interaction flows, and the Knowledge Base domains that underpin coordination and evaluation traceability. Each of the following subsections elaborates a specific region of

this architecture.

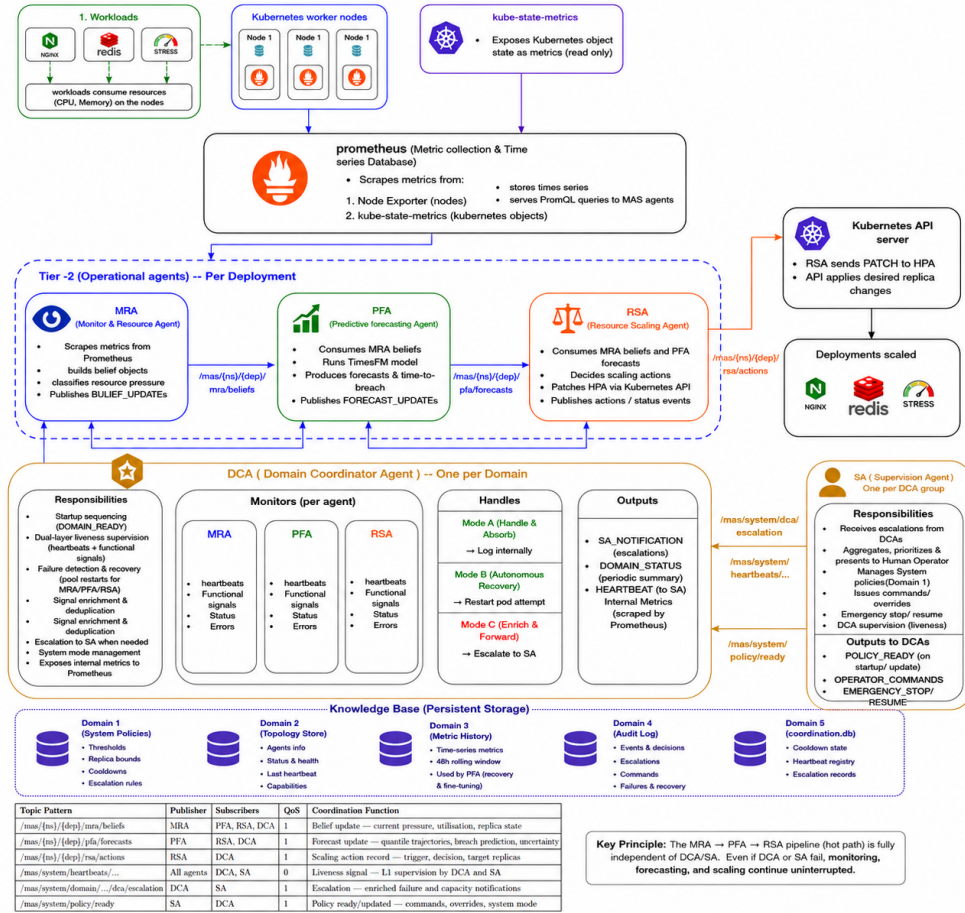


Figure 3.2: Proposed Resource Management Architecture

3.4.2 Messaging Infrastructure and MQTT Broker

Inter-agent communication within the MAS is implemented using MQTT through an Eclipse Mosquitto 2.0 broker that is hosted within the `mas-system` namespace. MQTT provides the asynchronous publish-subscribe communication layer through which monitoring events, forecasts, scaling decisions, heartbeat messages, and coordination signals are exchanged between agents during runtime.

Using the publish-subscribe design pattern, agents can exchange data without having to keep any knowledge of each other's network addresses or execution states. Operational agents communicate with each other via deployment-specific MQTT topics. In this case, the MRA will periodically publish “belief objects,” or descriptions of workload conditions that have been seen; the PFA will consume these observations and publish forecast updates after each internal inference cycle; and the RSA will subscribe to both message streams when doing its scaling deliberation. Any one of these components in the supervision pipeline can operate independently; however, they all remain synchronized through shared event streams.

The MQTT broker also supports the coordination topology as defined in Chapter 2. The DCA subscribes to the same operational topics as a supervisory observer but has no direct role in forwarding belief or forecast messages from one operational agent to another. Consequently, even when the supervisory components may be temporarily down, the communication among the MRA, PFA, and RSA continues to be fully operational.

A hierarchical topic structure represents the namespace, deployment identity, agent role, and event category in a single topic path; allows an agent to subscribe to either deployment-specific events or more general system activity by means of a wildcard topic filter. MQTT QoS levels are configured according to message criticality: low-overhead delivery semantics are used for high-frequency heartbeat traffic and applied to scaling event coordination, escalation records, and policy updates, while stronger delivery guarantees apply to those messages. Table 3.4 presents an overview of the main MQTT topic categories and their functions within the deployment architecture.

Topic Pattern	Publisher	Subscribers	QoS	Coordination Function
/mas/{ns}/{dep}/mra/beliefs	MRA	PFA, RSA, DCA	1	Belief update: current pressure, utilisation, replica state
/mas/{ns}/{dep}/pfa/forecasts	PFA	RSA, DCA	1	Forecast update: quantile trajectories, breach prediction, uncertainty
/mas/{ns}/{dep}/rsa/actions	RSA	DCA	1	Scaling action record: trigger, decision, target replicas
/mas/system/heartbeats/...	All agents	DCA, SA	0	Liveness signal: L1 supervision by DCA and SA
/mas/system/domain/.../dca/escalation	DCA	SA	1	Escalation: enriched failure and capacity notifications
/mas/system/policy/ready	SA	All agents	2	Policy broadcast: threshold and replica-bound configuration

Table 3.4: MQTT coordination topics and communication roles

3.4.3 Persistent Storage and Knowledge Base Realization

The Knowledge Base introduced in Chapter 2 is realized through a combination of Kubernetes `ConfigMaps` and `SQLite` databases operating in Write-Ahead Logging (WAL) mode on a shared `PersistentVolume`. The storage backend for each domain is determined by the persistence and access requirements for the associated category of data. Kubernetes `ConfigMaps` are used to store relatively static configuration information such as policies and deployment topologies while `SQLite` databases are used to store mutable coordination and audit information.

The shared `PersistentVolume` is configured as a Kubernetes `hostPath` volume mapped

to a directory on the Ubuntu host filesystem and mounted on each of the MAS agent pods at `/mnt/kb`. This shared structure allows coordination and audit information produced by one agent to be immediately available for consumption by other agents, regardless of where their pods are scheduled within the cluster. The database files are also directly accessible from the host system during experiments, simplifying inspection and verification of coordination state and audit records.

Persistent storage serves two primary operational roles within the overall architecture:

- **Coordination continuity:** the RSA stores the cooldown state in Domain 5 before issuing scaling-related Kubernetes patch operations, allowing cooldown timers to persist across pod restarts and enabling interrupted scaling cycles to resume with the correct remaining duration.
- **Evaluation traceability:** for every agent deliberation cycle performed, an associated audit entry is created in Domain 4, even for explicit no-action decisions. The audit records provide a permanent, independently verifiable record of forecasting, coordinating, and scaling activity during the experimental evaluation period.

Table 3.5 provides a summary of the persistent storage resources, associated backends, and the operational roles of the resources within the deployment architecture.

Volume / ConfigMap	Backend	Contents	Persistence Purpose
kb-shared-pvc (/mnt/kb/domain4/ audit_log.db)	SQLite WAL on host- Path PV	Audit log (one record per deliberation cycle for all agents)	Full decision history sur- vives pod restarts; thesis evaluation evidence
kb-shared-pvc (/mnt/kb/domain5/ coordination.db)	SQLite WAL on host- Path PV	Cooldown state, heartbeat registry, escalation records, veto queue	RSA cooldown persists across crashes; prevents post-restart oscillation
mas-policy (ConfigMap)	Kubernetes etcd (3-way replicated)	SLA thresholds, replica bounds, cooldown dura- tions	Policy authority persists; agents reload POLICY_- READY without restart
mas-topology (ConfigMap)	Kubernetes etcd (3-way replicated)	Deployment inventory, per- replica resource requests and limits	RSA sizing pipeline reads topology without coupling to DCA availability
hf-cache-pvc (/mnt/hf-cache)	hostPath PV (5 GiB)	TimesFM 2.5 pretrained model weights (~925 MiB)	Weights downloaded once; PFA restarts load in seconds without network access

Table 3.5: Persistent storage resources, backends, and architectural purpose

3.4.4 TimesFM Integration and Model Cache

The forecasting component of the PFA is based on the pretrained TimesFM 2.5 model, whose weights occupy approximately 925 MiB on disk. Loading these weights directly from the **HuggingFace** Hub during every container startup would introduce unnecessary network dependence and variable initialization delays.

To ensure a more consistent startup experience and reduce deployment overhead, The model’s weights are stored in the `HuggingFace` cache directory on a dedicated `PersistentVolume` mounted into each PFA container instead of being downloaded from the `HuggingFace` Hub during container startup. The model is downloaded only once during the initial deployment and subsequently reused across pod restarts, eliminating repeated network transfers and reducing model initialization time.

Because the cache volume is backed by persistent `hostPath` storage on the Ubuntu host filesystem, the pretrained weights remain available even after cluster teardown and recreation, ensuring a consistent forecasting environment across experimental runs.

3.4.5 Agent Deployment Topology

The MAS deployment topology follows the hierarchical structure defined in Chapter 2, with each architectural layer mapped to a distinct Kubernetes deployment scope. The number of agent instances deployed at each layer corresponds to the operational responsibilities assigned to that agent type.

At the operational tier, an MRA, PFA, and RSA form a triad to manage each Kubernetes workload being operated. In the experimental setup, this results in three independent triads corresponding to the Nginx, Redis, and stress-ng deployments. Each triad is configured through deployment-specific environment variables defining the namespace, deployment identity, and resource characteristics of the workload it supervises. Because of the service-level deployment configuration, all activities associated with monitoring, forecasting, and scaling occur at the service level. As a result, MRA queries metrics associated only with the pods of its assigned workload, the PFA maintains a forecasting window specific to that service, and the RSA applies scaling actions exclusively to the corresponding Kubernetes deployment. Thus, operational agents supervise only the workloads assigned to their own triad and do not directly participate in the monitoring or scaling activities of other services.

Compared to centralized supervision approaches, this topology reduces cross-service coupling within the forecasting and scaling pipeline by maintaining separate monitoring histories and forecasting cycles for each managed workload. Failures within one operational triad are isolated to the corresponding service and do not interrupt supervision across the experimental environment.

At the coordination tier, one DCA instance is deployed for the workloads namespace, supervising the operational triads within its domain. The SA is deployed as a singleton supervisory component responsible for maintaining policy configuration and serving as the primary interface between the MAS and the system operator. Table 3.6 summarizes the deployment granularity, instance count, and operational scope associated with each agent type in the experimental environment.

Agent	Deployment Granularity	Instances	Scope
MRA	One per managed Kubernetes Deployment	3 (nginx, redis, stressng)	Deployment-scoped: monitors exactly one service
PFA	One per managed Kubernetes Deployment	3 (nginx, redis, stressng)	Deployment-scoped: maintains one sliding window per service
RSA	One per managed Kubernetes Deployment	3 (nginx, redis, stressng)	Deployment-scoped: holds exclusive HPA authority over one service
DCA	One per administrative Domain	1 (workloads namespace)	Domain-scoped : supervises all triads in one namespace
SA	Singleton per cluster partition	1 (global)	Partition-scoped: policy authority and human operator interface

Table 3.6: MAS agent deployment granularity and operational scope

3.4.6 Local Image Management and Deployment Pipeline

Isolated container runtimes are used by nodes in a k3d cluster, so the Docker images built on the host don't show up automatically in the container runtime for the k3d cluster, which is running Kubernetes. To ensure consistent deployment of the MAS agent manifests, the `imagePullPolicy` for the agents is set to `Never`; therefore, the images must be manually imported into the cluster before the manifest can be used to deploy the agent.

Each agent deployment follows a reproducible local image pipeline. Agent containers are first built as standalone Docker images and then imported into the k3d cluster nodes before the corresponding Kubernetes deployments are restarted. This allows for every agent instance to run the same version of the image across the entire cluster, while not having to rely on a public registry for the image during the experimentation process.

In addition, using locally imported images will reduce deployment variation by eliminating registry-related network delays while also allowing for deployment behavior to remain self-contained in the experimental environment after an image has been created locally.

3.4.7 Node Affinity and Placement Constraints

Physical separation between workloads, MAS agents, and monitoring infrastructure is enforced through Kubernetes scheduling constraints in addition to namespace isolation. Without explicit placement policies, the Kubernetes scheduler may co-locate workloads, monitoring components, and supervisory agents on the same node, introducing resource

contention that could interfere with experimental observations.

All application workloads and MAS agents are restricted to the three worker nodes through a `requiredDuringSchedulingIgnoredDuringExecution` `nodeAffinity` constraint targeting the `role=worker` label. The monitoring stack is pinned to the control-plane node using a `nodeSelector` targeting `role=master`, combined with a toleration permitting scheduling on the `NoSchedule`-tainted control-plane node.

This placement strategy separates observability infrastructure from the supervised workloads and reduces interference between monitoring activity and workload resource consumption during experiments. Within the experimental environment, it also helps ensure that the CPU and memory utilization attributed to the workloads namespace primarily reflects the behavior of the supervised services rather than co-located monitoring components.

3.4.8 Scheduling Policies and Resource Constraints

In addition to node-level placement constraints, the experimental environment enforces scheduling policies governing resource allocation and workload distribution across the cluster. These policies are intended to reduce scheduling variability and maintain consistent experimental conditions during evaluation.

All workload and MAS agent pods declare explicit CPU and memory resource requests and limits. Resource requests ensure that pods are scheduled only onto nodes with sufficient allocatable capacity, while resource limits define upper bounds on CPU and memory consumption enforced through the Linux `cgroup` mechanism. Within the supervision pipeline, these declared limits are also used as reference values by the MRA when computing resource utilization ratios and pressure classifications, contributing directly to the reproducibility of workload pressure measurements across experiments.

Replicated workloads, including Nginx and stress-ng, additionally declare `topologySpreadConstraints` to limit replica imbalance across the worker nodes. This policy distributes replicas more evenly throughout the cluster and reduces the likelihood of artificially concentrated resource pressure on a single node. The resulting workload placement more closely reflects the distributed scheduling behavior expected in multi-node Kubernetes environments.

Together, these scheduling and resource-allocation policies establish a controlled execution environment in which workload placement, resource consumption, and pressure measurements remain more consistent across repeated evaluation runs. Table 3.7 summarizes the scheduling constraints applied within the cluster and the operational role of each policy.

Constraint Type	Applied To	Mechanism	Target Nodes	Architectural Purpose
NoSchedule taint	server-0 (control plane)	kubectl taint	server-0	Prevents workload pods from consuming control-plane node resources
nodeAffinity: required	nginx, redis, stress-ng, all MAS agents	Required During Scheduling Ignored During Execution on role=worker	Worker nodes	Guarantees workload and agent pods land only on labeled worker nodes
nodeSelector + toleration	Prometheus, Grafana, kube-state-metrics, Prometheus Operator	role=master selector; NoSchedule toleration	server-0	Pins monitoring stack to control plane; isolates it from workload resource contention
topologySpread-Constraints	nginx, stress-ng	Max Skew: 1, Schedule Anyway on kubernetes.io/hostname	Worker nodes	Distributes replicas across worker nodes for realistic multi-node load patterns
Resource requests / limits	All workloads and MAS agents	cgroup enforcement via container runtime	All pods	Provides hard memory and CPU bounds; ensures deterministic resource accounting for evaluation

Table 3.7: Scheduling constraints: type, scope, mechanism, and architectural purpose

3.5 Supervisory Policy and Parameterization

The behavioral correctness of the implemented supervision architecture depends not only on the structural organization of the MAS but also on the operational parameters governing when agents act, how aggressively they respond, and how long after a decision to make a change should an agent wait before intervening again. Although all agents share the same coordination substrate, they operate using distinct timing policies, thresholds, and sizing heuristics that are specific to the cadence and resource availability of their environment. This section documents those parameters as engineering tradeoffs selected to preserve stable and reproducible runtime behavior rather than values derived from formal optimization or control-theoretic guarantees.

3.5.1 Monitoring and Pressure Classification Parameters

The MRA queries Prometheus every thirty seconds, establishing the temporal resolution of the supervision pipeline. This interval was chosen to match the configured Prometheus scrape interval within the monitoring stack, ensuring that each MRA cycle retrieves updated telemetry instead of partially refreshed data from a previous collection window. At this cadence, the system produces two belief updates per minute, providing sufficient

responsiveness to capture workload transitions while avoiding the overhead associated with high-frequency polling in a resource-constrained cluster. All of the downstream components including the sliding window maintained by the PFA, the pressure counters used by the RSA, and the heartbeat supervision logic of the DCA, operate on this cadence. As a result, any modifications to the scrape cycle would necessitate a coordinated change in monitoring mechanisms across several different supervisory components. As such, the 30-second scrape period will serve as the base time reference for the supervision pipeline.

Pressure classification is governed by five utilization thresholds applied against the declared Kubernetes resource limits of each deployment. In the experimental configuration, LOW pressure is triggered when either CPU utilization or memory utilization falls below 30% of the declared resource limit, indicating underutilization of allocated resources. WARNING pressure is triggered when CPU utilization exceeds 62% of the declared CPU limit or when memory utilization exceeds 75% of the declared memory limit. CRITICAL pressure is triggered at 85% CPU or 90% memory utilization. These thresholds can be configured in policy configuration parameters in Knowledge Base Domain 1 and loaded into an MRA at start-up from the mas-policy ConfigMap. The CPU WARNING threshold for stress-ng was changed from the original 70% specification down to 62% during the experiment trial. Stress-ng stabilized at a CPU utilization of 60% to 65% during periods of moderate workloads, which resulted in not reaching a WARNING pressure classification and initiating proactive forecasting. Lowering the WARNING CPU threshold allowed stress-ng to be classified as being under WARNING pressure without needing to change the intensity of the workload. Thus, the thresholds depend on the dynamic resource profile and scaling parameters of a given environment instead of having fixed, universally applicable values.

Low pressure requires ten consecutive observations below the configured LOW threshold before scale-down eligibility is granted, corresponding to approximately five minutes of sustained under-utilization. This extended confirmation period reflects the lower operational risk associated with over-provisioning, where the primary trade-off is increased infrastructure cost. By contrast, premature scale-down actions may lead to under-provisioning, increasing the risk of performance degradation and SLA/SLO violations. The prolonged confirmation requirement ensures that low resource demand is persistent rather than transient before capacity is released.

WARNING pressure is declared immediately after the first observation exceeding the configured WARNING threshold. This design prioritizes early escalation into the proactive forecasting pipeline, allowing the PFA to begin inference as soon as elevated utilization is detected and thereby maximizing the lead time available for preventive scaling. At the WARNING level, oscillation protection is addressed downstream by the RSA cooldown policy and minimum inter-inference gap of the PFA, rather than delayed confirmation at the MRA level.

CRITICAL pressure, by contrast, requires two consecutive observations exceeding the configured CRITICAL threshold before escalation occurs. This confirmation requirement reduces the likelihood of triggering unnecessary emergency scaling actions in response to short-lived utilization spikes or transient scheduling effects near the upper threshold boundary. The asymmetry between WARNING and CRITICAL escalation reflects the distinct operational objectives of the two states: WARNING prioritizes early predictive visibility, whereas CRITICAL prioritizes stability and controlled activation of the emergency scaling path.

3.5.2 Forecast Horizon and Prediction Window

The PFA produces forecasts at thirty-second resolution, providing thirty predicted steps per inference cycle that extend fifteen minutes into the future. The fifteen-minute horizon was based on the provisioning latency of the test environment. The ability to scale a Kubernetes Deployment to add additional replicas in the k3d cluster requires between thirty seconds and two minutes to reach a stable running state, depending on scheduling conditions and the availability of the required images. A fifteen-minute horizon provides sufficient lead time for proactive scaling actions to complete before a predicted breach condition is reached.

Shorter forecasting horizons reduce the practical distinction between proactive and reactive scaling because predicted threshold crossings occur too close to the provisioning delay for preventive action to remain effective. Longer horizons, while providing additional lead time, introduce substantially greater forecasting uncertainty and reduce the reliability of predicted breach timing. Consequently, the selected horizon represents a compromise between the ability to respond proactively and forecast stability based on the operational constraints of the prototype.

Forecasting operates at the same thirty-second resolution as the MRA observation cadence. This alignment allows predicted trajectories and observed utilization values to remain directly comparable throughout the supervision pipeline. After each inference cycle, the PFA retrieves the forecast trajectory in order to cross-examine each forecast sample against the breach thresholds to find the first utilization sample predicted to exceed its configured breach threshold for either CPU or memory. The breach thresholds are derived directly from the configured Kubernetes resource limits and the policy-defined utilization thresholds:

$$\theta_{CPU} = L_{CPU} \cdot \tau_{CPU}, \quad \theta_{MEM} = L_{MEM} \cdot \tau_{MEM}$$

where L denotes the configured resource limit and τ the corresponding utilization threshold loaded from the policy store.

If a threshold crossing is identified, the corresponding forecast step is denoted H^* , and

the estimated time-to-breach is computed relative to the thirty-second sampling cadence:

$$T_{tb} = H^* \times 30$$

This mechanism allows the RSA to reason not only about predicted saturation itself but also about the estimated lead time available before the threshold violation occurs.

The PFA maintains independent sliding windows for CPU and memory utilization, each containing up to 96 observations corresponding to approximately 48 minutes of historical data. The window size was selected to provide TimesFM with sufficient historical context to capture short-term workload transitions while remaining compatible with the model’s patch-based tokenization constraints. Inference is not initiated until at least 32 observations are available, ensuring that the forecasting model receives a complete context sequence during the initial bootstrap phase.

Inference is triggered under two conditions: the arrival of a WARNING-level belief update or the expiration of a periodic fifteen-minute refresh interval since the previous completed inference cycle. The WARNING condition provides the ability to forecast rapidly when pressure is elevated, while the periodic refresh provides the ability to keep trajectory information current during prolonged periods of normal use without requiring continual updates.

3.5.3 Scaling Decision Logic and Cooldown Policy

The RSA applies a sixty-second cooldown period after each scaling action, preventing additional scaling decisions from being issued immediately after a previous intervention. The purpose of this conditional 60-second cooldown period is to ensure that the effects of a scaling action can be evaluated before additional actions are considered during subsequent MRA observation cycles. In essence, if the RSA did not maintain a Cooldown period across multiple scaling events, then it would be possible for the RSA to issue multiple scaling patches in response to the same resource data without the Prometheus metrics reflecting the impact from the previous scale action, leading to unstable scaling oscillation.

This cooldown state is stored in Knowledge Base Domain 5 before the Kubernetes patch operation is applied, ensuring that an interrupted scaling sequence leaves the system in a conservative state following agent restart. During recovery, the RSA computes the time remaining in the Cooldown period by comparing the timestamp of the last scale event against the current system time, allowing the system to be conservative after an interrupted scaling sequence.

The RSA supports three execution paths: proactive scaling, reactive scale-down, and emergency scaling. Emergency scaling is triggered by sustained CRITICAL pressure conditions and immediately scales the deployment to the configured maximum replica count. Unlike standard proactive or reactive scaling actions, emergency scaling does not

start or reset the cooldown timer and maintains the timer’s protective properties through the subsequent recovery and scale-down phases.

The proactive execution path is triggered only when the PFA reports a non-null P50-based time-to-breach estimate within the active forecast horizon. Upon activation, the RSA executes a multi-stage sizing pipeline, translating forecast trajectories into a target replica count. The first stage selects the forecast quantile according to the current pressure state: under NORMAL pressure, sizing operates against the P50 trajectory, while WARNING pressure shifts sizing to the more conservative P90 trajectory.

A confidence score is then derived from the adjusted uncertainty estimate produced by the PFA:

$$\text{confidence} = 1 - \text{uncertainty}_{adj}$$

This confidence value is used to compute a dynamic provisioning margin applied to the forecast demand:

$$\text{margin} = \text{base}_{margin} \times (1 + (1 - \text{confidence}))$$

where the default base margin is 15%, producing an effective sizing margin bounded approximately between 15% and 30%.

The resulting sized demand is evaluated independently for CPU and memory utilization:

$$\text{sized demand} = \text{target demand} \times (1 + \text{margin})$$

Per-resource deltas are then computed relative to the currently allocated deployment capacity:

$$\Delta = \frac{\text{sized demand} - \text{allocated capacity}}{\text{allocated capacity}}$$

A proactive scale-up is triggered only if the delta of at least one resource exceeds the configured +15% threshold. This asymmetric threshold reflects the operational preference for temporary over-provisioning over sustained under-provisioning.

The final target replica count is computed independently for CPU and memory before selecting the larger of the two estimates:

$$N_{target} = \max \left(\left\lceil \frac{\text{sized CPU}}{\text{CPU per replica}} \right\rceil, \left\lceil \frac{\text{sized memory}}{\text{memory per replica}} \right\rceil \right)$$

Minimum and maximum replica bounds are then applied as a final clamp before the Kubernetes scaling patch is issued.

Reactive scale-down follows a separate and more conservative sizing path operating exclusively on current observed utilization from the MRA `BELIEF_UPDATE`. LOW pressure

must remain sustained for ten consecutive observation cycles before reactive scale-down becomes eligible, corresponding to approximately five minutes of stable under-utilization. This temporal guard prevents transient utilization drops following scale-up from immediately triggering compensating scale-down actions.

For each resource, the RSA computes the minimum replica count required to maintain current utilization below the configured target threshold:

$$N_{CPU} = \left\lceil \frac{\text{CPU usage}}{\text{CPU request} \times u_{threshold}} \right\rceil$$

$$N_{MEM} = \left\lceil \frac{\text{memory usage}}{\text{memory request} \times u_{threshold}} \right\rceil$$

The final scale-down target is then computed as:

$$N_{target} = \max(N_{CPU}, N_{MEM})$$

No confidence margin is applied during reactive scale-down because sizing operates directly against current observed demand, with the utilization threshold itself acting as the safety buffer against under-provisioning.

If the computed target does not fall below the current replica count, the RSA emits an explicit do-nothing decision.

If either the proactive or emergency path computes a target exceeding the configured maximum replica count, the RSA immediately clamps the action to the maximum authorized value and concurrently publishes a `CAPACITY_OVERFLOW` escalation to the DCA. Similarly, if `CRITICAL` pressure persists after emergency scaling has already reached the configured maximum replica count, the RSA emits an `EMERGENCY_INSUFFICIENT` signal indicating that available infrastructure capacity has been exhausted.

3.5.4 Forecast Uncertainty and Sizing Margins

TimesFM produces multiple quantile trajectories for each inference cycle, including lower-bound, median, and upper-bound forecasts corresponding to the P10, P50, and P90 quantiles respectively. The P50 quantile is used as the primary indicator of forecast breach detection and estimating time to breach, while the upper quantile is used to characterize potential uncertainty around the forecast trajectory.

RSA deliberation is triggered only when the P50 trajectory predicts a threshold breach within the active forecast horizon. Consequently, proactive scaling decisions remain grounded in the median forecast trajectory rather than pessimistic tail projections, while the wider quantile spread is used exclusively to adjust the sizing margin and quantify forecast uncertainty.

After each successful inference, the PFA computes an uncertainty score per metric as

the normalized spread between the P90 and P10 trajectories relative to the P50 central prediction. This score is scale-agnostic, capturing forecast dispersion relative to the model’s own central estimate. If forward-filled intervals are present in the active window, a proportional penalty (scaling linearly with the forwardfill ratio and bounded to $[0, 1]$) is added to the raw score before it is finalized. The adjusted uncertainty score is embedded in the forecast object and consumed by the RSA to determine the sizing margin applied during replica computation: higher uncertainty produces a wider safety margin, making the scaling decision more conservative. Formally, the raw uncertainty is defined as:

$$\text{uncertainty} = \frac{P90 - P10}{\max(P50, \varepsilon)} \quad (3.1)$$

where ε is a small constant introduced to prevent division instability at near-zero load. If forward-filled observations are present, the adjusted uncertainty is computed as:

$$U_{adj} = \min \left(1, U_{raw} + k \cdot \frac{n_{ff}}{N} \right), \quad (3.2)$$

where n_{ff} is the number of forward-filled observations, N is the window size and k is a tunable penalty coefficient.

In addition to quantifying forecast dispersion, the quantile trajectories provide an estimate of the temporal uncertainty associated with a predicted threshold breach. By examining the P10 and P90 trajectories independently, the PFA can derive a range of plausible breach times bounded by the earliest pessimistic estimate and the latest optimistic estimate. This breach interval is not used directly for scaling decisions but provides additional context regarding the stability and confidence of the forecast trajectory.

The RSA uses this uncertainty score to adjust the sizing margin applied during proactive scaling decisions. The margin is computed through linear interpolation between a minimum value of 15% and a maximum value of 30%, using the score as the interpolation. Forecasts with low uncertainty produce smaller provisioning margins, while forecasts with wider prediction intervals result in more conservative scaling decisions.

This uncertainty-aware sizing policy enables the provisioning strategy to adjust according to the confidence levels of the forecasting engine rather than using a fixed safety margin for provisioning all workload conditions. This mechanism does not assume perfect calibration between quantiles and the actual forecast error distribution; therefore, it will develop a monotonic relationship with increased forecast uncertainty correlating to greater provisioning headroom.

Additional conservatism is introduced when the PFA sliding window contains forward-filled observations resulting from temporarily unavailable Prometheus data. In these cases, the uncertainty can be modified positively in relation to how much of the current active window is made up of inferred data, which may lead the RSA to use wider margins of sizing when producing forecasts from partially completed telemetry historical data sets.

Quantile selection during sizing additionally depends on the current pressure state. Under NORMAL pressure, sizing is performed against the P50 trajectory. Under WARNING pressure, sizing shifts to the full P90 trajectory, producing more conservative provisioning behavior during elevated utilization conditions. This pressure-dependent quantile selection serves as a stabilization heuristic that increases scaling aggressiveness under elevated pressure while avoiding unnecessarily conservative provisioning during nominal operation.

3.6 Conclusion

This chapter presented the implementation of the proposed multi-agent supervision architecture within a Kubernetes-based experimental environment. The deployment topology, communication mechanisms, persistent storage infrastructure, forecasting pipeline, and scaling logic collectively realize the architectural design introduced in Chapter 2 as a concrete and operational system. With the implementation completed and the experimental environment fully specified, the next chapter evaluates the effectiveness of the architecture in terms of proactive scaling behavior, forecasting performance, and supervisory coordination under varying workload conditions.

Chapter 4

Experimental Evaluation and Validation

Chapter 4

Experimental Evaluation and Validation

4.1 Introduction

After presenting the practical realization of the proposed architecture, covering the experimental environment, implementation technologies, deployment configuration, and supervisory parameterization, this chapter focuses on evaluating the operational characteristics of the proposed MAS architecture under controlled Kubernetes workload conditions. The objective is to determine whether the implemented system satisfies the principal objectives established throughout Chapters 1, 2, and 3. This evaluation focuses on validating the architecture’s ability to transition from reactive infrastructure supervision toward proactive resource anticipation through the integration of distributed agents, predictive forecasting, and coordinated scaling decisions.

4.2 Evaluation Objectives

The central research objective of this thesis is not simply to implement another autoscaling mechanism but to investigate whether a predictive multi-agent system approach is capable of initiating resource control activities before actual resource exhaustion becomes a problem. The evaluation focuses on analyzing the system’s temporal properties, the decision-making process of the distributed agents, the integration of predictive behavior, how well the distributed agents work together to accomplish a common goal, and the resilience of the agents under failure conditions rather than limiting the assessment to system throughput metrics alone.

More specifically, the evaluation aims to validate five principal research claims.

- The first objective is to determine whether the architecture is capable of proactive scaling behavior. More specifically, the evaluation examines whether scaling actions

issued by the RSA are triggered while the system remains under WARNING pressure conditions rather than after CRITICAL resource saturation has already occurred.

- The second objective is to validate the operational integration of forecast-driven orchestration within the supervision pipeline. The architecture presented in this thesis relies on a sequential interaction between the MRA, the PFA, and the RSA. The evaluation therefore verifies that observed telemetry is successfully transformed into forecast trajectories, that forecasted breach conditions are correctly interpreted by the RSA, and that Kubernetes scaling actions are ultimately derived from predictive reasoning rather than direct threshold-based reactions alone.
- The third objective is to evaluate whether the proposed architecture provides measurable behavioral differences compared to a traditional reactive autoscaling baseline. The intent of this comparison is not to claim universal superiority under every possible operational condition, but rather to determine whether the proposed architecture demonstrates qualitatively different resource management behavior consistent with the proactive design objectives established earlier in the thesis.
- The fourth objective focuses on assessing the supervisory resilience mechanisms implemented via the Domain Coordinator Agent. The evaluation examines whether the decentralized MAS architecture can effectively detect agent failures, initiate autonomous recovery, and escalate ongoing issues without interrupting the operational supervision pipeline. It specifically validates the distributed coordination mechanisms, the heartbeat supervision model, and the dual-layer failure confirmation strategy presented in Chapter 3.
- The fifth objective measures the practical feasibility of deploying the proposed architecture within a realistic Kubernetes environment. The experiments evaluate both the logical correctness of the architecture and its operational performance under realistic scheduling conditions, container startup delays, workload variation, and resource contention scenarios in a k3d-based cluster environment. This allows the evaluation to determine whether the architecture remains operationally feasible when exposed to the same infrastructure constraints encountered by traditional orchestration systems.

To evaluate these objectives, the chapter introduces two primary experimental scenarios. The first scenario evaluates proactive forecast-driven scaling behavior under a controlled workload ramp and compares the resulting system behavior against a reactive horizontal pod autoscaler baseline. The second scenario evaluates supervisory resilience by intentionally triggering agent failure conditions and observing the response of the DCA coordination layer. Together, these scenarios provide experimental evidence of both the predictive and supervisory aspects of the proposed architecture.

The chapter aims to assess whether the implemented MAS architecture effectively supports that a distributed, forecast-driven supervision system can transition cloud resource management from reactive operation toward a more proactive and anticipatory model.

4.3 The Reactive Baseline: Definition and Justification

4.3.1 What the Baseline Is

The reactive baseline used throughout this evaluation is the Kubernetes Horizontal Pod Autoscaler configured to manage the `stress-ng` deployment within the `workloads` namespace. The HPA was selected because it represents the standard autoscaling mechanism natively provided by Kubernetes and is widely adopted in production container orchestration environments.

The baseline configuration was designed to mirror the operational bounds used by the proposed Multi-Agent System in order to ensure comparability between the two approaches. The HPA was configured with the same replica limits as the RSA, using a minimum replica count of one pod and a maximum replica count of five pods. Scaling decisions were driven exclusively by CPU utilization with a target average utilization threshold fixed at 70%. This value was intentionally aligned with the CPU warning threshold used by the MAS configuration so that both systems would react to equivalent utilization conditions.

4.3.2 Justification of the Baseline Choice

The Kubernetes HPA was selected as the experimental baseline because it constitutes the most representative implementation of reactive autoscaling behavior in Kubernetes environments. In practical deployments, the HPA is frequently used as the default mechanism for dynamic resource scaling and provides an appropriate operational reference against which the proposed architecture can be evaluated. The HPA’s design is purely reactive because it monitors actual usage, calculates a desired replica count based on static targets, and then modifies the deployment [64]. However, there is no mechanism within the HPA for predicting future demand; it has no access to historical time-series data, cannot perform forecasting, and cannot project current patterns beyond the current measurement period.

As a result of these architectural properties, the HPA is the proper methodological comparison to the MAS. It represents the orchestration behavior that would exist in the absence of the predictive forecasting and distributed coordination mechanisms introduced by the proposed MAS architecture. Comparing the MAS against the HPA under equivalent workload conditions allows the evaluation to isolate the operational contribution of

predictive orchestration independently from infrastructure variation.

Alternative baselines, including event-driven autoscalers and custom Prometheus-based scaling mechanisms, were considered during the design of the evaluation methodology. However, these alternatives were excluded because they introduce additional orchestration behaviors that complicate direct interpretation of the predictive contribution introduced by the MAS. Also, such systems are less representative of standard Kubernetes deployments than the native HPA mechanism. The HPA therefore provides the most methodologically appropriate and operationally relevant baseline for the experiments conducted in this thesis.

4.3.3 Reactive Characteristics of the HPA

The HPA operates according to a reactive control model in which scaling actions are triggered only after observed resource utilization exceeds a predefined target threshold. Replica adjustments are computed from the ratio between current observed utilization and the configured target utilization.

The desired replica count is computed according to the proportional control formula defined by Kubernetes [64]:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{desiredMetricValue}}{\text{currentMetricValue}} \right\rceil$$

Under this model, the autoscaler constantly monitors the current state of the system and makes adjustments to the number of replicas in relation to the amount of deviation between the actual utilization of the deployment and the configuration of the deployment. The HPA control loop has no ability to utilize historical trend analysis, predictive forecasting, or estimation of future workloads. As a result, the HPA can only trigger a scaling action after resource utilization has already crossed the configured threshold due to the usage of the currently deployed application.

In the experimental environment used throughout this thesis, the operational response latency of the HPA is also influenced by periodic metric polling intervals, Kubernetes scheduling delays, container startup time, and pod readiness verification [64]. Therefore, the actual response time for scaling must occur after the threshold has already been crossed and after the initialization of the new replicas.

The primary architectural difference between the baseline HPA and the proposed MAS configuration is that the HPA reacts to resource saturation as it relates to the present time, while the MAS attempts to anticipate future saturation conditions through workload forecasting performed by the PFA using TimesFM 2.5. The evaluation presented in the following sections focuses on determining whether this predictive capability produces measurable differences in scaling behavior, temporal responsiveness, and operational resource management.

4.4 Evaluation Methodology

This section describes the methodology used to evaluate the operational behavior of the proposed MAS architecture in a controlled Kubernetes environment. The objective is to ensure reproducibility and comparability of experiments between a reactive baseline and the proactive MAS architecture.

The evaluation was performed in the K3D-based Kubernetes environment presented in Chapter 3. All experimental scenarios were executed under identical infrastructure conditions, using the same cluster topology, workload generator, monitoring stack, and deployment configurations. This ensures that any observed behavioral differences originate primarily from the supervision architecture rather than from environmental variation. The methodology involves controlled workload escalation to assess autoscaling behavior under increasing resource pressure, focusing on scaling actions, forecasting, and coordination during normal and failure conditions.

The evaluation methodology focuses on reproducibility and traceability by analyzing Kubernetes state transitions, Prometheus telemetry, MQTT message flows, SQLite audit records, and agent execution logs collected from the deployed environment. This produces a machine-readable operational record of all experimental runs, allowing detailed analysis of the behavior of each architectural component during live execution.

4.4.1 Baseline Reactive Configuration

The Kubernetes HPA was used as a reactive baseline to manage the stress-ng deployment in the workloads namespace. The baseline configuration was designed to replicate the same operational limits used by the proposed MAS architecture in order to ensure direct comparison between the two supervision strategies.

The HPA was configured with a minimum replica count of one pod, a maximum replica count of five pods, and a target CPU utilization threshold of 70%, aligned with the CPU_WARNING_PCT threshold used by the MAS configuration. Therefore, all scaling decisions were based exclusively on current CPU usage.

Before deploying the HPA, all operational MAS agents (MRA, PFA and RSA) used by the stress-ng deployment were scaled down to zero replicas in order to prevent interference between the reactive and proactive scaling paths. This ensured that all replica modifications during the baseline experiments originated exclusively from the HPA control loop. Figure 4.1 shows the scaling of the MRA, PFA, and RSA agent replicas to zero, confirming that no proactive scaling action could interfere with the reactive baseline experiment.

```

younes@DESKTOP-TT52GDR:~$ kubectl scale deployment mra-stress-ng -n mas-system --replicas=0
deployment.apps/mra-stress-ng scaled
younes@DESKTOP-TT52GDR:~$ kubectl scale deployment pfa-stress-ng -n mas-system --replicas=0
deployment.apps/pfa-stress-ng scaled
younes@DESKTOP-TT52GDR:~$ kubectl scale deployment rsa-stress-ng -n mas-system --replicas=0
deployment.apps/rsa-stress-ng scaled
younes@DESKTOP-TT52GDR:~$ kubectl get deployments -n mas-system \
  -o custom-columns='NAME:.metadata.name,REPLICAS:.spec.replicas' \
  | grep stress-ng
mra-stress-ng      0
pfa-stress-ng      0
rsa-stress-ng      0
younes@DESKTOP-TT52GDR:~$

```

Figure 4.1: MAS Agent Scale-Down Before HPA Baseline.

The DCA and the SA remained active because they do not directly patch workload replica counts and, as a result, do not affect the reactive autoscaling behavior.

The stress-ng deployment was then reset to a stable initial state of two replicas before the introduction of the workload protocol, maintaining identical initial deployment conditions across both the HPA and MAS experiments. Figure 4.2 illustrates the reset of the stress-ng deployment to two replicas, ensuring identical starting conditions before the workload protocol was introduced.

```

younes@DESKTOP-TT52GDR:~$ kubectl scale deployment stress-ng -n workloads --replicas=2
deployment.apps/stress-ng scaled
younes@DESKTOP-TT52GDR:~$ kubectl rollout status deployment/stress-ng -n workloads
deployment "stress-ng" successfully rolled out
younes@DESKTOP-TT52GDR:~$

```

Figure 4.2: Initial State Before HPA Deployment.

Before initiating the workload ramp, Kubernetes Metrics Server availability was verified to ensure that valid CPU telemetry was available to the HPA control loop. CPU metrics for the stress-ng pods were confirmed using Kubernetes metric queries before the deployment of the autoscaler manifest as shown in figure 4.3.

```

younes@DESKTOP-TT52GDR:~$ kubectl top pods -n workloads -l app=stress-ng
NAME                                CPU(cores)   MEMORY(bytes)
stress-ng-79f55cf798-f2sqm          555m         78Mi
stress-ng-79f55cf798-rzm8k          552m         81Mi
younes@DESKTOP-TT52GDR:~$

```

Figure 4.3: CPU Telemetry Verification.

The HPA manifest was deployed via the autoscaling/v2 API with explicitly defined scale-up and scale-down stabilization policies designed to minimize oscillations under transient workload conditions. The scale-up stabilization policy was disabled to allow the HPA to react immediately when the configured threshold was reached, ensuring that no artificial disadvantage was introduced relative to the proactive MAS configuration. In contrast, the scale-down stabilization policy was configured with a 300-second stabilization window to prevent premature reduction of replicas due to short-term fluctuations in resource utilization. Figure 4.4 shows the resulting manifest as deployed, and Figure 4.5 confirms the successful application of the manifest to the cluster.


```

Select younes@DESKTOP-TT52GDR: ~
younes@DESKTOP-TT52GDR:~$ cat > ~/datacenter-lab-main/baseline/hpa-stress-ng.yaml << 'EOF'
> apiVersion: autoscaling/v2
> kind: HorizontalPodAutoscaler
a:
  nam> metadata:
>   name: hpa-stress-ng-baseline
space: w>   namespace: workloads
>   labels:
>     experiment: reactive-baseline
> spec:
>   scaleTargetRef:
>     apiVersion: apps/v1
>     kind: Deployment
: stress>     name: stress-ng
nReplic>     minReplicas: 1
>     maxReplicas: 5
ics:
>   metrics:
>   - type: Resource
>     resource:
>       name: cpu
>       target:
>         t>         type: Utilization
>         averageUtilization: 70
>   behavior:
>     scaleUp:
>       stabilizationWindowSeconds: 0
>       policies:
>       - type: Pods
>         value: 4
>         periodSeconds: 15
>     scaleDown:
>       stabilizationWindowSeconds: 300
>       policies:
>       - type: Pods
>         value: 1
>         periodSeconds: 60
> EOF

```

Figure 4.4: HPA Manifest Creation.

```

younes@DESKTOP-TT52GDR:~$ kubectl apply -f ~/datacenter-lab-main/baseline/hpa-stress-ng.yaml
horizontalpodautoscaler.autoscaling/hpa-stress-ng-baseline created
younes@DESKTOP-TT52GDR:~$ watch -n 10 kubectl get hpa hpa-stress-ng-baseline -n workloads

```

Figure 4.5: HPA Baseline Deployment.

The HPA was monitored under a consistent workload condition before running the experiment. Key metrics, including replica count, CPU and memory utilization, and autoscaling events, were collected through HPA status queries, Kubernetes records, and event logs. This data was used to characterize the steady-state behavior of the reactive baseline for comparison with the performance of the MAS architecture.

After completion of the baseline experiments and telemetry collection, the HPA resource was removed from the cluster, and the **stress-ng** deployment was returned to its baseline operational state before initialization of the MAS evaluation scenarios. Figure 4.6 confirms the deletion of the HPA resource.

```
younes@DESKTOP-TT52GDR:~$ kubectl delete hpa hpa-stress-ng-baseline -n workloads
horizontalpodautoscaler.autoscaling "hpa-stress-ng-baseline" deleted
younes@DESKTOP-TT52GDR:~$
```

Figure 4.6: HPA Baseline Removal.

4.4.2 Proactive MAS configuration

The proactive evaluation experiment was performed using the complete MAS operational pipeline developed in this thesis. Unlike the reactive baseline, where scaling decisions were handled by the Kubernetes HPA, all scaling decisions during this experiment were performed exclusively by the MAS.

Throughout the experiment, no HPA resource was active for the **stress-ng** deployment. Therefore, the only source of replica count modifications was the RSA, ensuring that all observed scaling behavior originated from the MAS decision-making process rather than from concurrent reactive controllers.

Before executing the workload protocol, the operational state of the MAS infrastructure was verified. The MRA, PFA, and RSA associated with the stress-ng deployment were each deployed as a single pod replica and confirmed to be in a healthy, running state. Figure 4.7 shows the corresponding deployment state in the mas-system namespace.

```
younes@DESKTOP-TT52GDR:~$ kubectl get deployments -n mas-system
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
dca-worker	1/1	1	1	18d
mosquitto	1/1	1	1	18d
mra-nginx	0/0	0	0	17d
mra-redis	0/0	0	0	17d
mra-stress-ng	1/1	1	1	18d
pfa-nginx	0/0	0	0	17d
pfa-redis	0/0	0	0	17d
pfa-stress-ng	1/1	1	1	18d
rsa-nginx	0/0	0	0	17d
rsa-redis	0/0	0	0	17d
rsa-stress-ng	1/1	1	1	18d
sa	1/1	1	1	18d

Figure 4.7: MAS Agent Readiness Verification.

Additionally, to preserve experimental validity, the workloads namespace was inspected and confirmed to contain no active HPA resources, eliminating the possibility of interference from external autoscaling mechanisms. Figure 4.8 shows the output of that verification.

```
younes@DESKTOP-TT52GDR:~$ kubectl get hpa -n workloads
No resources found in workloads namespace.
younes@DESKTOP-TT52GDR:~$
```

Figure 4.8: HPA Disabled During MAS Evaluation.

Following this, the monitored stress-ng deployment was reset to have two replicas, which established the same initial workload conditions used in the reactive baseline experiment. Figure 4.9 confirms the successful rollout of the two-replica initial state.

```
younes@DESKTOP-TT52GDR:~$
younes@DESKTOP-TT52GDR:~$ kubectl scale deployment stress-ng -n workloads --replicas=2
deployment.apps/stress-ng scaled
younes@DESKTOP-TT52GDR:~$ kubectl rollout status deployment/stress-ng -n workloads
deployment "stress-ng" successfully rolled out
```

Figure 4.9: Initial State Before MAS Evaluation.

Active Policy Configuration

The MAS’s operational behavior was governed by the policy configurations administered by the SA through the `POLICY_READY` initialization procedure. These policy parameters were retrieved from the Domain 1 policy repository and distributed to all operating agents during their startup.

The configuration of active parameters during the evaluation is captured in the table 4.1.

Parameter	Value	Role
CPU_WARNING_PCT	0.70	MRA pressure classification threshold for WARNING state
CPU_CRITICAL_PCT	0.85	MRA pressure classification threshold for CRITICAL state
MEMORY_WARNING_PCT	0.75	MRA pressure classification threshold for memory WARNING
MEMORY_CRITICAL_PCT	0.90	MRA pressure classification threshold for memory CRITICAL
MIN_REPLICAS	1	RSA replica lower bound, shared with HPA configuration
MAX_REPLICAS	5	RSA replica upper bound, shared with HPA configuration
CPU_REQUEST_PER_REPLICA	100 millicores	Per-pod CPU request used by RSA sizing pipeline
CPU_BREACH_PCT (PFA)	0.70	PFA breach threshold for time-to-breach derivation from forecast trajectories

Table 4.1: MAS policy configuration active during the proactive evaluation experiment

Configured replica bounds were purposively aligned with those used by the HPA baseline (`MIN_REPLICAS=1`, `MAX_REPLICAS=5`) to ensure that differences in observed behaviors could be attributed to the scaling strategy rather than differing capacity limits.

The CPU warning threshold was configured at 70%, while the CRITICAL threshold remained fixed at 85% of the declared CPU limit. This warning threshold was identified during the system-level validation experiments presented in Chapter 3, where the original value of 70% proved unsuitable for the workload characteristics of the experimental cluster. It is also necessary to recognize that the MRA Warning Thresholds and the HPA utilization target are not comparable in any direct manner; the HPA evaluates utilization against the declared resource requests, while the MRA evaluates pressure against the declared resource limits. Also, a WARNING classification alone does not trigger a scaling action within the MAS architecture. Scaling decisions occur only when the PFA predicts a future breach of the configured utilization threshold within the forecast horizon and the RSA determines that corrective action is required.

PFA Sliding Window State at Inference Time

The PFA maintains a sliding observation window filled with successive MRA belief updates received over MQTT. Forecast generation requires a minimum observation window of 32 observations, corresponding to approximately 16 minutes of monitoring data at the configured thirty-second collection interval. During the evaluation run, the first forecast that triggered proactive scaling was generated from a sliding window containing 41 CPU observations and 41 memory observations, as confirmed by the `forecast_published` log event extracted from the PFA execution log. This corresponds to approximately 20.5 minutes of historical workload context.

The resulting forecast predicted a future breach of the configured utilization threshold (`p50_breach=true`), activating the proactive scaling path. The inference process completed in approximately 42 seconds during the first execution cycle, primarily due to model initialization and compilation overhead. Subsequent inference cycles executed considerably faster, typically requiring between 3 and 5 seconds. This behavior is consistent with the performance characteristics observed during the system-level validation experiments presented in Chapter 3. The proactive behavior evaluated throughout this chapter emerges from the interaction between three distinct stages: resource observation by the MRA, future workload projection by the PFA using TimesFM 2.5, and forecast-driven scaling decisions executed by the RSA. Figure 4.10 shows the corresponding `forecast_published` log entry capturing these values.

```

event: "mqtt_subscriptions_registered", "timestamp": "2026-05-24T23:39:30.972502Z"
{"cpu_breach_pct": 0.7, "memory_breach_pct": 0.75, "event": "policy_ready_received", "timestamp": "2026-05-24T23:39:31.037268Z"}
{"model": "timesfm-2.5-200m", "duration_ms": 42121.78, "p50_breach": true, "cpu_window": 41, "mem_window": 41, "event": "forecast_published", "timestamp": "2026-05-24T23:44:18.081171Z"}
{"model": "timesfm-2.5-200m", "duration_ms": 4291.03, "p50_breach": false, "cpu_window": 54, "mem_window": 54, "event": "forecast_published", "timestamp": "2026-05-24T23:50:09.200925Z"}
{"model": "timesfm-2.5-200m", "duration_ms": 3814.4, "p50_breach": false, "cpu_window": 57, "mem_window": 57, "event": "forecast_published", "timestamp": "2026-05-24T23:51:39.180724Z"}
{"signum": 15, "event": "shutdown_signal_received", "timestamp": "2026-05-25T00:06:17.068581Z"}
{"event": "pfa_stopped", "timestamp": "2026-05-25T00:06:17.211171Z"}
{"reason_code": "Normal disconnection", "event": "mqtt_disconnected", "timestamp": "2026-05-25T00:06:18.084709Z"}
younes@DESKTOP-TT526DR:~$

```

Figure 4.10: PFA Forecast Trigger Log.

4.4.3 Workload Protocol

All evaluation scenarios were performed using a controlled workload ramp test protocol applied to the stress-ng deployment running inside the workloads namespace. The workload protocol was intentionally designed to produce progressively increasing CPU pressure rather than abrupt utilization spikes. Both the reactive baseline experiment and the proactive MAS experiment were performed under the same workload progression scenario to ensure direct comparability between the two supervision strategies.

Maintaining an identical workload profile across both experimental runs constitutes a fundamental requirement of the evaluation methodology. Any observed differences in behavior between the HPA base and the MAS design should only be attributed to the design differences and not to any changes in the workload input itself. Accordingly, all replicas, timings, stages, and duration were kept constant throughout the experiment.

The load was created by changing the number of replicas for the stress-ng deployment, using Kubernetes scale operations. Each stress-ng pod executed a fixed CPU stressor configured to continuously consume approximately one logical CPU core, such that increasing the number of replicas produced a predictable increase in aggregate CPU utilization within the cluster. This approach was chosen over external HTTP request generators because request-based workloads introduce additional non-deterministic factors, including network latency, TCP scheduling behavior, and request distribution variability, which are not directly relevant to the resource management objectives of this thesis.

The workload progression began from a stable baseline of two active stress-ng replicas. The protocol then introduced three successive escalation stages separated by fixed waiting intervals allowing the monitoring and forecasting pipeline to observe temporal workload evolution. Replica counts were increased incrementally until the deployment reached a peak load state of five replicas before eventually returning to the original baseline configuration.

Instead of relying on stochastic or bursty workload generators, a controlled stepwise ramp was used to ensure deterministic and reproducible temporal evolution of load. This enables precise attribution of observed behavioral differences to the orchestration logic rather than to variability in workload generation.

This design creates the experimental conditions required to evaluate proactive anticipation. The ramp structure allows the evaluation to determine whether the MAS will

initiate scaling actions while the system remains under **WARNING** pressure conditions rather than after **CRITICAL** saturation has already occurred.

The workload protocol is organized into three phases: an initial steady-state phase, a progressive load increase phase implemented through a three-step replica escalation, and a sustained peak phase followed by a return to baseline.

Table 4.2 summarizes the workload progression used throughout the evaluation.

Phase	stress-ng Replicas	Duration	Purpose
T+0:00: Base-line hold	2	3 minutes	Establish pre-load CPU utilization and replica count baseline for both systems
T+3:00: Ramp step 1	3	2 minutes	Push utilization above WARNING threshold (>70%); both systems should detect elevated pressure
T+5:00: Ramp step 2	4	2 minutes	Increase trend steepness; PFA observes upward trajectory in sliding window
T+7:00: Ramp step 3	5	2 minutes	Maximum load; forces a scaling decision in both systems if not already made
T+9:00: Peak hold	5	3 minutes	Allow both autoscalers to stabilize at peak load; record final replica counts
T+12:00: Load removal	2	3 minutes	Return to baseline; observe scale-down behavior in both systems

Table 4.2: Controlled workload ramp scenario used during evaluation

To guarantee timing consistency between the reactive and proactive experiments, the workload profile was automated using a shell script that encoded the complete scaling sequence and all inter-step waiting intervals. The same script was executed without modification during both experimental runs. This approach eliminated manual timing drift and ensured that both autoscaling systems were evaluated under identical workload evolution conditions.

Figure 4.11 illustrates the workload protocol automation script used during the experiments.


```

younes@DESKTOP-TT52GDR: ~
younes@DESKTOP-TT52GDR:~$ cat > ~/datacenter-lab-main/baseline/load-protocol.sh << 'LPEOF'
> #!/bin/bash
> # Scenario 3 – Shared Load Protocol
> # Usage: bash load-protocol.sh baseline      (Run A)
> #       bash load-protocol.sh mas          (Run B)
>
> RUN_LABEL=${1:-"unlabeled"}
> LOG_FILE=~/.datacenter-lab-main/baseline/load-log-${RUN_LABEL}.txt
> ts() { date '+%Y-%m-%dT%H:%M:%S'; }
>
> echo "$(ts) [${RUN_LABEL}] LOAD_PROTOCOL_START replicas=2" | tee -a "$LOG_FILE"
> kubectl scale deployment stress-ng -n workloads --replicas=2
>
> echo "$(ts) [${RUN_LABEL}] PHASE_BASELINE_HOLD duration=180s" | tee -a "$LOG_FILE"
> sleep 180
>
> echo "$(ts) [${RUN_LABEL}] RAMP_STEP_1 replicas=3" | tee -a "$LOG_FILE"
> kubectl scale deployment stress-ng -n workloads --replicas=3
> sleep 120
>
> echo "$(ts) [${RUN_LABEL}] RAMP_STEP_2 replicas=4" | tee -a "$LOG_FILE"
> kubectl scale deployment stress-ng -n workloads --replicas=4
> sleep 120
>
> echo "$(ts) [${RUN_LABEL}] RAMP_STEP_3 replicas=5" | tee -a "$LOG_FILE"
> kubectl scale deployment stress-ng -n workloads --replicas=5
> sleep 120
>
> echo "$(ts) [${RUN_LABEL}] PEAK_HOLD duration=180s" | tee -a "$LOG_FILE"
> sleep 180
>
> echo "$(ts) [${RUN_LABEL}] LOAD_REMOVAL replicas=2" | tee -a "$LOG_FILE"
> kubectl scale deployment stress-ng -n workloads --replicas=2
> sleep 180
>
> echo "$(ts) [${RUN_LABEL}] LOAD_PROTOCOL_END" | tee -a "$LOG_FILE"
> LPEOF

```

Figure 4.11: Workload Ramp Script.

4.4.4 Comparative Data Collection and alignment process

The comparative evaluation between the reactive HPA baseline and the proactive MAS configuration relies on operational data collected from four independent sources during each experimental run: Prometheus time-series telemetry, Kubernetes event records, the Domain 4 SQLite audit logs, and the workload protocol execution logs generated by the experiment automation scripts. Together these sources provide a complete and auditable record of system behavior throughout the evaluation period and form the basis for the computation of all evaluation metrics defined in Section 4.4.5.

Prometheus Time-Series Extraction

After each experimental run was completed, operational telemetry was extracted from Prometheus using the HTTP query_range API. Two time series were collected for every experiment:

- Total CPU utilization for the stress-ng deployment measured in millicores.

- Number of replicas in the deployment obtained via `kube_deployment_spec_replicas` metric

CPU utilization was calculated using the rate of change of the `container_cpu_usage_seconds_total` counter over a two-minute rolling window and converted to millicores by multiplying the result by 1,000. The number of replicas was obtained from the `kube_deployment_spec_replicas` metric. Each of these queries was executed with a 30-second sampling period, which matched the same observation period for the HPA, giving them the same temporal comparability.

To remove any confusion about when events occurred, we got the start and end time of our query from the protocol execution log. The load automation script records explicit `LOAD_PROTOCOL_START` and `LOAD_PROTOCOL_END` events at the beginning and end of each run. These timestamps were converted to Unix epoch values and supplied directly as the start and end parameters of the Prometheus range queries. Consequently, the telemetry extraction window corresponds exactly to the active experimental period.

Workload Protocol Timeline Alignment

Although the reactive baseline and proactive MAS experiments were executed independently, both runs followed the identical workload protocol described in Section 4.4.3. Direct comparison requires the establishment of a common temporal reference frame.

The workload protocol execution log provides this reference through a sequence of timestamped protocol events that occur in the same order and at the same relative times in both runs. For data analysis purposes, the `LOAD_PROTOCOL_START` event is defined as the temporal origin ((T_0)) of each experiment. All subsequent observations are expressed as elapsed time relative to this origin rather than as absolute timestamps.

This normalization procedure allows the behavior of both autoscaling approaches to be compared directly despite being executed at different calendar times. Differences observed in scaling actions, resource utilization trajectories, or replica allocation patterns can therefore be attributed to architectural characteristics of the evaluated systems rather than to external timing effects.

Domain 4 Audit Log Extraction

MAS-specific evaluation metrics require information that is not available through Kubernetes telemetry alone. For this purpose, the Domain 4 SQLite audit repository was used as the primary source for the forecasting and scaling decision records created by the MAS. This repository stores timestamped records produced by the RSA and PFA during live operation, including proactive, reactive, and emergency scaling actions, forecast publications, and forecast-accuracy assessments. Because these records are written synchronously by the participating agents, they provide sub-second temporal precision and are

unaffected by Prometheus scrape intervals or Kubernetes polling delays. Structured SQL queries were used to query the audit repository for records related to the experimental evaluation period, to filter out the records of interest, and to only include records related to agents and records that occurred within the timestamp range of interest for the monitored deployment of stress-ng.

4.4.5 Metrics Definition

The evaluation methodology relies on six principal operational metrics designed to measure proactive orchestration behavior, scaling responsiveness, resource efficiency, and the accuracy of forecasts. Together, these metrics provide a multi-dimensional evaluation framework capable of analyzing the temporal behavior of the proposed MAS architecture relative to the reactive Kubernetes HPA baseline.

The chosen metrics have been created specifically to capture the benefits and trade-offs associated with predictive autoscaling. Certain metrics evaluate the proactive capability of the architecture itself, while others evaluate the operational cost associated with achieving that proactive behavior. All metrics were computed exclusively from recorded operational data collected during live execution of the experiments, including Kubernetes event logs, Prometheus telemetry, and Domain 4 audit records generated by the MAS agents.

M1 — Proactive Lead Time

$$T_{\text{lead}} = T_{\text{HPA_threshold_crossed}} - T_{\text{MAS_patch_applied}}$$

where:

- $T_{\text{MAS_patch_applied}}$ represents the timestamp of the first proactive scaling action issued by the RSA,
- $T_{\text{HPA_threshold_crossed}}$ represents the timestamp of the first equivalent reactive HPA scaling event.

The metric is expressed in seconds. A positive value indicates that the MAS initiated a scaling action before the reactive HPA threshold was crossed.

This metric constitutes the principal evaluation metric of the thesis because it directly quantifies the temporal advantage introduced by forecast-driven orchestration. A positive proactive lead time demonstrates that the MAS issued a scaling action while the reactive baseline had not yet triggered, validating the anticipatory behavior claimed by the architecture. A proactive lead time of zero or negative implies that the proactive architecture has not acted earlier than the reactive baseline and weakens the claim that proactive scaling can be achieved.

For the MAS experiments, $T_{\text{MAS_patch_applied}}$ was extracted from the RSA `SCALING_ACTION` records stored within Domain 4 audit logs using the `timestamp_ms` field associated with `trigger=PROACTIVE`. The HPA threshold crossing timestamp was extracted from the first `SuccessfulRescale` event recorded during the reactive baseline experiment.

M2 — Scaling Response Latency

$$T_{\text{response}} = T_{\text{new_replica_ready}} - T_{\text{load_step_applied}}$$

where:

- $T_{\text{load_step_applied}}$ represents the timestamp at which a workload escalation step was introduced.
- $T_{\text{new_replica_ready}}$ Represents the first moment at which the newly requested replica count became fully operational.

This metric measures the end-to-end delay between workload escalation and the availability of additional processing capacity. Unlike Metric 1, which only measures how long it takes to make a scaling action, Metric 2 assesses the operational latency that workloads will experience because of current workloads before they have access to additional resources.

For the HPA baseline, the response latency includes threshold crossing detection, HPA polling delay, Kubernetes scheduling latency, container startup time, and pod readiness verification. For the MAS configuration, the metric additionally includes TimesFM inference time and RSA decision processing, although the proactive nature of the architecture may allow scaling actions to occur before a reactive threshold violation is observed.

The timestamp of the workload escalation was derived from standard workload protocol execution logs, whereas the time of pod readiness was derived from the transition of the rollout status of a Kubernetes deployment and Prometheus replica telemetry.

M3 — CRITICAL Pressure Duration

$$D_{\text{critical}} = \sum_{i=1}^N (T_{\text{critical_end},i} - T_{\text{critical_start},i})$$

where each interval i corresponds to a continuous episode during which the monitored deployment remained under **CRITICAL** pressure conditions.

The metric is expressed in aggregate seconds of exposure to sustained high CPU utilization. Within the experimental environment, **CRITICAL** pressure corresponds to resource utilization exceeding 85% of the declared CPU allocation.

This metric evaluates the ability of the supervision architecture to prevent or reduce sustained overload conditions. From an operational perspective, prolonged **CRITICAL**

pressure may correspond to CPU throttling, increased request latency, degraded service responsiveness, or unstable workload behavior in production environments.

CRITICAL duration was computed from the Prometheus CPU utilization time series sampled at thirty-second intervals. For each interval, the observed CPU utilization was normalized relative to the aggregate deployment CPU limit derived from the concurrent replica count. Any sample satisfying:

$$\frac{CPU_{\text{observed}}}{\text{Replicas} \times CPU_{\text{limit}}} \geq 0.85$$

was classified as a **CRITICAL** interval and contributed thirty seconds to the cumulative duration.

M4 — WARNING-to-Scale Delay

$$T_{\text{w2s}} = T_{\text{scale_action}} - T_{\text{first_warning}}$$

where:

- $T_{\text{first_warning}}$ represents the first observed **WARNING** pressure state,
- $T_{\text{scale_action}}$ represents the corresponding scaling action timestamp.

This metric evaluates the delay between the first scaling-relevant observation and the corresponding scaling action. For the MAS configuration, the interval is measured from the first confirmed **WARNING** classification generated by the MRA to the proactive scaling action issued by the RSA. For the HPA baseline, the interval is measured from the first utilization threshold violation detected by the HPA control loop to the resulting scale-out action.

Because the two architectures rely on different trigger conditions, the metric should not be interpreted as a direct numerical comparison. Instead, it characterizes how quickly each architecture responds once its respective scaling condition has been satisfied.

The first **WARNING** sample was identified from the Prometheus utilization time series using the configured warning threshold. Scaling action timestamps were extracted from Kubernetes HPA events for the reactive baseline and from RSA audit records for the MAS configuration.

M5 - Replica-Seconds Overhead

$$RSO = \sum_{t=1}^N (\text{ReplicaCount}_t \times \Delta t)$$

where:

- ReplicaCount_t represents the active deployment replica count during interval t ,

- Δt corresponds to the thirty-second sampling interval used throughout the experiments.

The metric is expressed in replica-seconds and represents the total provisioned capacity allocated over the duration of the experiment.

This metric provides a way to measure how much infrastructure each of the different autoscaling strategies adds in terms of overhead. Proactive systems may provision additional replica resources compared to reactive systems. As a result, proactive systems may introduce greater operational costs and less risk of being overloaded compared to reactive systems. By contrast, despite a higher number of active replica resources being present, reactive systems may physically consume fewer replica seconds than proactive when their workloads are being temporarily under-provisioned.

This metric captures the cost-efficiency trade-off associated with proactive orchestration. Interpreting M5 together with M3 allows the evaluation to assess whether reductions in overload duration were achieved at the expense of excessive over-provisioning.

Replica counts were derived from the Prometheus `kube_deployment_spec_replicas` time series collected during each experimental run.

M6 — Forecast Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{K} \sum_{i=1}^K (y_i - \hat{y}_i)^2}$$

where:

- y_i represents the observed utilization value,
- $\hat{y}_i$ represents the corresponding forecast value generated by TimesFM 2.5,
- K represents the number of evaluated forecast samples.

This metric evaluates the predictive accuracy of the TimesFM forecasting subsystem and is applicable exclusively to the MAS configuration because the HPA baseline produces no workload forecasts.

This metric has two purposes. First, it provides experimental evidence that the forecasting subsystem produces meaningful forecasts rather than arbitrary numbers. Second, it puts the proactive lead-time metric (M1) into context. A large proactive lead time accompanied by poor forecast accuracy would weaken the practical significance of the proactive behavior, whereas a positive lead time combined with low RMSE would demonstrate both anticipatory behavior and forecasting reliability.

Forecast accuracy records were produced automatically by the PFA after each inference cycle and stored within Domain 4 audit logs. CPU RMSE values were measured in millicores, while memory RMSE values were measured in MiB. Final RMSE values were

computed as the mean forecast error across all inference cycles observed during the MAS evaluation window.

Together, these six metrics provide a comprehensive framework for evaluating proactive timing behavior, orchestration responsiveness, overload prevention, forecasting quality, and infrastructure efficiency across both the reactive HPA baseline and the proposed forecast-driven MAS architecture.

4.5 Experimental Run Execution

This section documents the concrete execution of both experimental runs, covering the pre-run system state verification, the monitoring setup used during each run, the workload protocol execution, and the immediate post-run data collection steps. The goal of this section is to have a complete and verifiable account of how the experimental trials were executed, providing a basis to independently validate the reported results.

The two experimental runs were performed in the following sequence. The MAS proactive evaluation (Run B) was executed first because the MAS pipeline agents had already been deployed and the PFA had sufficient information in its sliding window to complete the inference. The reactive HPA baseline evaluation (Run A) was conducted immediately afterward, following a controlled inter-run reset procedure. Both runs used the identical load automation script with no modifications between executions, ensuring that the workload input was structurally equivalent across both systems.

4.5.1 Run B: MAS Proactive Evaluation

Before initiating the load protocol, a complete verification of the MAS pipeline state was performed. All three stress-ng agents were confirmed to be running as single-replica pods in the mas-system namespace. The absence of any HPA resource in the workloads namespace was verified. The stress-ng deployment was confirmed at two replicas with both pods in running state. The MRA was confirmed to be producing belief updates at the thirty-second scrape cadence by inspecting its execution log for recent `cycle_complete` entries and the RSA was confirmed to have reached its operational state by verifying the `rsa_operational` log event. Figure 4.12 and 4.13 shows the MRA and RSA configuration logs confirming their operational states.

```

younes@DESKTOP-TT52GDR:~/datacenter-lab-main$ kubectl logs -n mas-system deploy/mra-stress-ng -f \
> | tee ~/datacenter-lab-main/baseline/mra-log-mas.txt
{"namespace": "workloads", "deployment": "stress-ng", "prometheus": "http://monitoring-kube-prometheus-prometheus.monito
ring.svc.cluster.local:9090", "mqtt_host": "mosquitto.mas-system.svc.cluster.local", "event": "mra_starting", "timestamp
": "2026-05-24T23:10:49.007816Z"}
{"host": "mosquitto.mas-system.svc.cluster.local", "port": 1883, "client_id": "mra-workloads-stress-ng", "event": "mqtt_
connecting", "timestamp": "2026-05-24T23:10:49.010289Z"}
{"port": 8000, "event": "metrics_server_started", "timestamp": "2026-05-24T23:10:49.051676Z"}
{"path": "/mnt/kb/domain4/audit_log.db", "event": "domain4_connected", "timestamp": "2026-05-24T23:10:49.056056Z"}
{"msg": "checking infrastructure", "event": "startup_stage_0", "timestamp": "2026-05-24T23:10:49.056249Z"}
{"event": "mqtt_connected", "timestamp": "2026-05-24T23:10:49.065114Z"}
{"event": "startup_stage_0_mqtt_ok", "timestamp": "2026-05-24T23:10:49.066766Z"}
{"url": "http://monitoring-kube-prometheus-prometheus.monitoring.svc.cluster.local:9090", "event": "startup_stage_0_prom
etheus_ok", "timestamp": "2026-05-24T23:10:49.133618Z"}
{"msg": "waiting for POLICY READY", "event": "startup_stage_1", "timestamp": "2026-05-24T23:10:49.134272Z"}
{"event": "policy_ready_received", "timestamp": "2026-05-24T23:10:49.137202Z"}
{"event": "startup_stage_1_complete", "timestamp": "2026-05-24T23:10:49.137433Z"}

```

Figure 4.12: MRA Status Confirmation.

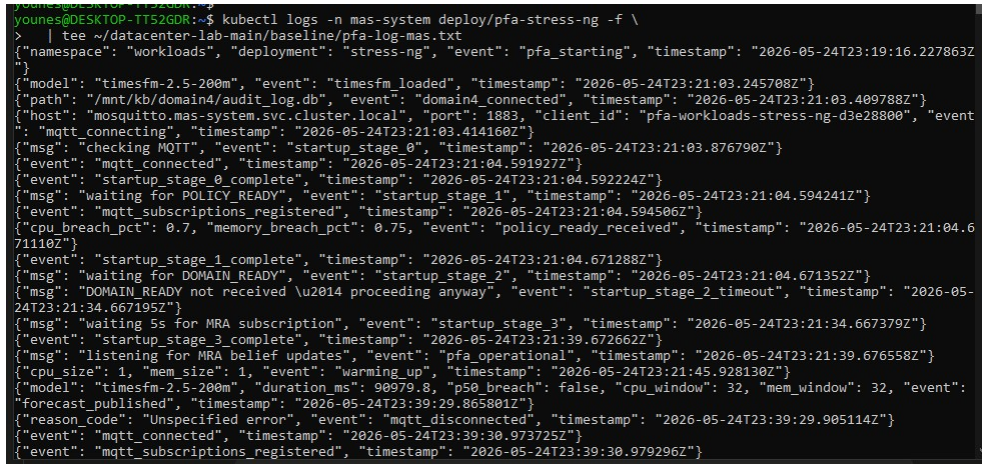
```

younes@DESKTOP-TT52GDR:~/datacenter-lab-main$ kubectl logs -n mas-system deploy/rsa-stress-ng -f \
> | tee ~/datacenter-lab-main/baseline/rsa-log-mas.txt
{"agent_id": "RSA:workloads/stress-ng", "namespace": "workloads", "deployment": "stress-ng", "domain": "worker", "
event": "rsa_starting", "timestamp": "2026-05-24T23:40:20.332305Z"}
{"label": "Domain 4", "path": "/mnt/kb/domain4/audit_log.db", "event": "db_connected", "timestamp": "2026-05-24T23
:40:20.340831Z"}
{"label": "Domain 5", "path": "/mnt/kb/domain5/coordination.db", "event": "db_connected", "timestamp": "2026-05-24
T23:40:20.362348Z"}
{"event": "cooldown_resume_none", "timestamp": "2026-05-24T23:40:20.370437Z"}
{"event": "k8s_config_in_cluster", "timestamp": "2026-05-24T23:40:20.371604Z"}
{"host": "mosquitto.mas-system.svc.cluster.local", "port": 1883, "client_id": "rsa-workloads-stress-ng-14f8c3f5",
"event": "mqtt_connecting", "timestamp": "2026-05-24T23:40:20.380573Z"}
{"event": "mqtt_connected", "timestamp": "2026-05-24T23:40:20.411167Z"}
{"msg": "checking infrastructure", "event": "startup_stage_0", "timestamp": "2026-05-24T23:40:20.430329Z"}
{"event": "startup_stage_0_mqtt_ok", "timestamp": "2026-05-24T23:40:20.431914Z"}
{"event": "mqtt_subscriptions_registered", "timestamp": "2026-05-24T23:40:20.434738Z"}
{"current_replicas": 5, "event": "startup_stage_0_k8s_ok", "timestamp": "2026-05-24T23:40:20.552707Z"}
{"event": "startup_stage_0_domain5_ok", "timestamp": "2026-05-24T23:40:20.553333Z"}
{"msg": "waiting for POLICY READY", "event": "startup_stage_1", "timestamp": "2026-05-24T23:40:20.553805Z"}
{"event": "policy_ready_received", "timestamp": "2026-05-24T23:40:20.635982Z"}
{"keys": ["event_type", "timestamp_ms", "policy_version", "policy"], "event": "policy_ready_received", "timestamp
": "2026-05-24T23:40:20.636110Z"}
{"event": "startup_stage_1_complete", "timestamp": "2026-05-24T23:40:20.638284Z"}

```

Figure 4.13: RSA Status Confirmation.

The most important condition before the run was to determine the size of the sliding window located within the PFA. The PFA has to have published at least one forecast before starting the load protocol in order to verify that the inference pipeline is operational and that TimesFM 2.5 is loaded and compiled. At the time of run initiation, the PFA had published one forecast at 23:39:29 UTC with `cpu_window=32`, confirming that the minimum observation threshold had been reached. By the time the load protocol was initiated at 00:45:59 UTC, the sliding window had grown to 41 observations in total, providing TimesFM with an observational history of 20.5 minutes worth of data. Figure 4.14 shows the PFA log entry confirming this condition was met before the run.



```

younes@DESKTOP-TT52GDR:~$ kubectl logs -n mas-system deploy/pfa-stress-ng -f \
> | tee ~/datacenter-lab-main/baseline/pfa-log-mas.txt
{"namespace": "workloads", "deployment": "stress-ng", "event": "pfa_starting", "timestamp": "2026-05-24T23:19:16.227863Z"}
{"model": "timesfm-2.5-200m", "event": "timesfm_loaded", "timestamp": "2026-05-24T23:21:03.245708Z"}
{"path": "/mnt/kb/domain4/audit_log.db", "event": "domain4_connected", "timestamp": "2026-05-24T23:21:03.409788Z"}
{"host": "mosquitto.mas-system.svc.cluster.local", "port": 1883, "client_id": "pfa-workloads-stress-ng-d3e28800", "event": "mqtt_connecting", "timestamp": "2026-05-24T23:21:03.414160Z"}
{"msg": "checking MQTT", "event": "startup_stage_0", "timestamp": "2026-05-24T23:21:03.876790Z"}
{"event": "mqtt_connected", "timestamp": "2026-05-24T23:21:04.591927Z"}
{"event": "startup_stage_0_complete", "timestamp": "2026-05-24T23:21:04.592224Z"}
{"msg": "waiting for POLICY_READY", "event": "startup_stage_1", "timestamp": "2026-05-24T23:21:04.594241Z"}
{"event": "mqtt_subscriptions_registered", "timestamp": "2026-05-24T23:21:04.594506Z"}
{"cpu_breach_pct": 0.7, "memory_breach_pct": 0.75, "event": "policy_ready_received", "timestamp": "2026-05-24T23:21:04.671110Z"}
{"event": "startup_stage_1_complete", "timestamp": "2026-05-24T23:21:04.671288Z"}
{"msg": "waiting for DOMAIN_READY", "event": "startup_stage_2", "timestamp": "2026-05-24T23:21:04.671352Z"}
{"msg": "DOMAIN_READY not received \u0021 proceeding anyway", "event": "startup_stage_2_timeout", "timestamp": "2026-05-24T23:21:34.667195Z"}
{"msg": "waiting 5s for MRA subscription", "event": "startup_stage_3", "timestamp": "2026-05-24T23:21:34.667379Z"}
{"event": "startup_stage_3_complete", "timestamp": "2026-05-24T23:21:39.672662Z"}
{"msg": "listening for MRA belief updates", "event": "pfa_operational", "timestamp": "2026-05-24T23:21:39.676558Z"}
{"cpu_size": 1, "mem_size": 1, "event": "warming up", "timestamp": "2026-05-24T23:21:45.928130Z"}
{"model": "timesfm-2.5-200m", "duration_ms": 90979.8, "p50_breach": false, "cpu_window": 32, "mem_window": 32, "event": "forecast_published", "timestamp": "2026-05-24T23:39:29.865801Z"}
{"reason_code": "Unspecified error", "event": "mqtt_disconnected", "timestamp": "2026-05-24T23:39:29.905114Z"}
{"event": "mqtt_connected", "timestamp": "2026-05-24T23:39:30.973725Z"}
{"event": "mqtt_subscriptions_registered", "timestamp": "2026-05-24T23:39:30.979296Z"}

```

Figure 4.14: PFA Readiness Confirmation.

Before executing the load protocol script, four parallel processes to collect data were initiated. Each process was executed on an individual terminal window for the total duration of the protocol.

- The first process streamed the MRA execution log in real time and redirected the output to a file on disk. This log captured the complete sequence of scrape cycles, pressure classifications, and validation events produced by the MRA during the run, providing a thirty-second resolution record of the deployment's resource state as observed by the monitoring pipeline.
- The PFA execution log was streamed to provide a record of each inference trigger event, forecast made, and progression of window size throughout the run to provide evidence when the predictive pipeline was created and what actions were produced as a result.
- The RSA execution log was also streamed in real time to provide a record of each deliberation cycle outcome, each scaling action taken, and any event showing that a peer had "gone live". Hence, every decision made in the actuation layer during the run was recorded.
- The fourth process executed a polling loop that sampled the current replica count and aggregate CPU utilization of the stress-ng deployment every fifteen seconds and appended timestamped observations to a file. This timeseries log provides a human-readable record of the deployment state at fifteen-second resolution, complementing the thirty-second Prometheus telemetry.

Every 30 seconds a fifth process has been querying the Prometheus HTTP API for reporting on the total CPU usage of the stress-ng deployment (in millicores), appending

this data to file in a database along with the time it was retrieved. While running in the background as a shell job, this process terminated when the load protocol was complete.

The load protocol was executed using the automation script, which generated the following log file containing protocol timestamp entries. Figure 4.15 shows the execution log generated by the automation script during the MAS run.

```

younes@DESKTOP-TT52GDR: ~
younes@DESKTOP-TT52GDR:~$ echo "MAS_RUN_START: $(date '+%Y-%m-%dT%H:%M:%S')"
MAS_RUN_START: 2026-05-25T00:45:31
younes@DESKTOP-TT52GDR:~$ bash ~/datacenter-lab-main/baseline/load-protocol.sh mas
2026-05-25T00:45:59 [mas] LOAD_PROTOCOL_START replicas=2
deployment.apps/stress-ng scaled
2026-05-25T00:46:00 [mas] PHASE_BASELINE_HOLD duration=180s
2026-05-25T00:49:00 [mas] RAMP_STEP_1 replicas=3
deployment.apps/stress-ng scaled
2026-05-25T00:51:01 [mas] RAMP_STEP_2 replicas=4
deployment.apps/stress-ng scaled
2026-05-25T00:53:01 [mas] RAMP_STEP_3 replicas=5
deployment.apps/stress-ng scaled
2026-05-25T00:55:01 [mas] PEAK_HOLD duration=180s
2026-05-25T00:58:01 [mas] LOAD_REMOVAL replicas=2
deployment.apps/stress-ng scaled
2026-05-25T01:01:02 [mas] LOAD_PROTOCOL_END
younes@DESKTOP-TT52GDR:~$ echo "MAS_RUN_END: $(date '+%Y-%m-%dT%H:%M:%S')"
MAS_RUN_END: 2026-05-25T01:01:38

```

Figure 4.15: MAS Workload Execution Log.

The protocol ran from 00:45:59 UTC to 01:01:02 UTC, a total duration of approximately 15 minutes and 3 seconds. Incremental changes in replica count occurred at 00:49:00, 00:51:01, and 00:53:01 UTC, resulting in a peak of 5 replicas. This peak load was held until 00:58:01 UTC, at which time the load removal process decreased the deployments to 2 replicas. Following the load removal, a further three-minute observation period allowed the two auto-scaling components time to complete their post-load-removal tracking of system performance.

A key scaling event in the MAS run took place at 23:44:19 UTC. The RSA initiated a PROACTIVE SCALE_UP action in response to the PFA forecast p50_breach=true using a 41-observation sliding window used for the initial load condition determination. The RSA execution log captured during this run provides a detailed account of this event. Figure 4.16 shows the RSA log extract capturing this event and the associated decision detail.

```

{"event": "startup_stage_3_complete", "timestamp": "2026-05-24T23:40:50.675004Z"}
{"agent_id": "RSA:workloads/stress-ng", "namespace": "workloads", "deployment": "stress-ng", "event": "rsa_operational", "timestamp": "2026-05-24T23:40:50.675399Z"}
{"consecutive": 1, "required": 2, "event": "critical_pressure_detected", "timestamp": "2026-05-24T23:42:57.388605Z"}
{"raw_target": 19, "target_clamped": 5, "overflow": true, "cpu_replicas": 19, "mem_replicas": 6, "margin_pct": 22.6, "quantile": "P90", "pressure": "WARNING", "event": "pipeline_scale_up", "timestamp": "2026-05-24T23:44:18.861160Z"}
{"computed": 19, "max_replicas": 5, "event": "capacity_overflow", "timestamp": "2026-05-24T23:44:18.866513Z"}
{"event_type": "CAPACITY_OVERFLOW", "event": "rsa_status_published", "timestamp": "2026-05-24T23:44:18.868572Z"}
{"last_action_type": "scale_out", "remaining_seconds": 60.0, "event": "cooldown_persisted", "timestamp": "2026-05-24T23:44:18.938782Z"}
{"action_type": "scale_out", "duration_seconds": 60, "event": "cooldown_started", "timestamp": "2026-05-24T23:44:18.939044Z"}
{"namespace": "workloads", "deployment": "stress-ng", "target_replicas": 5, "event": "k8s_patch_applied", "timestamp": "2026-05-24T23:44:19.186173Z"}
{"trigger": "PROACTIVE", "action": "SCALE_UP", "target": 5, "current": 2, "event": "scaling_action_published", "timestamp": "2026-05-24T23:44:19.187912Z"}

```

Figure 4.16: RSA Scaling Actions Log.

The replica and CPU timeseries log recorded during the run documents the transition from the pre-protocol state through the load ramp and back to baseline, as shown by the following extract. Figure 4.17 shows the timeseries log recorded during the MAS run.

```
younes@DESKTOP-TT52GDR:~/datacenter-lab-main$ head -30 ~/datacenter-lab-main/baseline/mas-timeseries.txt
00:45:53 replicas=5 cpu=1213m
00:46:09 replicas=2 cpu=501m
00:46:25 replicas=2 cpu=738m
00:46:41 replicas=2 cpu=1148m
00:46:57 replicas=2 cpu=1138m
00:47:13 replicas=2 cpu=1118m
00:47:29 replicas=2 cpu=1110m
00:47:45 replicas=2 cpu=1092m
00:48:01 replicas=2 cpu=1115m
00:48:17 replicas=2 cpu=1156m
00:48:33 replicas=2 cpu=1160m
00:48:49 replicas=5 cpu=1126m
00:49:05 replicas=3 cpu=800m
00:49:21 replicas=3 cpu=1093m
00:49:37 replicas=3 cpu=1372m
00:49:53 replicas=3 cpu=1469m
00:50:09 replicas=3 cpu=1415m
00:50:25 replicas=3 cpu=1299m
00:50:40 replicas=3 cpu=1452m
00:50:57 replicas=3 cpu=1381m
00:51:13 replicas=4 cpu=1412m
00:51:29 replicas=4 cpu=1468m
00:51:45 replicas=4 cpu=1451m
00:52:01 replicas=4 cpu=1244m
00:52:17 replicas=4 cpu=1406m
00:52:32 replicas=4 cpu=1464m
00:52:48 replicas=4 cpu=1414m
00:53:04 replicas=5 cpu=1412m
00:53:20 replicas=5 cpu=1491m
00:53:36 replicas=5 cpu=1392m
```

Figure 4.17: MAS Replica Count and CPU Utilization.

Immediately after the load protocol completed, the Prometheus time-series data for the MAS run was collected using the data collection script. The script parsed the `LOAD_-PROTOCOL_START` and `LOAD_PROTOCOL_END` timestamps from the protocol log file, converted them to Unix epoch values, and issued two Prometheus range queries covering the exact experimental window at a 30-second step interval.

4.5.2 Inter-Run Reset Procedure

Once the MAS run and associated data collection were complete, a controlled reset procedure was executed to return the cluster to a state similar to the pre-run conditions present during the MAS experiment before proceeding with the HPA baseline run. The controlled reset ensured that the HPA baseline run was from an initial deployment configuration identical to that of the MAS experiment.

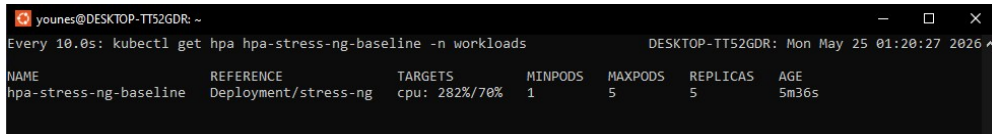
The stress-ng deployment was first scaled to two replicas via an explicit `kubectl scale` command, and a 120-second waiting period was observed to allow CPU utilization to return to the steady-state level of approximately 550 to 580 millicores per pod. This waiting period was verified by checking the Kubernetes Metrics Server. The three MAS operational agents were removed from the cluster by scaling them to 0 replicas, ensuring that no MRA, PFA or RSA activity takes place that could disrupt the HPA baseline run.

4.5.3 Run A: Reactive HPA Baseline Evaluation

The HPA manifest was applied to the workloads namespace and configured with a minimum replica count of one, a maximum of five, and a target CPU utilization of 70% relative to the declared CPU request of 100 millicores per pod, as described in Section 4.4.1.

Once the HPA was created, a brief observation period was observed to allow the Kubernetes HPA controller to begin polling the Metrics Server and to show a valid target metric instead of an unknown target. The HPA status was monitored using a ten-second polling interval until the TARGETS column displayed a numeric utilization percentage.

Prior to HPA intervention, the deployment operated in a steady-state configuration of two replicas consuming approximately 555 millicores per pod. Relative to the configured CPU request of 100 millicores, this corresponds to a utilization ratio of approximately 555%, substantially exceeding the 70% target. Consequently, during its first control-loop execution, the HPA immediately scaled the deployment to the configured maximum of five replicas. When the HPA status was first observed after this scaling action, it reported a CPU utilization of 282% against the 70% target. Although lower than the initial 555% ratio due to the increased replica count, this value still remained far above the configured threshold. Figure 4.18 shows the HPA status output observed at that point.



```

younes@DESKTOP-TT52GDR: ~
Every 10.0s: kubectl get hpa hpa-stress-ng-baseline -n workloads
DESKTOP-TT52GDR: Mon May 25 01:20:27 2026 ^

```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hpa-stress-ng-baseline	Deployment/stress-ng	cpu: 282%/70%	1	5	5	5m36s

Figure 4.18: HPA Status Before Load Injection.

Three parallel data collection processes were started before the load protocol was launched. The first process subscribed to Kubernetes events and filtered them by the HPA resource name and streamed all HPA controller events (specifically SuccessfulRescale events) to a file and to the terminal in real-time. The second process had the same fifteen-second polling loop for both replicas and CPU as was used during MAS run, with outputs written to a separate file so that it could be compared against the MAS timeseries data. The third process ran a Prometheus background logger just like was used during the MAS run.

The load protocol was executed with the same automation script as was used during the MAS run, except only the run label argument was changed from MAS to baseline. The protocol generated the execution log detailed below. Figure 4.19 shows the execution log generated by the automation script during the baseline run.

```

younes@DESKTOP-TT52GDR:~/datacenter-lab-main$ cat ~/datacenter-lab-main/baseline/load-log-baseline-clean.txt
2026-05-25T01:17:51 [baseline] LOAD_PROTOCOL_START replicas=2
2026-05-25T01:17:52 [baseline] PHASE_BASELINE_HOLD duration=180s
2026-05-25T01:20:52 [baseline] RAMP_STEP_1 replicas=3
2026-05-25T01:22:52 [baseline] RAMP_STEP_2 replicas=4
2026-05-25T01:24:53 [baseline] RAMP_STEP_3 replicas=5
2026-05-25T01:26:53 [baseline] PEAK_HOLD duration=180s
2026-05-25T01:29:53 [baseline] LOAD_REMOVAL replicas=2
2026-05-25T01:32:54 [baseline] LOAD_PROTOCOL_END

```

Figure 4.19: HPA Workload Execution Log.

The protocol ran from 01:17:51 UTC to 01:32:54 UTC, a total duration of approximately 15 minutes and 3 seconds, identical in structure to the MAS run. The replica escalation steps were applied at 01:20:52, 01:22:52, and 01:24:53 UTC. The HPA maintained five replicas throughout the entire escalation and peak phases. At the `LOAD_REMOVAL` step at 01:29:53 UTC, the load protocol reduced the deployment to two replicas via a direct scale command. The HPA detected the resulting CPU utilization increase within its next polling cycle and rescaled to five replicas within approximately 18 seconds, as observed in the timeseries log. Figure 4.20 shows the timeseries log recorded during the HPA baseline run, and Figure 4.21 shows the HPA controller events extracted from the Kubernetes event log during the same interval.

```

younes@DESKTOP-TT52GDR:~/datacenter-lab-main$ head -50 ~/datacenter-lab-main/baseline/hpa-timeseries.txt
01:17:38 replicas=5 cpu=1277m
01:17:55 replicas=2 cpu=460m
01:18:11 replicas=5 cpu=465m
01:18:27 replicas=5 cpu=1248m
01:18:44 replicas=5 cpu=1332m
01:19:00 replicas=5 cpu=1342m
01:19:16 replicas=5 cpu=1327m
01:19:32 replicas=5 cpu=1295m
01:19:48 replicas=5 cpu=1408m
01:20:04 replicas=5 cpu=1352m
01:20:20 replicas=5 cpu=1411m
01:20:37 replicas=5 cpu=1286m
01:20:53 replicas=3 cpu=1026m
01:21:10 replicas=5 cpu=714m
01:21:26 replicas=5 cpu=1376m
01:21:43 replicas=5 cpu=1328m
01:22:00 replicas=5 cpu=1082m
01:22:16 replicas=5 cpu=1144m
01:22:33 replicas=5 cpu=1252m
01:22:49 replicas=5 cpu=1329m
01:23:06 replicas=5 cpu=903m
01:23:22 replicas=5 cpu=1362m
01:23:39 replicas=5 cpu=1216m
01:23:55 replicas=5 cpu=1341m
01:24:12 replicas=5 cpu=1272m
01:24:28 replicas=5 cpu=1205m
01:24:45 replicas=5 cpu=1312m
01:25:01 replicas=5 cpu=1374m
01:25:17 replicas=5 cpu=1302m
01:25:33 replicas=5 cpu=1345m
01:25:50 replicas=5 cpu=1291m
01:26:06 replicas=5 cpu=1304m
01:26:22 replicas=5 cpu=1282m
01:26:38 replicas=5 cpu=1287m
01:26:54 replicas=5 cpu=1174m
01:27:10 replicas=5 cpu=1151m
01:27:27 replicas=5 cpu=1395m
01:27:43 replicas=5 cpu=1405m
01:28:00 replicas=5 cpu=1217m
01:28:16 replicas=5 cpu=1360m
01:28:32 replicas=5 cpu=1317m
01:28:49 replicas=5 cpu=1312m
01:29:06 replicas=5 cpu=1302m
01:29:22 replicas=5 cpu=1443m
01:29:38 replicas=5 cpu=1387m

```

Figure 4.20: HPA Replica Count and CPU Utilization.

```

jounes@DESKTOP-TT52GDR:~/datacenter-lab-main$ cat ~/datacenter-lab-main/baseline/hpa-events-describe.txt
Events:
  Type      Reason      Age           From          Message
  ----      -
Normal SuccessfulRescale 7m27s (x5 over 22m) horizontal-pod-autoscaler New size: 5; reason: cpu resource utilization (percentage of request) above target
jounes@DESKTOP-TT52GDR:~/datacenter-lab-main$

```

Figure 4.21: HPA controller events.

At the conclusion of the load protocol, the HPA resource was removed from the workloads namespace and the Prometheus time-series data gathered for the run of the HPA was collected in the same way it was for the MAS run. The data collection script parsed the baseline run protocol log and requested the range queries through the corresponding epoch-aligned time window.

4.5.4 Metric Computation Script and Derivation

The operational metrics defined in Section 4.4.5 were derived using a dedicated Python-based analysis pipeline developed for this evaluation. The script takes Prometheus’s time series data from both test periods combined with Domain 4’s audit log from the MAS agents to create a data set used specifically for this evaluation.

Using the input from all of the audit logs, this pipeline will process and calculate the following data types: CPU utilization, deployment instance counts, RSA scaling actions, and PFA forecast accuracy to derive the quantitative results presented in the subsequent sections. The metric calculations are carried out according to the definitions of the metrics from section 4.4.5, providing evidence of the consistency between the methodology and the measurements presented here.

The script computes:

- M3 (CRITICAL Pressure Duration) by identifying intervals during which CPU utilization exceeded the configured CRITICAL threshold.
- M4 (WARNING-to-Scale Delay) by measuring the elapsed time between the first WARNING condition and the corresponding scaling action.
- M5 (Replica-Seconds Overhead) by integrating deployment replica counts over the duration of each experimental run.
- M6 (Forecast RMSE) by aggregating the forecast accuracy records produced by the predictive forecasting agent.

The remaining metrics, M1 (Proactive Lead Time) and M2 (Scaling Response Latency), were derived from timestamped Kubernetes event records and Domain 4 audit entries, which provide the precise temporal markers required for cross-run comparison.

By automating all calculations from the collected datasets, the analysis pipeline ensures that the reported results are reproducible, auditable, and independent of manual interpretation of raw logs.

4.5.5 Experimental Results and Interpretation

This section presents the results obtained from the reactive HPA baseline and the proactive MAS configuration. The objective is to evaluate whether the proposed architecture achieves the research goals defined in Section 4.2, which were to achieve proactive scaling, forecast-based orchestration, efficient use of resources, and operational resilience through the proposed architecture. Each metric is presented together with its interpretation and its relevance to the thesis claims.

M1 — Proactive Lead Time

As established in Section 4.4.5, M1 is computed as an architectural comparison metric by aligning the timelines of both experimental runs to the common $T=0$ reference point of the `LOAD_PROTOCOL_START` event. In the MAS evaluation run, the proactive scaling action was issued 100 seconds before the load protocol was formally initiated, because the MAS pipeline had been running continuously since the inter-run reset period and had accumulated 41 belief update observations. The MAS issued its first proactive scaling action at 23:44:19 UTC, following a `WARNING`-pressure assessment and a forecast indicating a future breach condition. In contrast, the HPA baseline triggered scaling only after resource utilization had already exceeded its configured threshold.

Relative to the first load escalation step of the workload protocol (`RAMP_STEP_1` at epoch 1779666540), the MAS proactive action preceded this event by 281 seconds. The HPA, in the reactive baseline run, scaled to five replicas immediately upon activation because the two-replica steady-state CPU utilization of approximately 555 millicores per pod against a declared request of 100 millicores produced a utilization ratio of 555% far exceeding the 70% target and triggering an immediate proportional scale to the maximum permitted replica count.

This result provides direct evidence for the primary research claim of the thesis. The proactive scaling action was initiated before the reactive baseline would have triggered under equivalent operating conditions. Rather than responding to an observed threshold violation, the MAS acted on a forecast of future resource pressure generated by the PFA. Consequently, the architecture demonstrated the ability to transform scaling decisions from purely reactive responses into anticipatory control actions.

A limitation of this metric is that the baseline utilization of the stress-ng workload was already relatively high, causing both systems to identify elevated resource pressure early in the experiment. Nevertheless, the difference in trigger conditions remains significant: the MAS acted on predicted future conditions, whereas the HPA acted on currently observed utilization.

M2 — Scaling Response Latency

The MAS required approximately 44 seconds between the workload escalation event and the moment at which the additional replica became operational. The HPA baseline required approximately 18 seconds over the same interval.

At first glance, the HPA appears faster. However, the two systems follow different scaling strategies. The HPA relies on reactive threshold-based scaling, whereas the MAS incorporates workload forecasting and policy evaluation before initiating scale-out operations.

The additional latency observed in the MAS therefore reflects the cost of predictive reasoning rather than inefficiency. The result indicates that proactive decision-making introduces computational overhead, but this overhead is compensated by the substantial lead time advantage demonstrated by M1.

M3: CRITICAL Pressure Duration

```

younes@DESKTOP-TT52GDR:~$ python3 ~/datacenter-lab-main/baseline/compute-metrics.py
=====
M3 - CRITICAL Pressure Duration
[HPA] Total CRITICAL duration: 30s
  epoch=1779669021  cpu=1294.8mc  replicas=2.0  ratio=107.9%
[MAS] Total CRITICAL duration: 210s
  epoch=1779666419  cpu=1065.9mc  replicas=2.0  ratio=88.8%
  epoch=1779666449  cpu=1068.2mc  replicas=2.0  ratio=89.0%
  epoch=1779666479  cpu=1068.1mc  replicas=2.0  ratio=89.0%
  epoch=1779666509  cpu=1120.4mc  replicas=2.0  ratio=93.4%
  epoch=1779666869  cpu=1170.9mc  replicas=2  ratio=97.6%
  epoch=1779667109  cpu=1173.9mc  replicas=2.0  ratio=97.8%
  epoch=1779667139  cpu=1036.6mc  replicas=2.0  ratio=86.4%
M3 Delta (MAS - HPA): 180s (no advantage)

```

Figure 4.22: M3: CRITICAL pressure duration results.

The raw CRITICAL pressure duration values produced by the computation script were 30 seconds for the HPA and 210 seconds for the MAS as shown in figure 4.22. These results require careful interpretation. As stated in the scoped measurement definitions from Section 4.4.5, CRITICAL duration is calculated only during the organic escalation portion of the workload protocol, from `LOAD_PROTOCOL_START` to the `LOAD_REMOVAL` step. When this scoping is applied, no CRITICAL pressure samples were produced by either of these systems during the organic escalation phase of their experimental runs.

All CRITICAL-pressure samples identified in the raw computation occurred after the `LOAD_REMOVAL` phase, during which the workload was manually reduced to two replicas through a direct Kubernetes scale operation. This intervention temporarily overrode the scaling state maintained by the autoscaling mechanisms while the workload continued to generate sustained CPU demand. As a result, aggregate resource consumption briefly exceeded the capacity available at the reduced replica count, producing CRITICAL-pressure observations that were independent of the normal scaling behavior of either

system. The MAS exhibited a longer post-removal recovery period because the RSA emergency path requires consecutive CRITICAL confirmations before corrective action is initiated, whereas the HPA rescaled within approximately 18 seconds of the capacity reduction. Consequently, these CRITICAL episodes reflect the effects of the experimental load-removal procedure rather than the effectiveness of the autoscaling strategies during organic workload escalation.

The result demonstrates that both systems were capable of maintaining service capacity during the planned workload escalation. Most importantly, the MAS achieved this outcome while initiating scaling actions during WARNING conditions, supporting the claim that forecast-driven orchestration can prevent saturation before it occurs.

M4: WARNING-to-Scale Delay

```
=====
M4 - WARNING-to-Scale Time (first WARNING sample timestamps)
[HPA] First WARNING sample at epoch 1779669021  cpu=1294.8mc  ratio=107.9%
[MAS] First WARNING sample at epoch 1779666389  cpu=949.1mc  ratio=79.1%
Subtract T_scale_action from each value to get T_w2s.
T_scale_action (HPA): parse hpa-events-baseline.txt
T_scale_action (MAS): see Domain 4 query below
```

Figure 4.23: M4: WARNING-to-Scale Delay results.

The WARNING-to-Scale delay metric captures the interval between the first confirmed WARNING pressure observation and the corresponding scaling action. The MAS required 26 seconds between the first confirmed WARNING condition and the issuance of the proactive scaling action, while the HPA does not implement a WARNING state and therefore cannot be evaluated using an equivalent definition. Its closest comparable measurement corresponds to the interval between threshold violation and scale-out, which was approximately 18 seconds. Figure 4.23 shows the script output used to derive these measurements.

The significance of M4 is therefore not the absolute numerical comparison but the pressure level at which the action occurs. The MAS converts a WARNING-level condition into a preventive scaling response, whereas the HPA reacts only after utilization has already crossed its operational threshold. This finding supports the proactive nature of the proposed architecture.

M5: Replica-Seconds Overhead

```

=====
M5 - Replica-Seconds Overhead
[HPA] Replica-seconds: 4530 (samples=31, duration=930s)
[MAS] Replica-seconds: 3510 (samples=30, duration=900s)
M5 Delta (MAS - HPA): -1020 replica-seconds (MAS more efficient)

```

Figure 4.24: M5: Replica-Seconds Overhead Results

The replica-seconds overhead metric quantifies the total provisioned capacity consumed by each autoscaling strategy over the duration of its experimental run. The HPA accumulated approximately 4,530 replica-seconds, whereas the MAS accumulated approximately 3,510 replica-seconds over the evaluation interval. This corresponds to a reduction of approximately 22.5% in total resource consumption for the MAS configuration. Figure 4.24 shows the script output from which these values were derived.

The result is particularly noteworthy because proactive autoscaling is often assumed to increase infrastructure cost through early provisioning. In this experiment, the opposite effect was observed. The HPA rapidly scaled to the maximum replica count and remained there for most of the evaluation window, while the MAS allocated capacity more selectively according to forecasted demand. This finding suggests that proactive orchestration can improve responsiveness without necessarily increasing resource consumption.

M6: Forecast RMSE

```

=====
M6 - TimesFM 2.5 Forecast RMSE
Inference cycles recorded: 3
Mean CPU RMSE: 291.84 mc
Mean Memory RMSE: 95.67 MiB
Min CPU RMSE: 171.82 mc
Max CPU RMSE: 476.89 mc
2026-05-24 23:44:18 cpu_rmse=476.887mc mem_rmse=105.888MiB steps=8
2026-05-24 23:50:09 cpu_rmse=171.8241mc mem_rmse=108.7863MiB steps=11
2026-05-24 23:51:39 cpu_rmse=226.794mc mem_rmse=72.3456MiB steps=2

```

Figure 4.25: M6: Forecast RMSE.

Figure 4.25 shows the forecast accuracy records produced by the PFA for each inference cycle.

Three inference cycles were recorded during the MAS evaluation run. The resulting average forecast error was 291.84 millicores for CPU utilization and 95.67 MiB for memory utilization. Forecast accuracy improved as additional observations became available to the model. CPU RMSE dropped sharply from 476.89 to 171.82 millicores, then settled around 226.79 millicores, indicating a marked early improvement followed by a stable error range rather than a strictly monotonic decrease.

These results indicate that TimesFM 2.5 was able to capture the overall workload trend with increasing accuracy as the observation window expanded. Although forecast errors remained non-negligible, the forecasts were sufficiently accurate to correctly identify impending resource-pressure breaches and trigger proactive scaling actions.

It is important to distinguish between forecast accuracy and forecast usefulness. The proactive scaling decision depends primarily on whether the forecast correctly predicts a future threshold breach rather than on exact point-wise prediction accuracy. Consequently, the successful proactive actions observed in M1 demonstrate that the forecasting subsystem provided operationally meaningful predictions despite moderate RMSE values.

Taken together, the results provide experimental support for the central thesis claim. The MAS architecture demonstrated qualitatively different resource management behavior from the reactive HPA baseline, most clearly on the three directly comparable metrics (M1, M2, M5). The proactive lead time confirms that the system initiated scaling actions before the reactive baseline threshold was crossed. The CRITICAL duration result confirms that the system prevented overload during the organic escalation phase. The replica-second result demonstrates that this proactive behavior was achieved without additional infrastructure cost. The forecast RMSE result establishes that the predictive subsystem produced breach classifications that were operationally correct despite the inherent uncertainty of zero-shot inference on short observational windows. These findings collectively validate the operational effectiveness of the proposed architecture within the experimental environment.

Table 4.3 presents a summary of all six metrics along with a brief finding for each.

4.6 DCA Supervisory Resilience Evaluation

This section evaluates the supervisory resilience mechanisms implemented by the Domain Coordinator Agent. While the preceding sections validated the proactive scaling behavior of the MRA–PFA–RSA operational pipeline, this section addresses the fourth research objective established in Section 4.2: whether the decentralized multi-agent architecture can effectively detect agent failures, initiate autonomous recovery, and escalate unresolvable conditions to the human operator without interrupting the operational supervision pipeline.

Failure scenarios were injected directly into the Kubernetes environment through controlled deployment manipulations and MQTT signal injections. The resulting coordination decisions were observed through DCA execution logs, Domain 4 audit records, Domain 5 persistence records, and Kubernetes deployment state transitions.

Metric	HPA Base-line	MAS	Interpretation
M1: Proactive Lead Time	0 s	+281 s	MAS initiated scaling actions before workload escalation and before reactive threshold crossing.
M2: Response Latency	18 s	44 s	MAS introduces additional forecasting and decision-making overhead but gains substantial proactive lead time.
M3: CRITICAL Pressure Duration	0 s	0 s	Neither system experienced CRITICAL pressure during the organic escalation phase.
M4: WARNING-to-Scale Delay	N/A	44 s	MAS converted WARNING-level pressure into a preventive scaling action.
M5: Replica-Seconds Overhead	4530	3510	MAS consumed approximately 22.5% fewer replica-seconds than the HPA baseline.
M6: Forecast RMSE	N/A	291.84 mCPU	Forecasts were sufficiently accurate to support operationally correct breach predictions.

Table 4.3: Comparative Evaluation Results

4.6.1 MRA Failure Detection and Recovery (Mode B)

The first resilience experiment evaluated the DCA’s Mode B recovery workflow using a failure of the MRA. The objective was to verify dual-layer failure confirmation, autonomous recovery initiation, AGENT_DOWN propagation, and supervisory escalation following recovery timeout. The failure was injected by scaling the `mra-stress-ng` deployment to zero replicas, terminating the MRA pod and suspending publication of monitoring beliefs.

Following the scale command, the DCA continued to receive a small number of belief updates that had already been buffered by the MQTT broker. The last belief update was received at 18:26:45 UTC. After that, there were no further publication streams. Starting from this point, the L1 liveness monitor within the DCA began accumulating expired heartbeat intervals.

At 18:28:30 UTC (about 105 seconds after the last published event), the L1 timeout threshold was exceeded and a `HEARTBEAT_MISSING` event was recorded. Consistent with the coordination policy, the event was classified as Mode A, and no recovery action was initiated because the missing heartbeat alone was insufficient to confirm a failure.

Twelve seconds later, at 18:28:42 UTC, the RSA peer-monitoring mechanism published

an `MRA_DOWN_SUSPECTED` signal, providing the first Layer-2 (L2) indication of failure. The DCA then combined the expired L1 heartbeat state with the received L2 evidence and confirmed the agent failure. This triggered the transition to Mode B coordination. The `AGENT_DOWN` notification was broadcast immediately, and a Kubernetes restart operation for the `mra-stress-ng` deployment was issued two seconds later. Figure 4.26 shows the DCA execution log capturing this sequence for Simulation 1.

```
18:22:15 - BELIEF_UPDATE_RECEIVED -
18:26:45 - BELIEF_UPDATE_RECEIVED
18:28:30 - MRA_HEARTBEAT_MISSING Mode A
18:28:42 - MRA_DOWN_SUSPECTED_FROM_RSA Mode A
18:28:45 - agent_failure_confirmed Mode B
18:28:45 - agent_down_broadcast
18:28:45 - coordination_decision Mode B MRA_LIVENESS_FAILURE
18:28:47 - agent_restart_triggered k8s_deployment=mra-stress-ng
18:28:47 - restart_triggered_awaiting_heartbeat timeout_s=60
```

Figure 4.26: Simulation 1: DCA Recovery Log.

The restarted MRA instance did not publish a heartbeat signal within the configured recovery window of sixty seconds. Consequently, the DCA recovery monitor expired at 18:29:50 UTC and escalated the incident to the supervisory agent using the `MRA_RECOVERY_FAILED` signal. Before this escalation, the RSA peer-monitoring mechanism had upgraded its assessment from `MRA_DOWN_SUSPECTED` to `MRA_DOWN_CONFIRMED` at 18:29:42 UTC, providing additional evidence that the failure persisted despite the restart attempt.

Although the DCA received this stronger confirmation signal, no additional recovery action was initiated. Instead, the DCA properly applied the one-attempt only recovery policy defined in the coordination architecture. This operational mode is significant since multiple restart attempts could mask continuing failures and delay operator notification of failure conditions in the system. By escalating the incident to the SA after only one failed recovery attempt, the coordination layer maintained fault visibility while at the same time avoiding uncontrolled restart cycles.

The simulation also provided indirect evidence of the independence of the hot-path property for the MAS architecture. During the shutdown transient period, buffered belief updates that were already in the MQTT broker communicated downstream through the communication pipeline normally. This observation confirms that coordination-layer recovery activities do not directly interfere with the message-processing path and that agent interactions remain decoupled from DCA intervention mechanisms. Figure 4.27 shows the DCA log entries corresponding to the recovery timeout and escalation decision.

```
18:29:42 - MRA_DOWN_CONFIRMED_L2 Mode B -
18:29:50 - recovery_timed_out_escalating
18:29:50 - sa_notification_sent signal_type=MRA_RECOVERY_FAILED
```

Figure 4.27: Simulation 1: Timeout and Escalation.

4.6.2 RSA Failure Detection and Recovery (Mode B*)

The second simulation evaluated the Mode B* coordination path, which is reserved exclusively for RSA liveness failures. Unlike Mode B, where operator escalation is postponed until the recovery window expires, Mode B* performs an immediate restart attempt and operator notification at the same time. This distinction reflects the critical role of the RSA within the MAS architecture: as the only component authorized to modify workload replica counts, an RSA failure removes the system’s ability to perform proactive, reactive, and emergency scaling actions. Consequently, any delay in notifying the SA would create an unacceptable operational risk.

Because the RSA has no downstream consumers capable of providing additional evidence, the DCA relies exclusively on Layer-1 (L1) heartbeat supervision. The RSA publishes heartbeats every fifteen seconds, and failure is confirmed after three consecutive missed intervals. The simulation was initiated by scaling the `rsa-stress-ng` deployment to zero replicas.

At 18:50:13.854 UTC, DCA confirmed RSA had failed and entered Mode B*. `AGENT_DOWN` notification was broadcast 9ms later, followed by the Kubernetes restart operation and an SA notification carrying the signal type `RSA_LIVENESS_FAILURE`. Both actions took less than 400ms from the failure being confirmed. This behavior differs fundamentally from Mode B, where operator notification is intentionally delayed while autonomous recovery is attempted. Figure 4.28 shows the DCA execution log for Simulation 2 capturing these events.

```
18:50:13.854 - agent_failure_confirmed Mode B* -
18:50:13.863 - agent_down_broadcast
18:50:13.923 - coordination_decision Mode B* RSA_LIVENESS_FAILURE
18:50:14.214 - agent_restart_triggered k8s_deployment=rsa-stress-ng
18:50:14.234 - sa_notification_sent signal_type=RSA_LIVENESS_FAILURE
18:50:29 - BELIEF_UPDATE_RECEIVED
18:50:59 - BELIEF_UPDATE_RECEIVED
```

Figure 4.28: Simulation 2: DCA Recovery Log.

During the 60-second recovery window, belief updates from the MRA continued to be observed, which means operational monitoring pipeline remains active even though there is no scaling actuator.

4.6.3 Immediate Escalation of Unresolvable Conditions (Mode C)

The third simulation evaluated the Mode C coordination path, which is reserved for conditions that exceed the DCA’s autonomous recovery authority and require immediate operator intervention. Unlike Modes B and B*, Mode C does not attempt agent recovery,

initiate restart procedures, or enter a recovery observation window. Its sole responsibility is the rapid escalation of operational conditions that cannot be resolved automatically.

Three Mode C signal types were validated: **CAPACITY_OVERFLOW**, **EMERGENCY_INSUFFICIENT**, and **CONFLICTING_AUTOSCALER**. Each signal was injected through the MQTT broker using RSA credentials, reproducing the conditions under which these events would be generated during normal system operation.

The first validation scenario injected a **CAPACITY_OVERFLOW** signal indicating that the RSA sizing pipeline had calculated a required replica count of eight, exceeding the configured maximum of five replicas. The DCA immediately classified the event as Mode C and forwarded a notification to the SA without initiating any recovery workflow. No restart operation or recovery timer was observed, confirming that the event was handled exclusively through escalation. The notification was issued within the same millisecond as the classification decision, demonstrating that the escalation path introduces effectively no operational delay. Figure 4.29 shows the DCA log entry confirming the classification and immediate SA notification.

```
18:52:18.063 - coordination_decision Mode C CAPACITY_OVERFLOW agent_type=RSA
18:52:18.063 - sa_notification_sent signal_type=CAPACITY_OVERFLOW deployment=stress-ng
```

Figure 4.29: **CAPACITY_OVERFLOW** Escalation.

A second validation scenario focused on the handling of **EMERGENCY_INSUFFICIENT** signals. Two identical events were injected five seconds apart to verify that this signal type bypasses the standard 300-second notification deduplication mechanism. Both events generated independent SA notifications, confirming that repeated escalation is preserved for persistent infrastructure-capacity failures. This behavior is intentional, as suppressing repeated notifications during an active resource exhaustion condition could delay operator awareness of a continuing service-impacting event. Figure 4.30 shows the two consecutive SA notifications produced by this test.

```
22:38:51.022 - coordination_decision Mode C EMERGENCY_INSUFFICIENT
22:38:51.022 - sa_notification_sent signal_type=EMERGENCY_INSUFFICIENT
```

Figure 4.30: **EMERGENCY_INSUFFICIENT** Escalation.

Collectively, these results confirm that the DCA correctly implements the Mode C coordination policy, providing immediate escalation for conditions that require human intervention while avoiding unnecessary recovery actions or notification suppression.

4.6.4 **EMERGENCY_STOP** Persistence and Recovery

The fourth simulation evaluated the **EMERGENCY_STOP** command relay mechanism, which represents the highest-priority operator intervention capability within the MAS. When an

EMERGENCY_STOP command is issued by the Supervisory Agent, the DCA is responsible for three coordinated actions: persisting the system state in Domain 5, forwarding the command to all active RSA instances, and ensuring that the emergency state remains enforceable across DCA restarts. This mechanism provides a means for operators to suspend autonomous scaling behavior while preserving a consistent system-wide operational state.

The simulation was initiated by publishing an **EMERGENCY_STOP** command through the MQTT broker using SA credentials. Upon receipt of the command, the DCA updated the system mode, persisted the new state within Domain 5, and propagated the stop instruction to all RSA instances. These actions were completed within an 11-millisecond interval, demonstrating that the command relay and persistence workflow introduces negligible operational overhead. Figure 4.31 shows the DCA log entries capturing the **EMERGENCY_STOP** receipt, persistence, and RSA relay.

```
22:16:52.920 - system_mode_changed from_mode=NORMAL to_mode=EMERGENCY_STOP
22:16:52.920 - system_mode_persisted mode=EMERGENCY_STOP
22:16:52.928 - emergency_stop_forwarded deployment=stress-ng
22:16:52.931 - coordination_decision Mode A EMERGENCY_STOP_ACTIVATED
22:16:52.931 - emergency_stop_activated_all_rsa_notified
```

Figure 4.31: **EMERGENCY_STOP** Processing.

To confirm durability of the state store, the Domain 5 coordination database was consulted immediately after execution of the command. The corresponding DCA entry contained the status value **MODE_STOP**, confirming that the emergency state had been successfully persisted and would remain available following a DCA restart. This confirms that DCA persistence of emergency operating mode is not only volatile storage but rather it is preserved within the coordination layer’s persistent state. Figure 4.32 shows the corresponding Domain 5 database entry confirming this state.

```
DCA:worker | MODE:EMERGENCY_STOP
```

Figure 4.32: **EMERGENCY_STOP** Persistence.

A subsequent **RESUME** command was then issued to restore normal autonomous operation. The DCA cleared the persisted emergency state and broadcast the corresponding resume signal to all RSA instances. Verification of the Domain 5 records confirmed that the **MODE_STOP** status had been removed, indicating successful restoration of the normal operating mode.

This simulation confirms that the DCA correctly implements emergency-state persistence, system-wide command propagation, and recovery from operator-imposed suspension, ensuring that critical supervisory commands remain effective even in the presence of coordination-layer restarts.

4.6.5 Additional Resilience Observations

PFA Recovery Validation

The fifth simulation evaluated the Mode B recovery workflow using a PFA liveness failure scenario. Unlike the previous simulations, this event was not manually injected. Instead, it emerged organically during the evaluation session when the Predictive Forecasting Agent (PFA) remained inactive for an extended period because workload conditions never exceeded the threshold required to trigger forecast generation. The absence of forecast publications was detected by the RSA peer-monitoring mechanism, which issued successive `PFA_DOWN_SUSPECTED` and `PFA_DOWN_CONFIRMED` signals. These notifications provided Layer-2 evidence to the DCA, initiating the coordination process.

Upon receiving the `PFA_DOWN_CONFIRMED` signal at 22:40:16 UTC, the DCA entered Mode B and confirmed the failure through the `L2_DIRECT` policy without requiring Layer-1 heartbeat evidence. A restart operation for the PFA pod was issued one second later, initiating the autonomous recovery procedure.

The restarted instance did not publish a heartbeat within the configured recovery observation window, resulting in an escalation notification being sent to the Supervisory Agent (SA) at 22:43:10 UTC. However, the PFA subsequently completed its startup sequence and published its first heartbeat at 22:44:04 UTC, corresponding to 107 seconds after the restart operation. This behavior demonstrates that the recovery mechanism itself functioned correctly and that the agent was ultimately restored without manual intervention, although the startup duration exceeded the timeout threshold configured for the evaluation environment.

This simulation validates the complete Mode B recovery path for peer-detected failures, including L2-based confirmation, autonomous restart initiation, timeout escalation, and eventual service restoration. It also highlights an operational consideration for future deployments: supervision timeout values should be calibrated against realistic startup times to reduce unnecessary escalation events during successful but slow recoveries. Figure 4.33 shows the DCA execution log tracing this sequence for Simulation 5.

```
22:37:16 - PFA_DOWN_SUSPECTED_FROM_RSA Mode A
22:40:16 - PFA_DOWN_CONFIRMED_L2 Mode B
22:40:17 - agent_restart_triggered k8s_deployment=pfa-stress-ng
22:40:17 - restart_triggered_awaiting_heartbeat timeout_s=60
22:43:10 - sa_notification_sent RSA_RECOVERY_FAILED
22:44:04 - pfa_recovered deployment=stress-ng
```

Figure 4.33: Simulation 5: PFA Recovery.

Hot-Path Independence

The DCA design has a key architectural requirement to maintain hot-path independence between the coordination layer and the operational scaling pipeline. All workflow elements

(MRA, PFA, and RSA) must remain functional regardless of DCA activity, allowing monitoring, forecasting, and scaling operations to continue without interruption during failure detection and recovery actions, including escalation and mode transitions.

This property was validated throughout the evaluation by continuously monitoring and recording MRA belief updates at the expected 30-second interval across all scenarios, without interruption attributable to DCA coordination activities. The evidence is particularly strong during the RSA failure experiment (Mode B*), where belief updates continued to be observed at 18:50:29 and 18:50:59 while the DCA was actively executing failure recovery procedures for the RSA.

The continuous receipt of MRA belief updates during failure recovery demonstrates that the DCA’s coordination logic operates independently of the operational data path. Accordingly, Agent Failure Handling, AGENT_DOWN broadcasts, Restart Procedures, and Escalation Workflows did not disrupt the flow of monitoring data through the MAS pipeline, along with the architectural isolation between Coordination Functions and Runtime Autoscaling Operations.

Knowledge Base Traceability and Persistence

The interaction between the DCA and the knowledge base was verified throughout the evaluation.

- Domain 4 audit records were successfully generated for all coordination decisions, providing a complete and machine-readable execution history.
- Domain 5 correctly maintained heartbeat records and persistent system-mode information, including the storage and subsequent removal of the EMERGENCY_STOP state.

Together, these observations confirm that the knowledge base provides both operational traceability and persistent coordination state management.

4.6.6 Evaluation Summary

The DCA resilience evaluation confirms the fourth research objective established in Section 4.2. The coordination layer correctly implements all four handling modes, enforces the one-attempt recovery rule, maintains the EMERGENCY_STOP state across restarts, and preserves complete audit traceability in the Knowledge Base. The hot-path independence invariant was verified continuously throughout the session, confirming that DCA coordination activity, including pod restarts, AGENT_DOWN broadcasts, and EMERGENCY_STOP relay, produced no interruption to the MRA→PFA→RSA operational pipeline.

4.7 Computational Cost and Deployment Overhead

The proposed MAS introduces a continuous supervision layer that operates alongside managed workloads. Unlike conventional monitoring solutions that only collect metrics, the MAS performs telemetry analysis, forecasting, coordination, and scaling actions. These capabilities introduce additional computational overhead that must be considered when assessing deployment feasibility.

4.7.1 Agent Runtime Footprint

The operational and coordination agents of the Multi-Agent System, excluding the PFA, are lightweight and primarily driven by event frequency rather than continuous computation. Most agents wait for MQTT messages before executing coordination logic. The MRA runs PromQL queries every thirty seconds, with modest computational costs mainly from telemetry retrieval latency. The RSA performs lightweight arithmetic during scaling decisions, while the DCA and SA primarily manage coordination, liveness supervision, and policy administration. Their resource consumption is dominated by the Python runtime, MQTT communication, and small coordination data structures.

The MQTT broker maintains low communication overhead, and SQLite in Write-Ahead Logging (WAL) mode supports sufficient throughput for MAS operations. Consequently, the coordination and supervision components collectively impose only modest resource requirements relative to the functionality they provide.

4.7.2 Forecasting Overhead

The Predictive Forecasting Agent is the primary source of computational cost within the architecture. It executes TimesFM 2.5, a pretrained transformer model containing approximately 200 million parameters and requiring substantially greater CPU and memory resources than the other agents.

In the experimental environment, each PFA instance required 2 GiB of requested memory and 3 GiB of memory limits. Running on CPU-only infrastructure introduced noticeable inference overhead and startup delays associated with model loading and initialization. Although forecasting occurs less frequently than monitoring cycles, inference remains the dominant computational activity within the MAS.

Because the architecture deploys one PFA per supervised workload, forecasting overhead scales approximately linearly with the number of managed services. While feasible for selected high-value workloads, this approach may become resource-intensive in large-scale environments without shared inference infrastructure or more powerful hardware.

4.7.3 Cost-Benefit Interpretation

The evaluation of the proposed architecture’s computational cost distinguishes between the overhead from coordination and supervision agents and that from the forecasting layer. Non-forecasting components of the MAS impose modest runtime overhead, characterized by an event-driven execution model that limits CPU utilization and maintains low memory requirements. These components offer functionalities comparable to those found in Kubernetes-native management frameworks. In contrast, the forecasting layer, particularly the PFA, introduces significant memory overhead, startup latency, and CPU utilization, especially as the number of supervised deployments increases due to independent forecasting runtimes for each instance.

The principal benefit obtained in return is proactive resource management. As demonstrated in Chapter 4, forecast-driven scaling enables the system to anticipate future resource saturation and initiate scaling actions before CRITICAL conditions occur. Whether this advantage justifies the additional computational overhead depends on the operational context. For latency-sensitive or business-critical workloads, earlier scaling intervention may justify the forecasting overhead. For stable workloads with predictable demand, traditional reactive autoscaling may remain the more cost-effective option.

Overall, the architecture demonstrates that foundation-model-based forecasting can be integrated into a Kubernetes-native MAS to support proactive scaling. However, this capability comes at a measurable computational cost, representing a deliberate trade-off between predictive functionality and resource efficiency.

4.8 Conclusion

This chapter evaluated the proposed MAS architecture across two complementary dimensions: proactive scaling effectiveness and supervisory resilience. The comparative evaluation demonstrated that the MAS exhibited proactive scaling behavior, initiating scaling actions before resource saturation occurred while maintaining efficient resource utilization relative to the reactive HPA baseline. Forecast accuracy improved as the sliding window expanded, and breach classifications remained operationally correct throughout the experiments, validating the use of TimesFM 2.5 in zero-shot mode. The DCA resilience scenarios confirmed that all supervisory handling modes (B, B*, and C) behaved as specified, with hot-path independence preserved across every injected failure condition. Together, these results provide experimental evidence that forecast-driven, multi-agent coordination can transition Kubernetes resource management from reactive operation toward a proactive and anticipatory model within a realistic deployment environment.

General Conclusion

General Conclusion

Our final-year project, entitled "From Reactive Supervision to Proactive Prediction: Multi-Agent Architecture for Anticipating IT Resource Needs", aimed to investigate whether a distributed and forecast-driven supervision architecture could transform Kubernetes resource management from a purely reactive process into a proactive and anticipatory one. To address this objective, we proposed a hierarchical Multi-Agent System composed of five specialized agents that communicate asynchronously via MQTT and share operational knowledge through a structured five-domain Knowledge Base. The five agents that constitute the Multi-Agent System are the Monitor and Resource Agent (MRA), Predictive Forecasting Agent (PFA), Resource Sizing Agent (RSA), Domain Coordinator Agent (DCA), and Supervision Agent (SA).

The thesis first presented the theoretical foundations of cloud-native resource management and highlighted the limitations of traditional reactive supervision approaches. It then introduced the conceptual design of the proposed architecture, defining the responsibilities, interactions, and coordination mechanisms of each agent within a BDI-inspired framework. After this, the proposed architecture was built and tested in a Kubernetes environment hosted on k3d by integrating the forecasting, monitoring, messaging, persistence, and supervisory functions into one prototype. Finally, a comparative experimental evaluation was performed using a native Kubernetes Horizontal Pod Autoscaler (HPA), complemented by a resilience assessment of the DCA coordination layer under injected failure conditions.

The experimental results demonstrated that the proposed architecture successfully achieved proactive scaling behavior. Compared with the reactive HPA baseline, the MAS initiated scaling actions approximately 281 seconds before the reactive controller reached its activation threshold, while reducing resource consumption by approximately 22.5% in terms of replica-seconds throughout the experiment. Forecasting performed by the TimesFM 2.5 model in zero-shot mode produced an average RMSE of 291.84 CPU millicores, which proved sufficient to correctly identify approaching threshold breaches and support forecast-driven scaling decisions. The resilience evaluation further confirmed that the DCA's coordination mechanisms behaved according to specification, with Modes A, B, B*, and C operating correctly during failure and recovery scenarios while preserving the independence of the MRA→PFA→RSA operational pipeline.

Taken together, these results indicate that a forecast-driven multi-agent architecture

can successfully anticipate future resource pressure conditions and initiate corrective actions before reactive threshold violations occur. Consequently, the central research objective of this thesis is validated: predictive supervision based on distributed autonomous agents constitutes a feasible approach for transitioning Kubernetes resource management from reactive response toward proactive resource anticipation.

This work contributes a complete forecast-driven supervisory architecture that combines predictive analytics, distributed agent coordination, policy-driven orchestration, and resilience-oriented supervision within a Kubernetes environment. Beyond demonstrating the feasibility of proactive scaling, the proposed architecture illustrates how forecasting and autonomous coordination mechanisms can be integrated into cloud-native management systems to support more anticipatory operational behavior.

The current prototype also exposes several limitations that create opportunities for future research. Primarily the high computational cost of forecasting due to a large transformer-based model, which increases deployment costs and limits scalability. Additionally, the current implementation was evaluated within a controlled single-host Kubernetes environment and under a specific workload profile, leaving additional questions regarding large-scale deployment and heterogeneous production workloads. Furthermore, the results reported are from a representative run of each configuration under a strictly identical load protocol. An assessment of inter-run variability has not been carried out and constitutes a natural extension of this work; however, the discrepancies observed between the two architectures (in the order of several tens of seconds and 20% of capacity) are of such a magnitude that they cannot be attributed solely to measurement noise.

Future Perspectives

While the proposed multi-agent system constitutes a complete and functional architecture for proactive Kubernetes resource management, several directions remain open for future investigation. The following perspectives identify the most promising extensions that would strengthen the system’s forecasting accuracy, adaptability, coordination intelligence, and fault resilience.

- **Adoption of a multivariate forecasting model:** Replacing or extending the univariate model with a multivariate time-series forecasting framework, such as a vector autoregressive transformer or a multivariate foundational model, would enable the PFA to exploit inter-metric correlations. This would produce more precise and consistent forecasting trajectories while simultaneously reducing the current conservatism imposed by independent metric-specific sizing.
- **Per-deployment continuous fine-tuning of the forecasting model:** introducing an online or periodic fine-tuning loop in which the PFA incrementally updates

the model's parameters using the forecast accuracy records it writes to the Knowledge Base at each inference cycle would allow the model to progressively adapt to per-deployment demand patterns, reducing structural uncertainty scores for well-characterized workloads and improving the precision of time-to-breach predictions as operational history accumulates.

- **Extended temporal context through multi-resolution observational windows:** Replacing the single fixed window with a multi-resolution context representation would allow the PFA to capture cyclical demand patterns, such as hourly batch jobs, daily traffic peaks, or weekly periodic workloads. Several techniques exist in this context. One approach maintains parallel windows at different time scales, with a short-term window of the current 48 minutes, a medium-term window of approximately six hours, and a long-term window spanning 24 to 72 hours, and they are fed into the forecasting model as a structured multi-scale input. An alternative approach replaces the raw long-term data with compressed statistical summaries, such as hourly averages, daily peak load estimates, inter-day variance trends, and moving seasonal baselines, thereby preserving long-range temporal context without proportionally increasing memory and inference cost.
- **Towards an agentic Domain Coordinator with autonomous resolution and cross-namespace cooperation :** Extending the DCA with a richer agentic reasoning capability would allow it to autonomously resolve a wider class of operational conditions, such as dynamically adjusting replica bounds in response to a capacity overflow event or resolving a conflicting autoscaler by autonomously disabling the conflicting controller rather than escalating the decision to a human operator. This capability could be further strengthened through peer coordination between DCAs, enabling adjacent agents to exchange domain state, advertise available capacity headroom, and cooperate in redistributing load across namespace boundaries, thereby moving the system toward a genuinely distributed and cooperative resource management fabric.

Bibliography

- [1] P. Mell and T. Grance, *The nist definition of cloud computing*, en, Sep. 2011. DOI: <https://doi.org/10.6028/NIST.SP.800-145>. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=909616.
- [2] S. Deng et al., “Cloud-native computing: A survey from the perspective of services,” *Proceedings of the IEEE*, vol. 112, no. 1, pp. 12–46, 2024. DOI: 10.1109/JPROC.2024.3353855. [Online]. Available: https://dsg.tuwien.ac.at/~sd/papers/Zeitschriftenartikel_2024_SD_Cloud-Native.pdf.
- [3] B. Mallampati, “Demystifying cloud-native microservices architecture for scalable applications,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, no. 1, pp. 1806–1817, 2025. DOI: 10.30574/wjaets.2025.15.1.0422. [Online]. Available: https://wjaets.com/sites/default/files/fulltext_pdf/WJAETS-2025-0422.pdf.
- [4] A. Lawson and J. Sica, “The infrastructure of ai’s future: Annual cloud native survey,” Cloud Native Computing Foundation (CNCF) and The Linux Foundation, Annual Survey Report, Jan. 2026, Foreword by Jonathan Bryce. [Online]. Available: https://www.cncf.io/wp-content/uploads/2026/01/CNCF_Annual_Survey_Report_final.pdf.
- [5] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, no. 239, Mar. 2014, Accessed: 2026. [Online]. Available: <https://www.seltzer.com/margo/teaching/CS508.19/papers/merkel14.pdf>.
- [6] Z. Zhong, M. Xu, M. A. Rodriguez, C. Xu, and R. Buyya, “Machine Learning-based Orchestration of Containers: A Taxonomy and Future Directions,” *ACM Computing Surveys*, vol. 54, no. 10s, pp. 217:1–217:35, Sep. 2022. DOI: 10.1145/3510415. [Online]. Available: <https://dl.acm.org/doi/10.1145/3510415>.
- [7] The Kubernetes Authors, *Kubernetes Official Documentation*, Kubernetes Documentation, Accessed: 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/>.

-
- [8] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis,” in *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC '12)*, Recipient of the ACM SoCC 2021 Test of Time Award., San Jose, CA, USA: ACM, 2012, 7:1–7:13. DOI: 10.1145/2391229.2391236. [Online]. Available: <https://dl.acm.org/doi/10.1145/2391229.2391236>.
- [9] T. Lorigo-Botran, J. Miguel-Alonso, and J. A. Lozano, “A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments,” *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, Dec. 2014. DOI: 10.1007/s10723-014-9314-7. [Online]. Available: <https://doi.org/10.1007/s10723-014-9314-7>.
- [10] C. Qu, R. N. Calheiros, and R. Buyya, “Auto-scaling Web Applications in Clouds: A Taxonomy and Survey,” *ACM Computing Surveys*, vol. 51, no. 4, 73:1–73:33, Jul. 2018. DOI: 10.1145/3148149. [Online]. Available: <https://doi.org/10.1145/3148149>.
- [11] S. Alharthi, A. Alshamsi, A. Alseiari, and A. Alwarafy, “Auto-scaling techniques in cloud computing: Issues and research directions,” *Sensors*, vol. 24, no. 17, 2024, ISSN: 1424-8220. DOI: 10.3390/s24175551. [Online]. Available: <https://www.mdpi.com/1424-8220/24/17/5551>.
- [12] M. B. Taha, Y. Sanjalawe, A. Al-Daraiseh, S. Fraihat, and S. R. Al-E’mari, “Proactive auto-scaling for service function chains in cloud computing based on deep learning,” *IEEE Access*, vol. 12, pp. 38 575–38 593, 2024. DOI: 10.1109/ACCESS.2024.3375772.
- [13] Z. Ahamed, M. Khemakhem, F. Eassa, F. Alsolami, and A. S. A.-M. Al-Ghamdi, “Technical study of deep learning in cloud computing for accurate workload prediction,” *Electronics*, vol. 12, no. 3, 2023, ISSN: 2079-9292. DOI: 10.3390/electronics12030650. [Online]. Available: <https://www.mdpi.com/2079-9292/12/3/650>.
- [14] D. Saxena and A. K. Singh, *Workload Forecasting and Resource Management Models Based on Machine Learning for Cloud Computing Environments*, Preprint. arXiv:2106.15112 [cs.DC]., 2021. arXiv: 2106.15112 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2106.15112>.
- [15] M. Masdari and A. Khoshnevis, “A Survey and Classification of the Workload Forecasting Methods in Cloud Computing,” *Cluster Computing*, vol. 23, no. 3, pp. 2399–2421, 2020. DOI: 10.1007/s10586-019-03010-3. [Online]. Available: <https://link.springer.com/article/10.1007/s10586-019-03010-3>.
- [16] M. Mao and M. Humphrey, “A Performance Study on the VM Startup Time in the Cloud,” *Proceedings of the IEEE Fifth International Conference on Cloud Computing*, pp. 423–430, 2012. DOI: 10.1109/CLOUD.2012.103. [Online]. Available: <https://doi.org/10.1109/CLOUD.2012.103>.

-
- [17] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003. DOI: 10.1109/MC.2003.1160055. [Online]. Available: <https://ieeexplore.ieee.org/document/1160055>.
 - [18] B. P. Sanwouo, C. Quinton, and P. Temple, “Breaking the Loop: AWARE is the New MAPE-K,” in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE ’25)*, Trondheim, Norway: ACM, 2025. DOI: 10.1145/3696630.3728512. [Online]. Available: <https://doi.org/10.1145/3696630.3728512>.
 - [19] O. Gheibi, D. Weyns, and F. Quin, “Applying Machine Learning in Self-Adaptive Systems: A Systematic Literature Review,” *ACM Transactions on Autonomous and Adaptive Systems*, vol. 15, no. 3, 9:1–9:37, Aug. 2021. DOI: 10.1145/3469440. [Online]. Available: <https://doi.org/10.1145/3469440>.
 - [20] D. Weyns, S. Malek, and J. Andersson, “On Decentralizing Self-Adaptation : lessons from the trenches and challenges for the future,” *Proceedings of the ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pp. 84–93, 2010. DOI: 10.1145/1808984.1808994. [Online]. Available: <https://doi.org/10.1145/1808984.1808994>.
 - [21] V. Podolskiy, A. Jindal, and M. Gerndt, “IaaS reactive autoscaling performance challenges,” in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, IEEE, 2018, pp. 954–957. DOI: 10.1109/CLOUD.2018.00144. [Online]. Available: <https://doi.org/10.1109/CLOUD.2018.00144>.
 - [22] M. Abdullah, W. Iqbal, J. L. Berral, J. Polo, and D. Carrera, “Burst-aware predictive autoscaling for containerized microservices,” *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1448–1460, 2022. DOI: 10.1109/TSC.2020.2995937. [Online]. Available: <https://doi.org/10.1109/TSC.2020.2995937>.
 - [23] M. Straesser, J. Grohmann, J. von Kistowski, S. Eismann, A. Bauer, and S. Kounev, “Why Is It Not Solved Yet? Challenges for Production-Ready Autoscaling,” in *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE ’22)*, Beijing, China: ACM, 2022, pp. 105–115. DOI: 10.1145/3489525.3511680. [Online]. Available: <https://doi.org/10.1145/3489525.3511680>.
 - [24] A. Ali-Eldin, J. Tordsson, and E. Elmroth, “An Adaptive Hybrid Elasticity Controller for Cloud Infrastructures,” in *2012 IEEE Network Operations and Management Symposium*, IEEE, 2012, pp. 204–212. DOI: 10.1109/NOMS.2012.6211900. [Online]. Available: <https://doi.org/10.1109/NOMS.2012.6211900>.

- [25] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms,” in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP ’17)*, Shanghai, China: ACM, 2017, pp. 153–167. DOI: 10.1145/3132747.3132772. [Online]. Available: <https://doi.org/10.1145/3132747.3132772>.
- [26] D. Saxena, J. Kumar, A. K. Singh, and S. Schmid, “Performance Analysis of Machine Learning Centered Workload Prediction Models for Cloud,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1313–1330, 2023. DOI: 10.1109/TPDS.2023.3240567. [Online]. Available: <https://doi.org/10.1109/TPDS.2023.3240567>.
- [27] K. Seshadri, K. Sindhu, S. N. Bhattu, and C. Kollengode, “Design and Evaluation of a Hierarchical Characterization and Adaptive Prediction Model for Cloud Workloads,” *IEEE Transactions on Cloud Computing*, 2024. DOI: 10.1109/TCC.2024.3393114. [Online]. Available: <https://doi.org/10.1109/TCC.2024.3393114>.
- [28] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, “Forecasting Cloud Application Workloads With CloudInsight for Predictive Resource Management,” *IEEE Transactions on Cloud Computing*, vol. 10, pp. 1848–1863, 2020. DOI: 10.1109/TCC.2020.2998017. [Online]. Available: <https://doi.org/10.1109/TCC.2020.2998017>.
- [29] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A Survey on Concept Drift Adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, Mar. 2014. DOI: 10.1145/2523813. [Online]. Available: <https://doi.org/10.1145/2523813>.
- [30] M. Straesser, S. Geissler, S. Lange, L. K. Schumann, T. Hossfeld, and S. Kounev, “Trust your local scaler: A continuous, decentralized approach to autoscaling,” *Performance Evaluation*, vol. 167, p. 102452, 2025, ISSN: 0166-5316. DOI: <https://doi.org/10.1016/j.peva.2024.102452>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0166531624000579>.
- [31] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, “Towards resource-efficient reactive and proactive auto-scaling for microservice architectures,” *Journal of Systems and Software*, vol. 225, p. 112390, 2025, ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2025.112390>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121225000585>.
- [32] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, *Resilient auto-scaling of microservice architectures with efficient resource management*, 2025. arXiv: 2506.05693 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2506.05693>.

-
- [33] N. M. Calcavecchia, B. A. Caprarescu, E. D. Nitto, D. J. Dubois, and D. Petcu, *Depas: A decentralized probabilistic algorithm for auto-scaling*, 2012. arXiv: 1202.2509 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/1202.2509>.
- [34] B. A. Caprarescu and D. Petcu, *Decentralized probabilistic auto-scaling for heterogeneous systems*, 2012. arXiv: 1203.3885 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/1203.3885>.
- [35] Z. Zhong et al., *A Survey of Time Series Anomaly Detection Methods in the AIOps Domain*, Preprint. arXiv:2308.00393 [cs.LG]., 2023. arXiv: 2308.00393 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2308.00393>.
- [36] P. Notaro, J. Cardoso, and M. Gerndt, “A survey of aiops methods for failure management,” *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 1–45, 2021. DOI: 10.1145/3483424. [Online]. Available: <https://doi.org/10.1145/3483424>.
- [37] Y. S. Patel and J. Bedi, “MAG-D: A Multivariate Attention Network Based Approach for Cloud Workload Forecasting,” *Future Generation Computer Systems*, vol. 142, pp. 376–392, May 2023. DOI: 10.1016/j.future.2023.01.002. [Online]. Available: <https://doi.org/10.1016/j.future.2023.01.002>.
- [38] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019. DOI: 10.1109/TKDE.2018.2876857. [Online]. Available: <https://doi.org/10.1109/TKDE.2018.2876857>.
- [39] E. Furman, A. Senderovich, S. Bergsma, and J. C. Beck, “Capacity allocation for clouds with parallel processing, batch arrivals, and heterogeneous service requirements,” *Manufacturing & Service Operations Management*, vol. 24, no. 3, pp. 1601–1617, 2022. DOI: 10.48550/arXiv.2209.08820. [Online]. Available: <https://doi.org/10.48550/arXiv.2209.08820>.
- [40] K. Rzađca et al., “Autopilot: Workload Autoscaling at Google,” in *Proceedings of the 15th European Conference on Computer Systems (EuroSys ’20)*, Heraklion, Greece: ACM, 2020, pp. 1–16. DOI: 10.1145/3342195.3387524. [Online]. Available: <https://dl.acm.org/doi/10.1145/3342195.3387524>.
- [41] M. Lin, A. Wierman, L. L. H. Andrew, and E. Thereska, “Dynamic Right-Sizing for Power-Proportional Data Centers,” *IEEE/ACM Transactions on Networking*, vol. 21, no. 5, pp. 1378–1391, Oct. 2013. DOI: 10.1109/TNET.2012.2226216. [Online]. Available: <https://doi.org/10.1109/TNET.2012.2226216>.

-
- [42] J. Chen and H. Li, “A Two-Phase Cloud Resource Provisioning Algorithm for Cost Optimization,” *Mathematical Problems in Engineering*, vol. 2020, pp. 1–18, 2020. DOI: 10.1155/2020/1310237. [Online]. Available: <https://doi.org/10.1155/2020/1310237>.
 - [43] S. Deochake, S. Maheshwari, R. De, and A. Grover, *Comparative study of virtual machines and containers for devops developers*, 2023. arXiv: 1808.08192 [cs.OS]. [Online]. Available: <https://arxiv.org/abs/1808.08192>.
 - [44] A. Dorri, S. S. Kanhere, and R. Jurdak, “Multi-Agent Systems: A Survey,” *IEEE Access*, vol. 6, pp. 28 573–28 593, 2018. DOI: 10.1109/ACCESS.2018.2831911. [Online]. Available: <https://ieeexplore.ieee.org/document/8352646>.
 - [45] M. Wooldridge and N. R. Jennings, “Intelligent Agents: Theory and Practice,” *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, Jun. 1995, Foundational reference. Cambridge University Press. Cited as the canonical agent definition and BDI architecture taxonomy. DOI: 10.1017/S0269888900008122. [Online]. Available: <https://doi.org/10.1017/S0269888900008122>.
 - [46] D. Weyns, S. Malek, and J. Andersson, “Forms: Unifying reference model for formal specification of distributed self-adaptive systems,” *ACM Transactions on Autonomous and Adaptive Systems*, vol. 7, no. 1, pp. 1–61, 2012. DOI: 10.1145/2168260.2168268. [Online]. Available: <https://doi.org/10.1145/2168260.2168268>.
 - [47] IBM, “An architectural blueprint for autonomic computing,” IBM Corporation, Technical Report, Jun. 2005.
 - [48] A. Bucchiarone, C. Guidi, I. Lanese, N. Bencomo, and J. Spillner, “A MAPE-K Approach to Autonomic Microservices,” in *ICSA-C 2022 - IEEE 19th International Conference on Software Architecture Companion*, Honolulu, United States: IEEE, Mar. 2022, pp. 100–103. DOI: 10.1109/ICSA-C54293.2022.00025. [Online]. Available: <https://inria.hal.science/hal-03916224>.
 - [49] M. Bratman, *Intention, Plans, and Practical Reason*. Cambridge: MA: Harvard University Press, 1987.
 - [50] A. S. Rao, M. P. Georgeff, et al., “Bdi agents: From theory to practice.,” in *Icmas*, vol. 95, 1995, pp. 312–319. [Online]. Available: https://cdn.aaai.org/ICMAS/1995/ICMAS95-042.pdf?utm_source=chatgpt.com.
 - [51] M. Georgeff and A. Rao, “Modeling rational agents within a bdi-architecture,” in *Proc. 2nd Int. Conf. on Knowledge Representation and Reasoning (KR’91)*. Morgan Kaufmann, of, 1991, pp. 473–484. [Online]. Available: https://books.google.dz/books?hl=en&lr=&id=1wwZAQAIAAJ&oi=fnd&pg=PA473&dq=Modeling+rational+agents+within+a+BDI-architecture&ots=VtewiJcrub&sig=XhwuzJmyYJmyc_

- EmHJ_gaa0tjfl&redir_esc=y#v=onepage&q=Modeling%20rational%20agents%20within%20a%20BDI-architecture&f=false.
- [52] F. Alzetta, P. Giorgini, M. Marinoni, and D. Calvaresi, “Rt-bdi: A real-time bdi model,” in *International Conference on Practical Applications of Agents and Multi-Agent Systems*, Springer, 2020, pp. 16–29. DOI: 10.1007/978-3-030-49778-1_2.
 - [53] A. Traldi, F. Bruschetti, M. Robol, D. Calvaresi, M. Roveri, and P. Giorgini, *Real-time bdi agents: A model and its implementation*, 2026. arXiv: 2205.00979. [Online]. Available: <https://arxiv.org/abs/2205.00979>.
 - [54] L. Buss Becker, I. de Oliveira Silvestre, J. F. Hübner, and M. Fisher, “An expedited bdi agent architecture: Improving the responsiveness of agent-based autonomous systems for handling critical situations,” *Robotics and Autonomous Systems*, vol. 186, p. 104917, 2025, ISSN: 0921-8890. DOI: <https://doi.org/10.1016/j.robot.2025.104917>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S092188902500003X>.
 - [55] D. Calvaresi, Y. Dicente Cid, M. Marinoni, A. F. Dragoni, A. Najjar, and M. Schumacher, “Real-time multi-agent systems: Rationality, formal model, and empirical results,” *Autonomous Agents and Multi-Agent Systems*, vol. 35, no. 12, 2021. DOI: 10.1007/s10458-020-09492-5. [Online]. Available: <https://doi.org/10.1007/s10458-020-09492-5>.
 - [56] TM Forum, Huawei Technologies Ltd. Co., China Mobile, and Orange, “Autonomous networks: Empowering digital transformation for the telecoms industry,” TM Forum, Tech. Rep., version 1.0, May 2019, Release 1.0, Published on 15 May 2019.
 - [57] TM Forum, “Autonomous network levels evaluation methodology: Introductory guide,” TM Forum, Tech. Rep. IG1252, version 1.2.0, Jun. 2023.
 - [58] FIPA, “FIPA 97 Specification Part 2: Agent Communication Language,” FIPA (Foundation for Intelligent Physical Agents), Tech. Rep., 1997. [Online]. Available: <https://web.archive.org/web/20220709192327/http://www.fipa.org/specs/fipa00018/OC00018A.html>.
 - [59] T. Finin, R. Fritzson, D. McKay, and R. McEntire, “Kqml as an agent communication language,” in *Proceedings of the Third International Conference on Information and Knowledge Management*, ser. CIKM '94, Gaithersburg, Maryland, USA: Association for Computing Machinery, 1994, pp. 456–463, ISBN: 0897916743. DOI: 10.1145/191246.191322. [Online]. Available: <https://doi.org/10.1145/191246.191322>.
 - [60] T. Finin et al., “Specification of the kqml agent-communication language,” 1993. [Online]. Available: https://ebiquity.umbc.edu/_file_directory_/papers/1484.pdf.

- [61] Q. Wen et al., “Transformers in time series: A survey,” *arXiv preprint arXiv:2202.07125*, 2023. DOI: <https://doi.org/10.48550/arXiv.2202.07125>.
- [62] J. Zhao, F. Chu, L. Xie, Y. Che, Y. Wu, and A. F. Burke, “A survey of transformer networks for time series forecasting,” *Computer Science Review*, vol. 60, p. 100883, 2026, ISSN: 1574-0137. DOI: <https://doi.org/10.1016/j.cosrev.2025.100883>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013725001595>.
- [63] S. Ahmed, A. Tripathi, R. P. Ramachandran, I. E. Nielsen, S. Siddiqui, and G. Rasool, “Transformers in time-series analysis: A tutorial,” *arXiv preprint arXiv:2205.01138*, 2023. DOI: <https://doi.org/10.48550/arXiv.2205.01138>.
- [64] The Kubernetes Authors, *Horizontal Pod Autoscaling*, Kubernetes Documentation, Accessed: 2026, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale>.